

Fifteen years of research on the Subgraph Isomorphism Problem

Vincenzo Bonnici

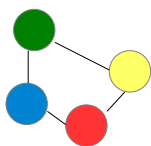
Department of Mathematical, Physical and Computer Sciences, University of Parma, Italy

Synbionics lab.

InfoLife lab. – Consorzio Interuniversitario Nazionale per l'Informatica

The subgraph isomorphism (subGI) problem

- A **Graph** is a tuple (V, E, α) where
- V is the set of vertices
 - $E : V \times V$ is the set edges (arcs)
 - $\alpha: V \rightarrow \Sigma$ a function which maps vertices to labels



Given a **pattern** graph $Q=(V,E,\alpha)$ and a **target** graph $R=(V',E', \alpha')$

Graph **isomorphism** problem:

- Find a function $M : V \rightarrow V'$ such that, for $u,v \in V$
- $\alpha(u) = \alpha'(M(u))$
- for each $(u,v) \in E$ than $(M(u), M(v)) \in E'$
- M is bijective

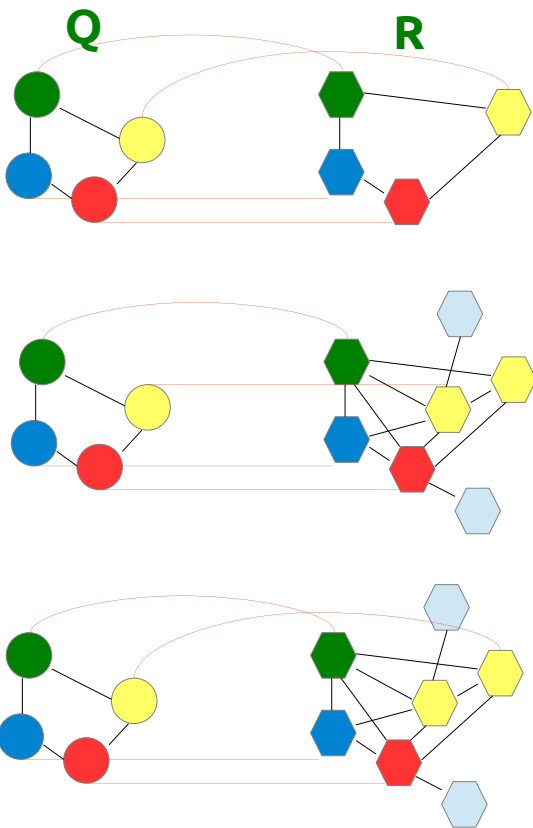
Subgraph isomorphism (monomorphism):

- M is injective

Induced subgraph isomorphism:

- M is injective
- for each $(u,v) \notin E$ than $(M(u), M(v)) \notin E'$

**NP-COMplete
(GENERAL CASE)**



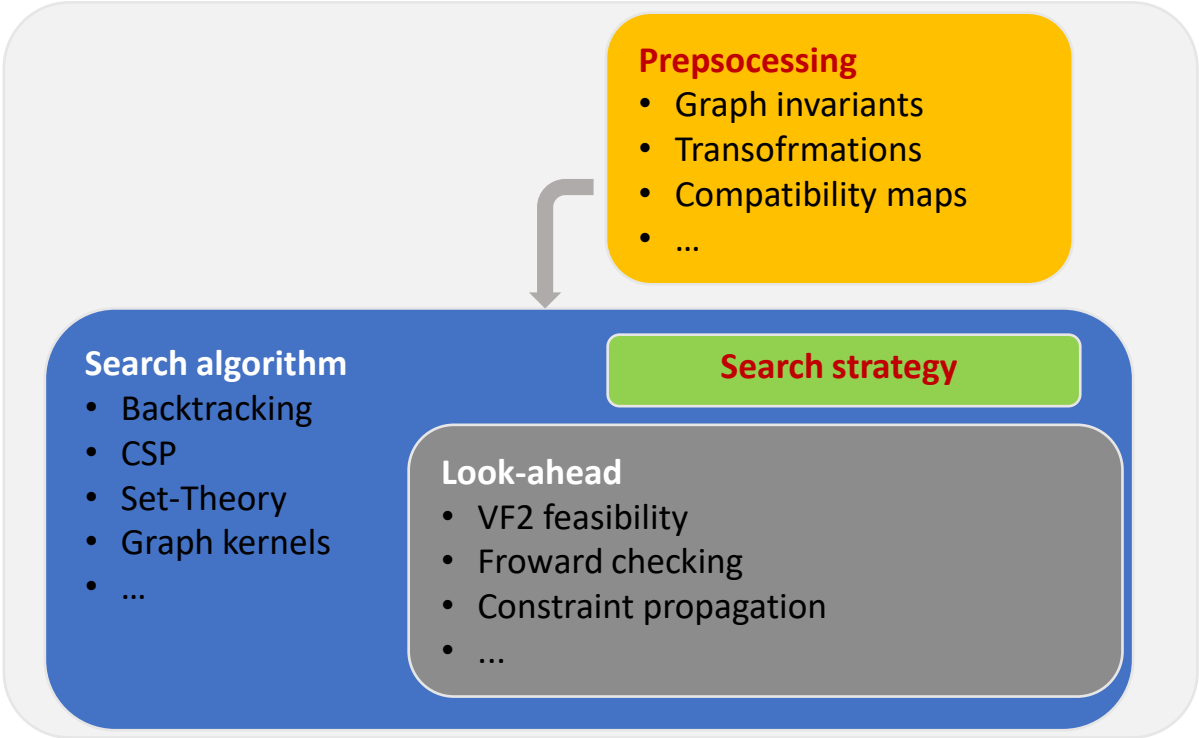
Applications

The subgraph isomorphism problem is widely implied in several scientific problems by

- **Testing:** there exists **at least one** occurrence of Q in R?
- **Counting:** **how many** occurrences of Q in R?
- **Listing/Enumerating:** **which are** those occurrences?

- Prediction of biological activity of chemical compounds
 - E. Gifford, M. Johnson, D. Smith, and C. Tsai. Structure-reactivity maps as a tool for visualizing xenobiotic structure-reactivity. *Network Science*, 2:1–33, 1996.
- Analysis of networks motifs (i.e. subgraphs) in complex networks
 - R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon. Network motifs: simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.
- Searching proteins complexes (evolutionary conserved modules) in PPI networks
 - Y. Tian, R. C. Mceachin, C. Santos, D. J. States, and J. M. Patel. Saga: a subgraph matching tool for biological graphs. *Bioinformatics*, 23(2):232–239, 2006.
- Detection of hardware trojans
 - Piccolboni, L., Menon, A., & Pravadelli, G. (2017). Efficient control-flow subgraph matching for detecting hardware Trojans in RTL models. *ACM Transactions on Embedded Computing Systems (TECS)*, 16(5s), 137.
- Detection of software design patterns for increasing reusability
 - DPVO: Design Pattern Detection Using Vertex Ordering a Case Study in JHotDraw with Documentation to Improve Reusability: First International Conference, CNC 2018, Gwalior, India, March 22-24, 2018
- Pattern recognition in computer vision
 - Bunke, H., & Messmer, B. T. (1995, September). Efficient attributed graph matching and its application to image analysis. In *International Conference on Image Analysis and Processing* (pp. 44-55). Springer, Berlin, Heidelberg.
- Querying systems of graph-based NoSQL databases
 - Micale, G., Di Maria, A., Grasso, R., Bonnici, V., Ferro, A., Shasha, D., Giugno, R. and Pulvirenti, A., 2025. *MultiGraphMatch: a subgraph matching algorithm for multigraphs*. *ACM Transactions on Knowledge Discovery from Data*, 19(5), pp.1-36.
- And so forth
 - Foggia, P., Percannella, G., & Vento, M. (2014). Graph matching and learning in pattern recognition in the last 10 years. *International Journal of Pattern Recognition and Artificial Intelligence*, 28(01), 1450001.

Approaches



The search-space tree

Backtracking Algorithms

.Approach

- .DepthFirstVisit of the **tree search space**
- .**Forward** extend a partial solution
- .**Backtrack** if no further extension is possible, then return to a previous state

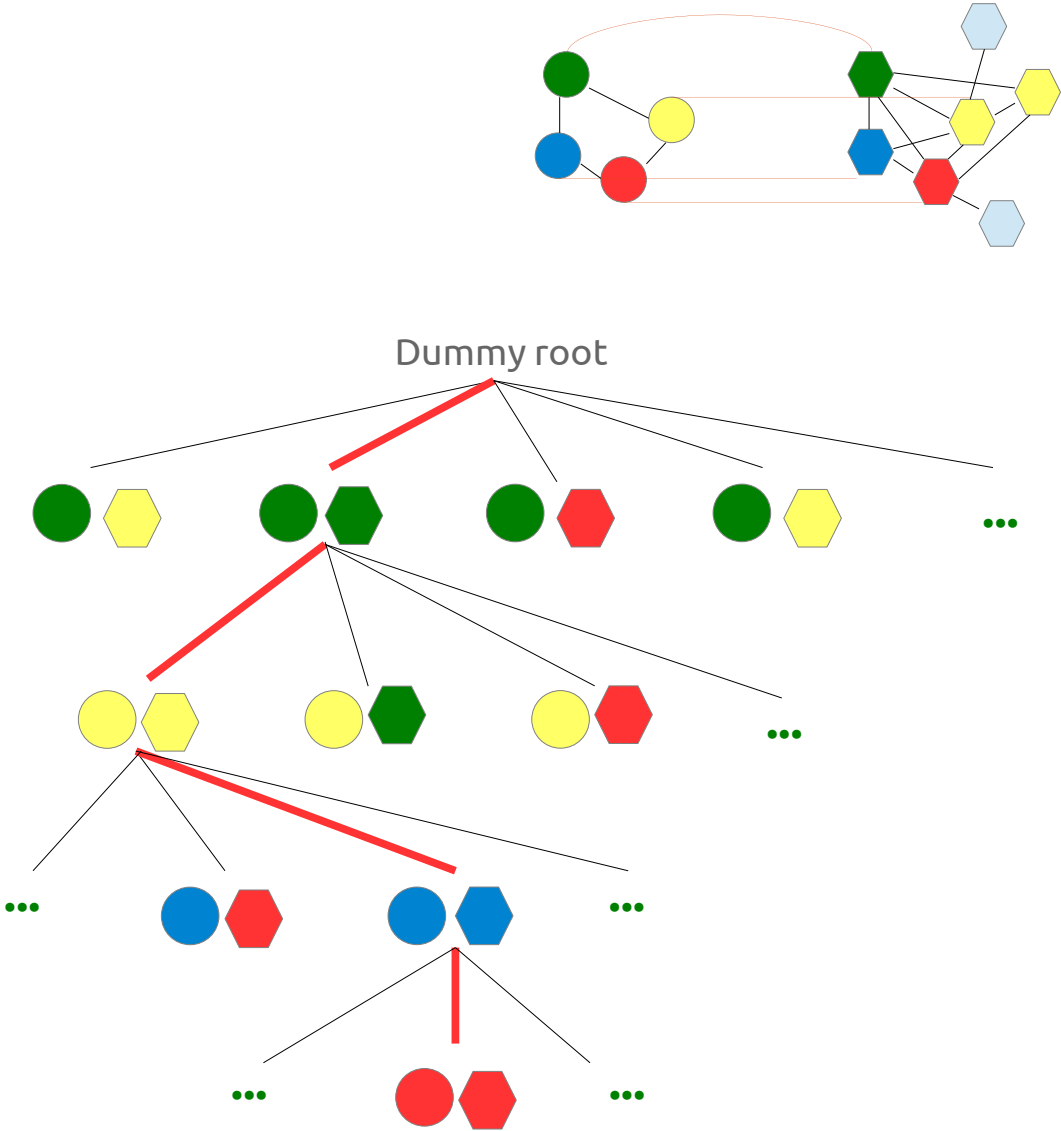
.Pruning\look-ahead

- .VF2 feasibility rules
- .Forward checking
- .Constraint propagation
(ConstraintSatisfactionProblem based)
-

.Backtrack \ back jumping

.Main issue \ goal

- .Prune dead branches (that do not lead to any valid solution) as soon as possible

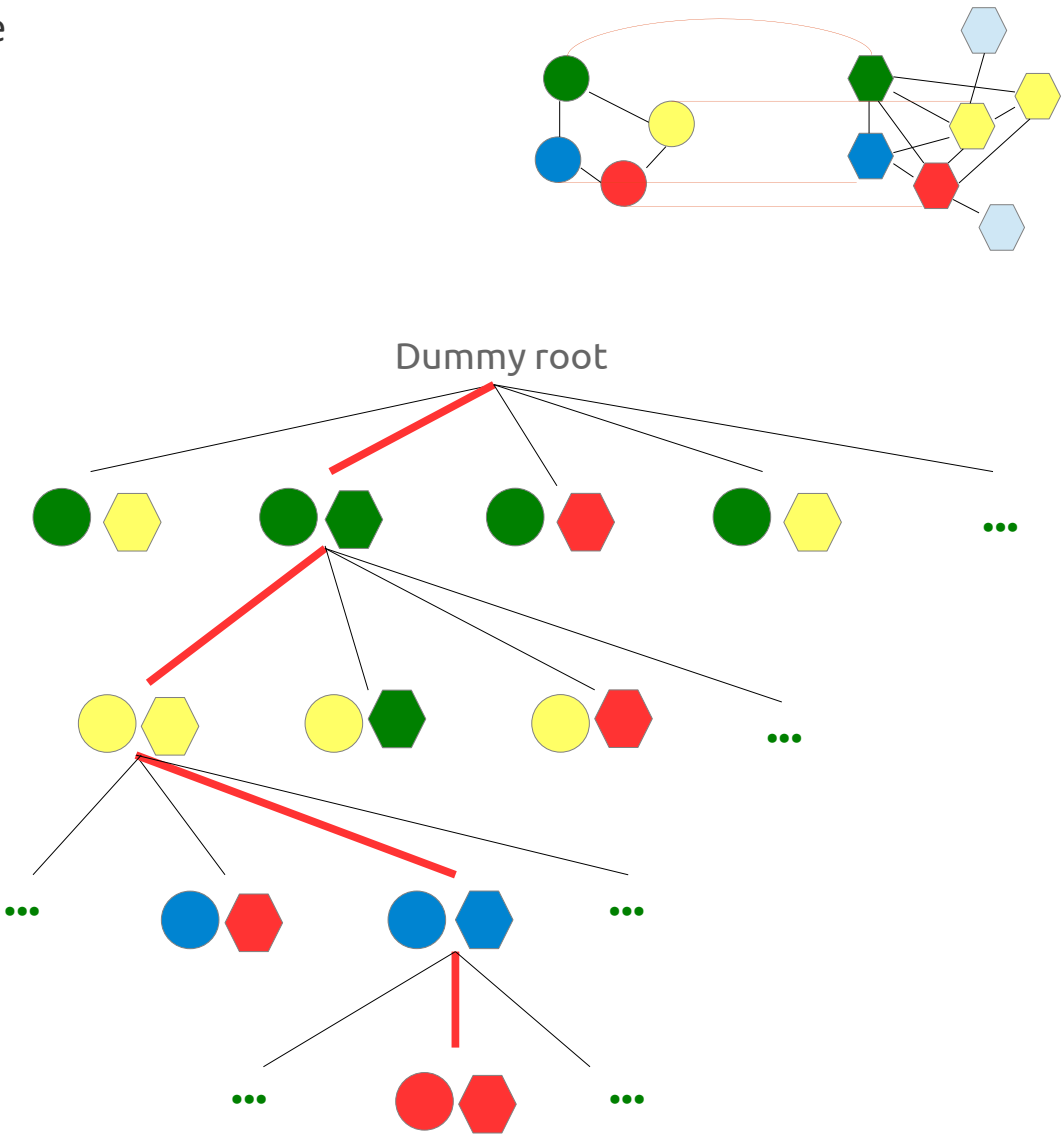


The search-space tree

- Each branch, from the root to an internal node of the tree, is a **partial solution**
- Each branch, from the root to a leaf, is a **global solution** (isomorphism)

- It's **not unique**
- It changes according to the order in which pattern and target vertices are examined

Search strategy



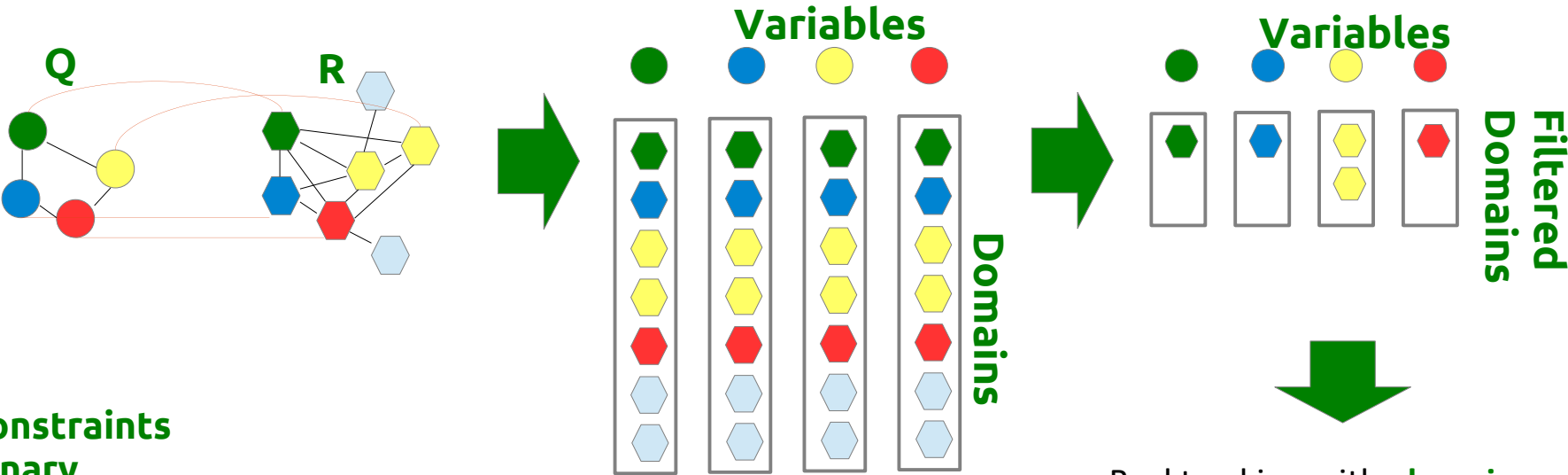
Constraint satisfaction problem (CSP)

Variables represent pattern vertices

Values represent target vertices

The set of candidate values of a variable is called **domain**

An **assignment** of a value to a variable is a **mapping** between a pattern and a target vertex



Constraints

.Unary

- .Label compatibility

.M's injectivity

(inside a partial solution, a target vertex cannot be mapped to more than one pattern vertex)

.Binary

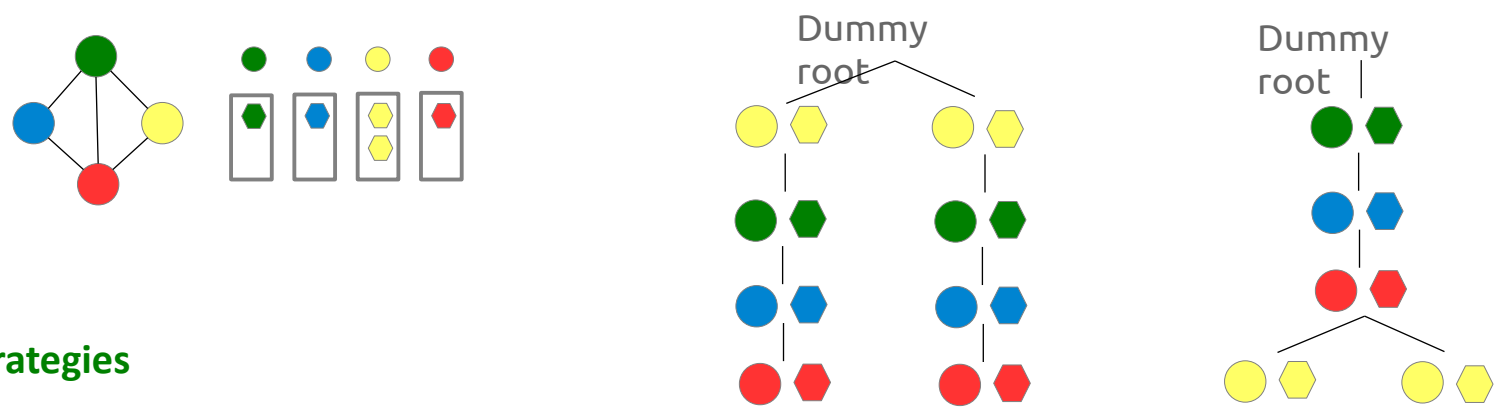
- .Topological constraints (edges \ arcs)

Backtracking with **domain reduction procedures**

- . After a value assignment, **propagate constraints** to unassigned value domains
- . If at least one **domain becomes empty**, then backtrack

Search strategies and variable (vertex) ordering

The order in which pattern vertices (variables) are examined has a strong impact on the SubGI search: it reduces the set of possible forward assignments of the variables by the fail first principle.



Search Strategies

- .Static**
 - The order is **fixed** before the backtracking phase
 - Each level** of the tree represents mappings between a **specific pattern vertex** and the target candidates
- .Dynamic**
 - The order is **chosen during the backtracking** phase
 - A search tree's level may represent possible mappings between different pattern and target vertices
 - The first level of a sub-tree still regards a single pattern vertex
- .Pattern dependent:** it depends on some properties of the pattern graph (centrality, betweenness,...)
- .Target dependent:** it depends on some properties of the target graph (label frequency, neighborhood,...)
- .State dependent:** it depends on the partial solution (candidate sets,...)

The RI project

Bonnici, V., Giugno, R., Pulvirenti, A., Shasha, D., & Ferro, A. (2013).

A subgraph isomorphism algorithm and its application to biochemical data.

BMC bioinformatics, 14(Suppl 7), S13.

Contribution

.RI

new search strategy for backtracking approach on solving exact subgraph listing

.RI-DS

domain based version

.Biological\biochemical datasets

molecules, proteins, proteins backbones, contact maps, protein-protein interactions

.Application examples

complexes search in biological networks

.Comparisons

VF2, LAD, FocusSearch

The RI project

RI

- .**Generic** backtracking algorithm
- .For graph isomorphism, subgraph isomorphism and induced subgraph isomorphism
- .With a special focus on biological \ biochemical graphs
- .**No** preprocessing
- .**No** look-ahead
- .**Memory** safe
- .Two main **ideas** (ordering)
- .New **search strategy** (variable ordering)
- .The order whit which pattern vertices (variables) are taken into account
- .Constraint **verification ordering**
- .The order whit which constraints are verified
- .Data structure
 - .Adjacency list
 - .Bit vectors

The RI project

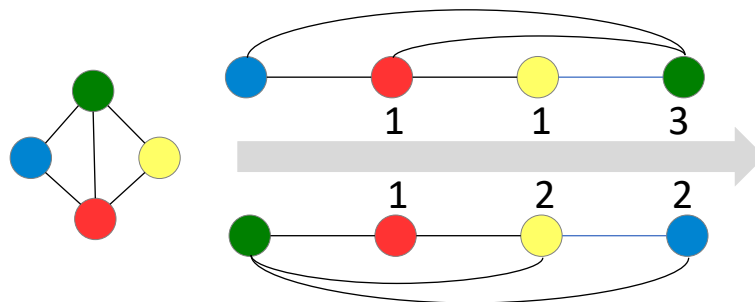
RI – Search strategy

.Aim

- .Find a good search strategy such that it consists in a good trade-off between
- .The cost to extend a partial solution (constraint verification)
- .The lack of no preprocessing or look-ahead procedure

.Idea

- .Maximize the number of constraints that can be verified at each extension step
- .Graph isomorphism → topological constraints



.Result

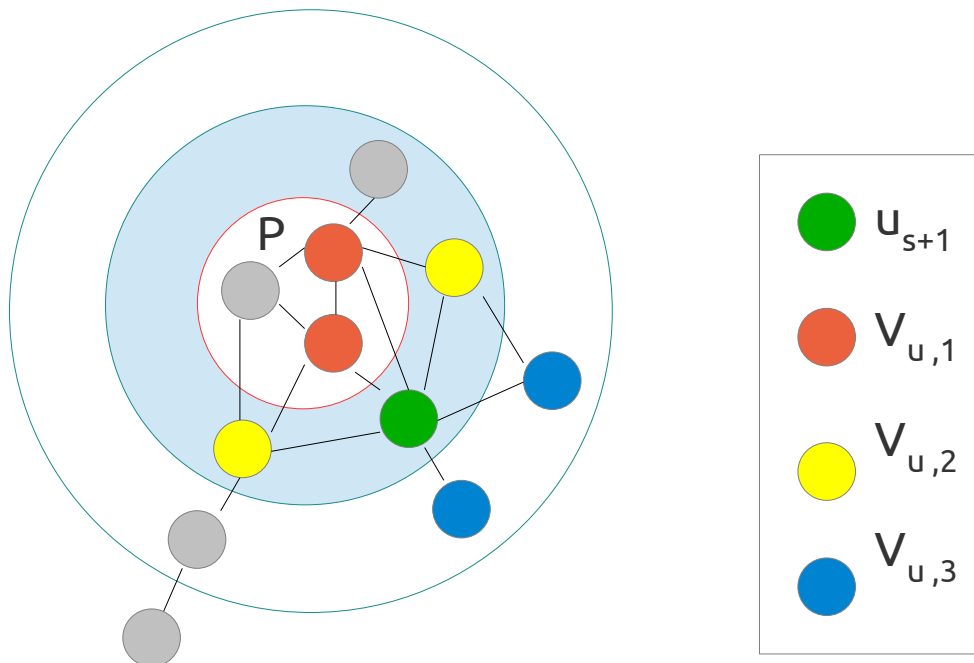
- .New search strategy
- .Static
- .Pattern dependent (only)
- .Via a greedy approach

The RI project

RI – Search strategy

- Given a partial variable ordering $P_s = (u_0, u_1, u_2, \dots, u_s)$, for $u_i \in E$
- The next vertex u_{s+1} is chosen such that it maximizes (in order) the following vertex sets

- $V_{u,1} = \{u_i : (u, u_i) \in E \text{ and } u_i \in P\}$
- $V_{u,2} = \{u_i : (u, u_i) \in E, u_i \in P \text{ and } \exists(u_i, u_j) \text{ for at least one } u_j \in P\}$
- $V_{u,3} = \{u_i : (u, u_i) \in E, u_i \in V_{u,1} \text{ and } u_i \in V_{u,2}\}$



Search strategies and variable (vertex) ordering

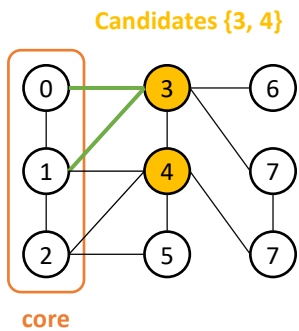
Tarjan 1984

Maximum cardinality

Linear time

Choses the next node with the highest number of edges towards the core .

1 Level of comparison.



Level 1: edges toward core

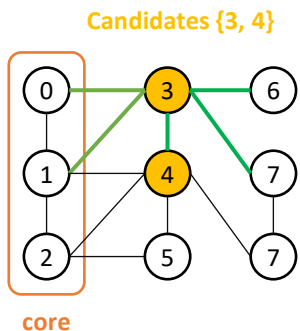
Ullmann 2012

Maximum cardinality
+ degrees

Linear time

Choses the next node with the highest Number of edges towards the core .
If two nodes have the same score, choses the one with the highest degree.

2 Level of comparison.



Level 1: edges toward core

Level 2: degree

CoreGraph (RI predecessor)

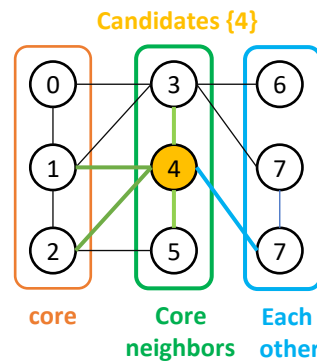
Maximize arc-consistency's elements

Not a simple Maximum cardinality
+ core's neighbors + degrees

Choses the next node with the highest Number of edges towards the core .
If two nodes have the same score, choses the one with the highest number of edges towards core's neighbors.
If nodes have too the same score, choses the one with the highest number of edges towards nodes that are not in the two previous sets.

No centrality based!!!
It does not consider paths or shortest paths.

3 Level of comparison.



Level 1: edges toward core

Level 2: edges toward core neighbors

Level 3: edges toward other nodes

Candidate node to be the next one in the sequence of initial ordering.

The RI project

RI – Verification ordering

• During an extension step, verify constraints taking them into account by their complexity

(1) M's injectivity

M(u) is not already mapped to a node of the partial solution

(2) Degree

$\text{degree}(u) \leq \text{degree}(M(u))$

(3) Label

$\alpha(u) = \alpha(M(u))$

(4) Topological constraints (edge existence)

1) for each $(u,v) \in E$ than $(M(u), M(v)) \in E'$ and $\beta((u,v)) = \beta((M(u), M(v)))$

2) for each $(u,v) \notin E$ than $(M(u), M(v)) \notin E'$

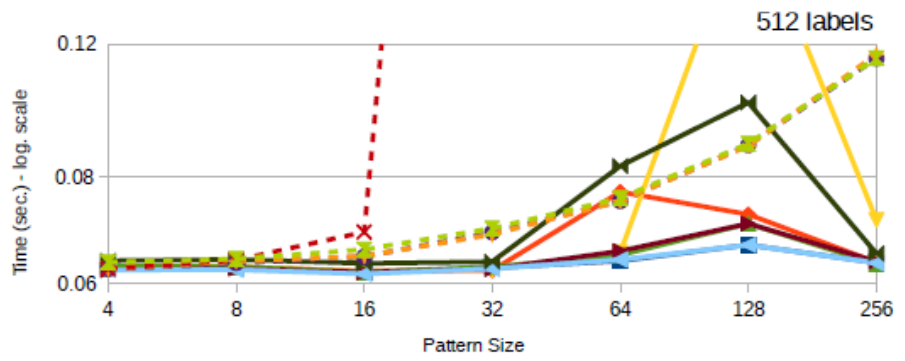
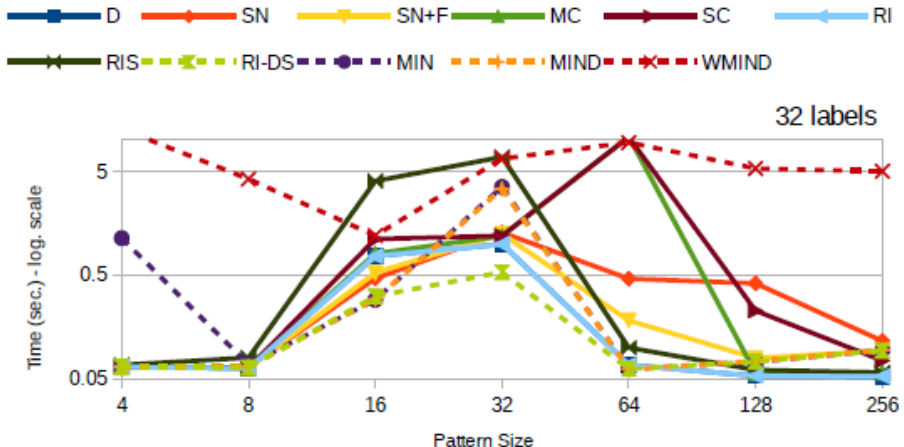
RI - Candidate selection

M(u) is chosen from a set of **candidates** which are the neighbors of a **parent vertex** of u in P

Search strategies and variable (vertex) ordering

Strategy	Static	Dynamic	Pattern	Target	State	Domains	Most-constraint	Fail-first
Degree	X		X				X	X
Unfrequently-feature	X			X				
Singleton-domain	X			X		X		
Minimum-domain		X			X	X		
Degree-weighted-domain	X	X	X		X	X	X	
Weighted-domain		X			X	X		X
State-neighbourhood	X		X		X			X
State-neighbourhood ⁺		X	X	X	X			
Maximum-cardinality	X		X		X		X	X
Sum-cardinality	X		X		X		X	X
3-way-N (RI)	X		X		X		X	X
3-way-N+D (RI-DS)	X		X	X	X	X	X	X

Bonnici, V., & Giugno, R. (2017). On the variable ordering in subgraph isomorphism algorithms. IEEE/ACM Transactions on Computational Biology and Bioinformatics (TCBB), 14(1), 193-203.



Constraint satisfaction problem (CSP) – Domain reduction

.Initial domains (compatibility maps)

- .Based on static graph invariants (labels, degree, topology, ...)

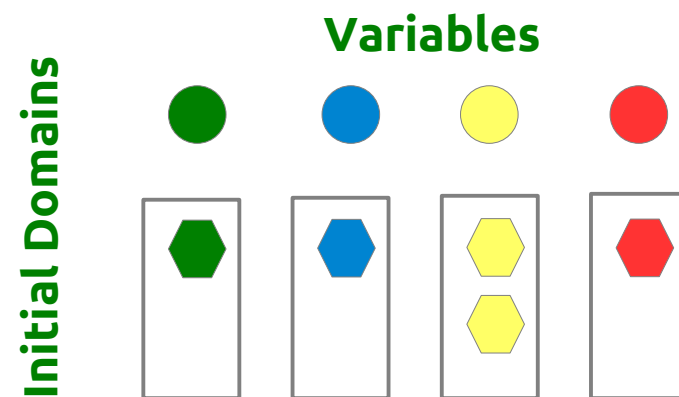
.Backtracking (or set-theory)

.Domain reduction procedures

- .After a value assignment, **propagate constraints** to unassigned value domains
- .If at least one **domain becomes empty**, then backtrack
- .**Save \ restore** domains at each step
- .**Local \ global** propagation
- .Propagation until **convergence**
- .Repeat until at least one domain is changing

.Search strategy

- .**Most constraint value**
- .**Min-dom** chose the variable with the smallest domain
- .Target and state dependent
-

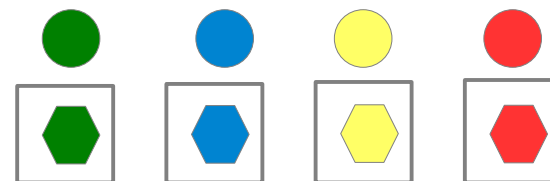
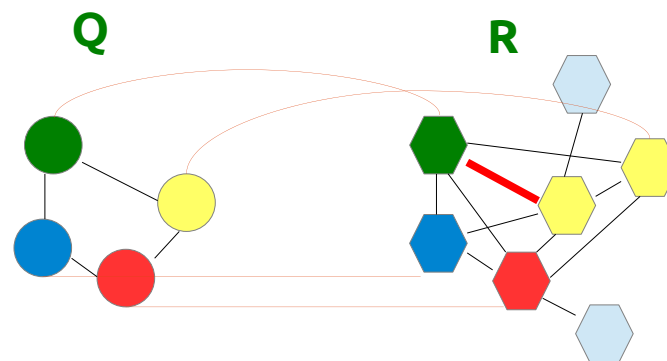


Constraint satisfaction problem (CSP) – Domain reduction

Topological (arc-)consistency

If $(u,v) \in E$, then
 for each value u' in the domain of u ,
 and for each value v' in the domain of v ,
 (u',v') must be in E' .

It can be used to build **initial domains**
 and/or **propagated** through reduction procedures.



Constraint satisfaction problem (CSP) – Domain reduction

Trade-offs (no free lunch)

Complexity vs Time

.Preprocessing and Look-ahead

- .The more complex is an approach, the more powerful it is
- .Ability to reduce the search space by pruning death branches
- .Es. global vs local
- .Domains (save \ restore)
- .Es. The more global is the reduction, the more domains we need to save and restore

Memory vs Time

.Data structures

- .Adjacency matrix (array vs bit-vectors) vs adjacency list (ordered vs unordered)
- .Precomputed compatibility maps vs on-line checking
- .Flags (Es. VF2 vertex flags)
- .Model states representation

Goal

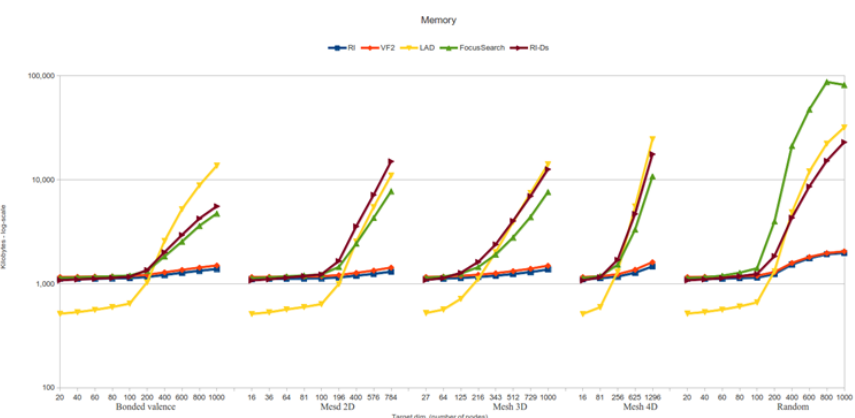
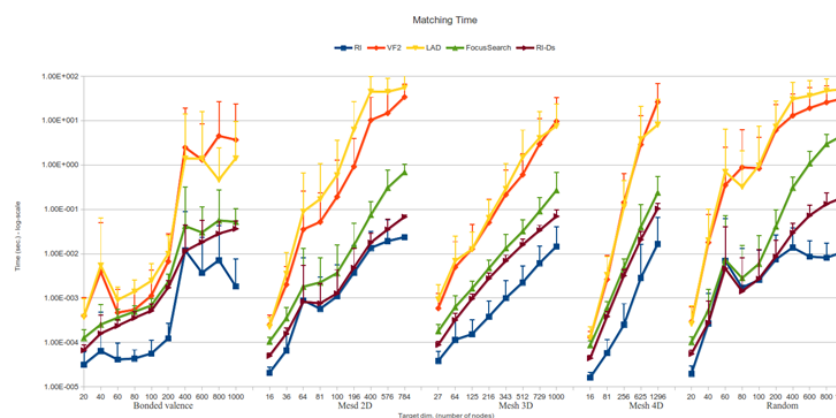
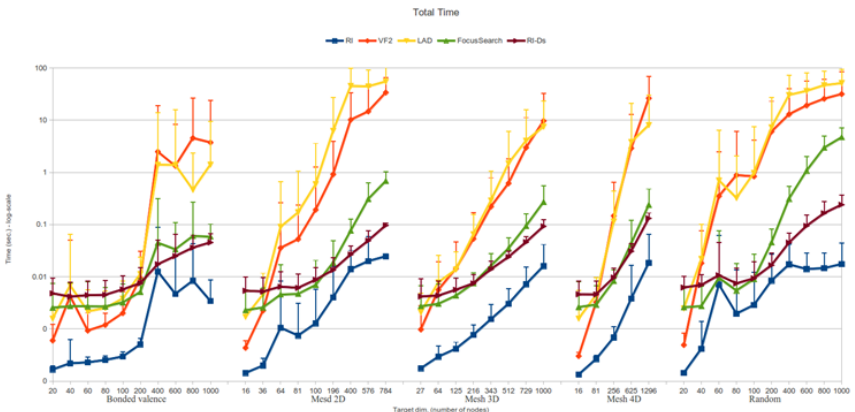
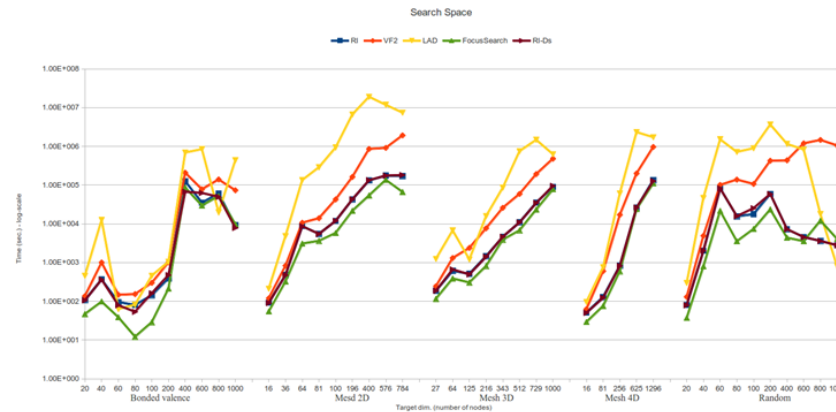
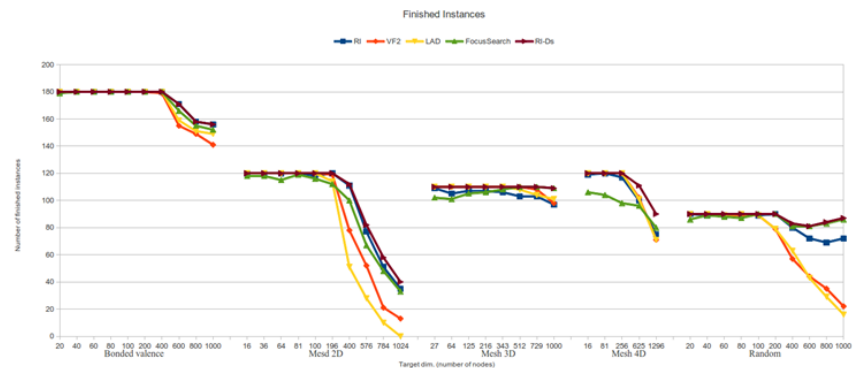
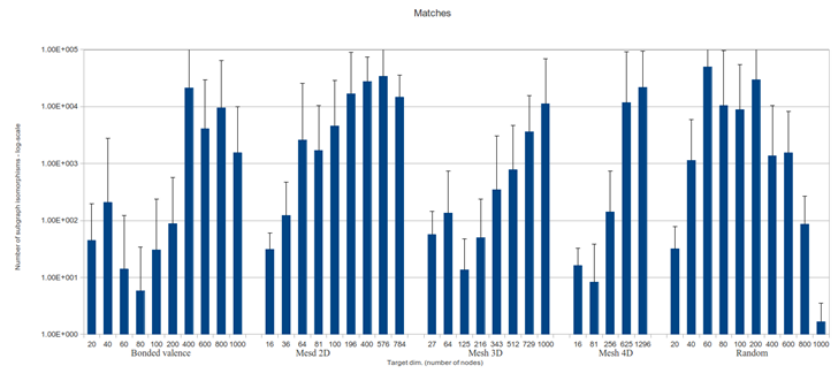
- .Find the best compromise between costs and advantages
- .Problem instance (pattern, target) and application domain dependent

The RI project

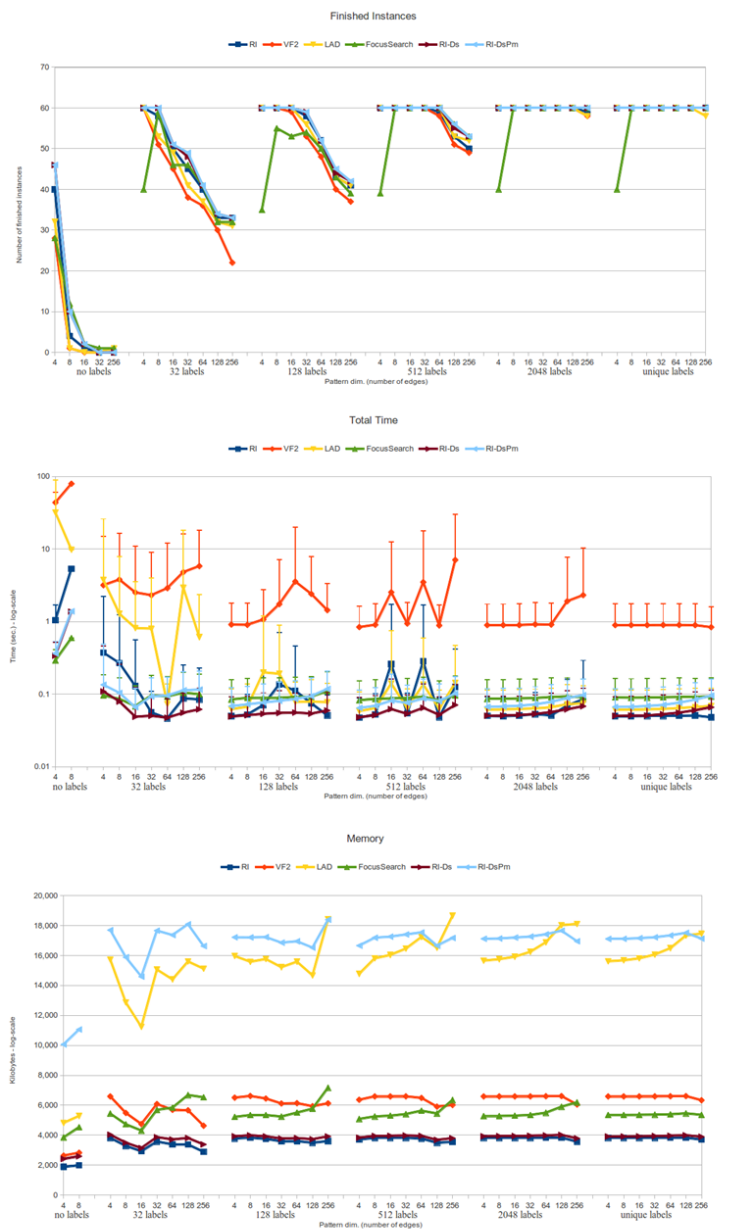
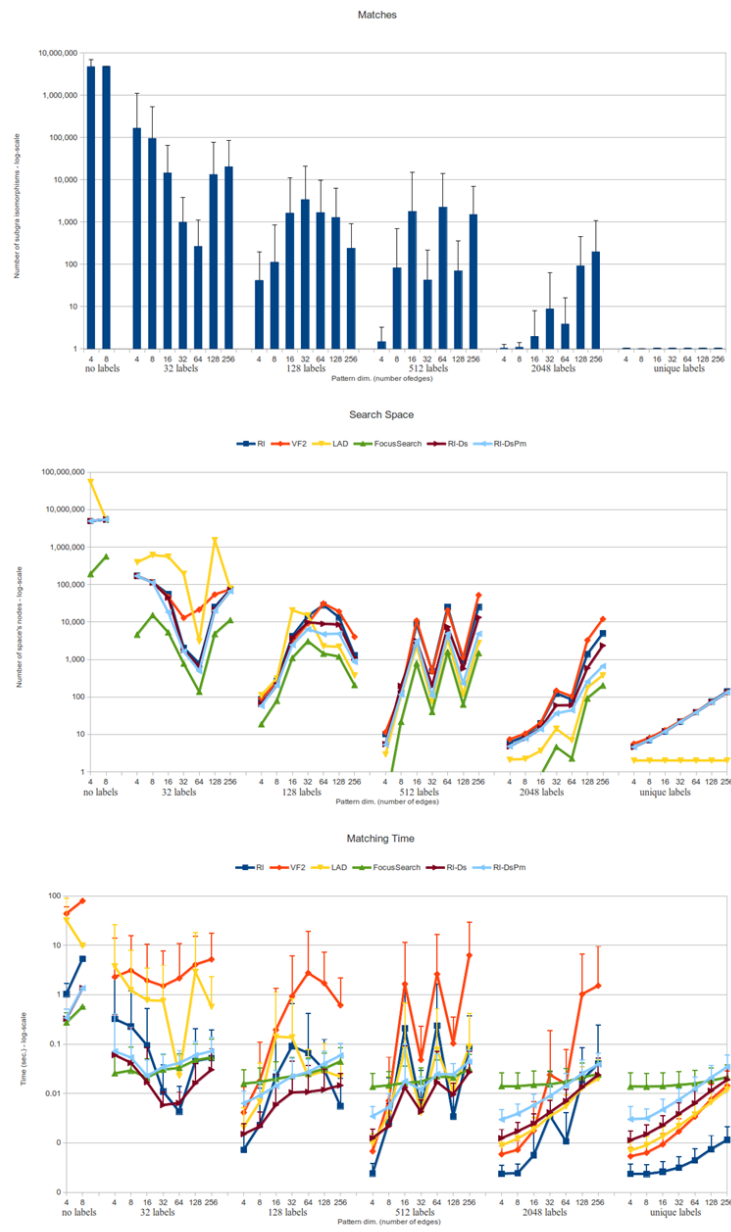
RI-DS

- Domain based version to deal with typical trade-offs
- Compatibility maps \ initial domains
- Degree and label
- During the backtracking, the same pair $u, M(u)$ can be verified multiple times
- Increases memory requirements
- Single pass of topological consistency over initial domains
- Variables with singleton domains (only one value) are putted at the head of the variable ordering

General results – Cordella et al. data set



General results - RI Datasets – Graemlin (microbial networks)



General results

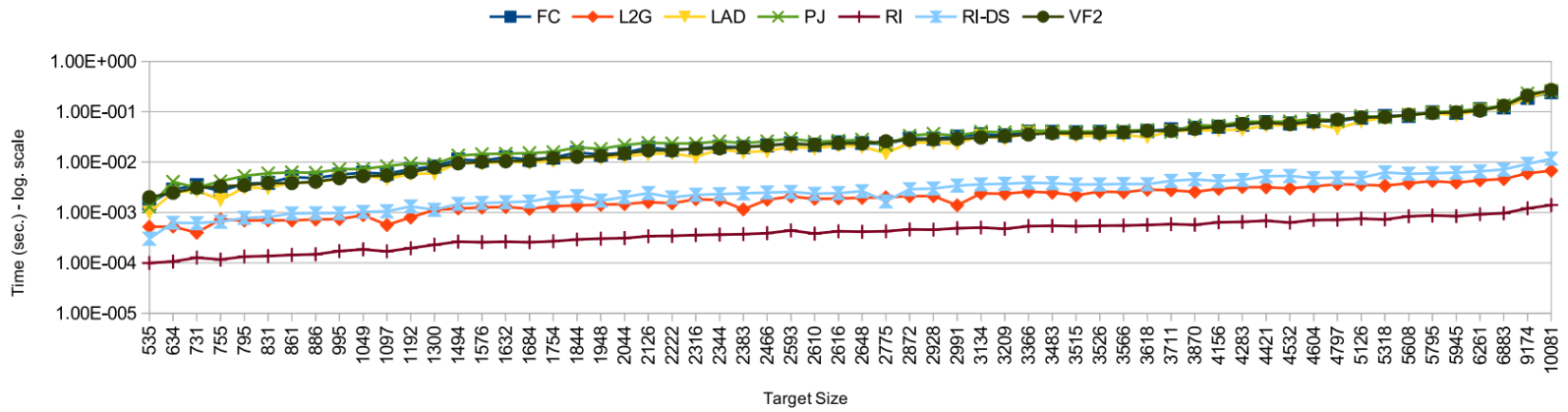


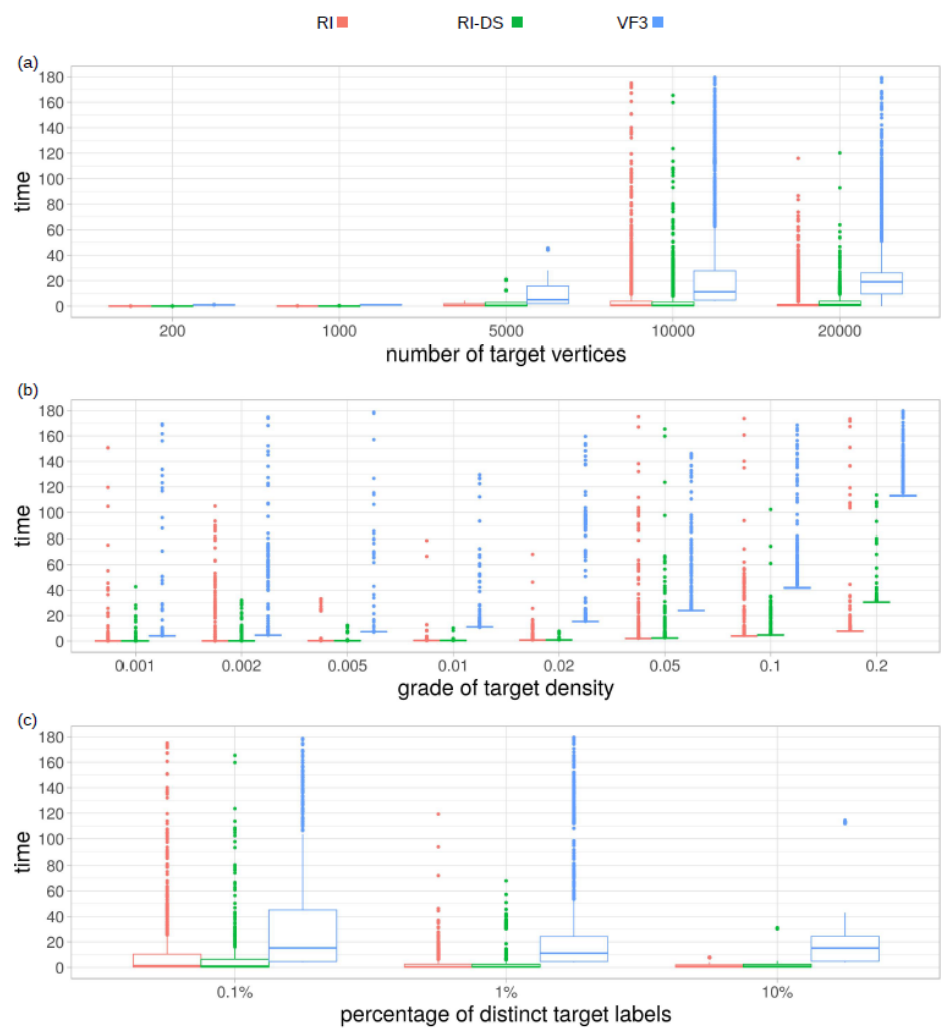
Fig. 9. Running times for finding all the pattern occurrences over the protein structure dataset.

V. Bonnici, R. Giugno. *On the Variable Ordering in Subgraph Isomorphism Algorithms*. IEEE/ACM TCBB 2017.



Winner of ICPR2014 contest on graph matching algorithms for pattern search in biological databases

General results: still the best (in average) in 2018



Aparo et al. "Simple Pattern-only Heuristics Lead to Fast Subgraph Matching Strategies on Very Large Networks." International Conference on Practical Applications of Computational Biology & Bioinformatics. Springer, Cham, 2018.

General results, by the others

RI seems to be currently the best algorithm for subgraph isomorphism, but for dense graphs the CSP approach makes LAD to perform better than RI on isomorphism.

V. Carletti, P. Foggia, M. Vento. Performance Comparison of Five Exact Graph Matching Algorithms on Biological Databases. New Trends in Image Analysis and Processing – ICIAP 2013. Lecture Notes in Computer Science Volume 8158, 2013, pp 409-417.

*These methods are verified by three social network datasets whose node's number vary from thousands to millions. According to the experiments, we can find that VF2 and RI is applicable for rather small graphs while SPath performs better in large graph when average degree of graph is small. When graph with high average degree, TurboISO and BoostISO performs better. What's more, we also compare the efficiency of different matching order when searching. **The order choosing strategy proposed by RI is better.***

Ma, Tinghuai, et al. "A Comparative Study of Subgraph Matching Isomorphic Methods in Social Networks." *IEEE Access* (2018).

Limitations

RI approach is inherently **local**

No exploration of the **global** pattern graph **structure**

No information about **target** graph

- **Global structure** and **target information** may improve the effectiveness in evaluating future constraints.
- A centrality measure identifies the **most central** vertices within a graph.
- The concept of **central** is relative and depends on how to analyze the properties of the vertices

A vertex's centrality is a function of the centrality of the vertices it is associated with. Centralities distinguish themselves on how association is defined. Bonacich showed that if association is defined in terms of **walks**, then a **family of centralities** can be defined.

- The **degree** counts walks of length one
- The **eigenvalue** centrality counts walks of length infinity
- The **Estrada's subgraph** centrality proposes only counting closed paths

Bonacich, Phillip (1987). "Power and Centrality: A Family of Measures". *American Journal of Sociology*. University of Chicago Press. **92**: 1170–1182

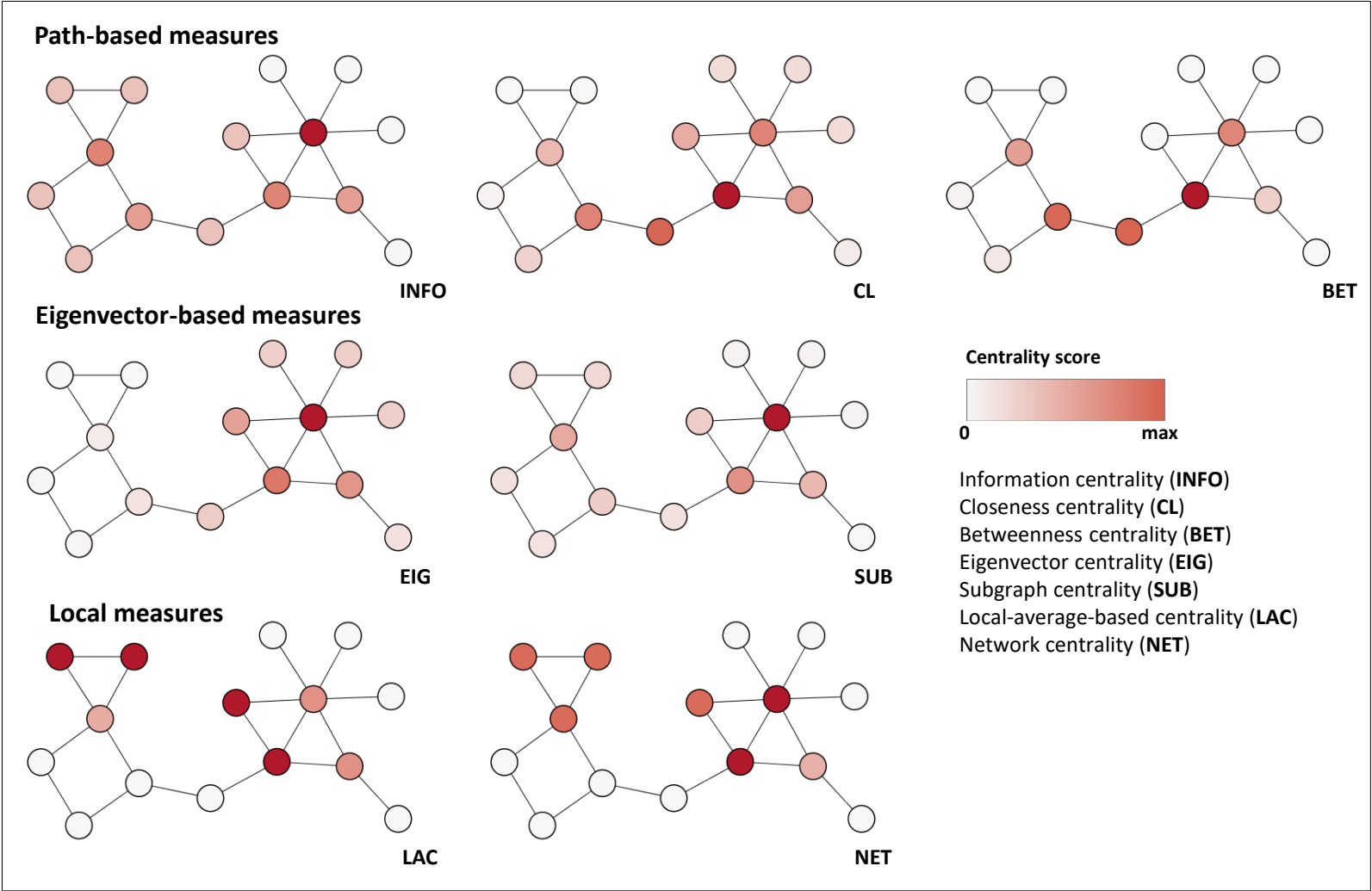
Ordering by weighted centrality: a try to make the RI strategy less local

Centrality name	Description	Refs.
Betweenness Centrality (BC)	$C_B(u) = \sum_{s \neq u \neq t} \frac{\rho(s,u,t)}{\rho(s,t)}$ $\rho(s,t)$ is the total number of shortest paths from node s to node t $\rho(s,u,t)$ is the number of those paths that pass through u	Anthonisse, 1971
Closeness Centrality (CC)	$C_C(u) = \sum_{v \in V} \frac{ N_u -1}{\text{dist}(u,v)}$ $ N_u $ is the number of node u 's neighbors. $\text{dist}(u,v)$ is the distance of the shortest path from node u to node v .	Sabidussi, 1966
Degree Centrality (DC)	$C_D(u) = N_u $ $ N_u $ is the number of node u 's neighbors.	Jeong et al., 2001
Eigenvector Centrality (EC)	$C_E(u) = \alpha_{\max}(u) \alpha_{\max}$ is the eigenvector corresponding to the largest eigenvalue. of the adjacency matrix A 's.	Bonacich, 1987
Local average connectivity-based method (LAC)	$LAC(u) = \sum_{\omega \in N_u} \frac{\deg_{C_u}(\omega)}{ N_u }$ C_u is the subgraph induced but, N_u , $\deg_{C_u}(\omega)$ is the number of nodes is directly connected in C_u .	Li et al., 2011
Network Centrality (NC)	$C_N(u) = \sum_{v \in N_u} ECC(u,v) \sum_{v \in N_u} \frac{ N_u \cap N_v }{\min(N_u -1, N_v -1)}$ N_u is the node set containing all the neighbors of node u . is the node set containing all the common neighbors of node u and node v .	Wang et al., 2012
Subgraph Centrality (SC)	$C_S(u) = \sum_{l=0}^{\infty} \frac{\mu_l(u)}{l!} = \sum_{v=1}^N (v_v^u)^2 e^{\lambda_v} \mu_l(u)$ is the u th diagonal entry of the k th power of the adjacency matrix of network. $v_1, v_2, \dots, v_N$ is R^N be an orthonormal basis of composed by eigenvectors of A associated to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$ v_v^u is the u th component of v_v .	Estrada and Rodriguez-Velazquez, 2005
Information Centrality (IC)	$I_{u,v} = (c_{u,u} + c_{v,v} - 2 \times c_{u,v})^{-1}$ $I_{u,v} = (c_{u,u} + c_{v,v} - 2 \times c_{u,v})^{-1} c = (c_{u,v}) = (D - A + J)^{-1} C = A$ is the adjacency matrix of network, D is the diagonal matrix of all nodes' degree, J is a matrix whose all elements are 1.	Stephenson and Zelen, 1989

Ordering by weighted centrality: a try to make the RI strategy less local

Centrality name	Description
Betweenness Centrality (with weight)	$cost(s, t) = \frac{1}{w_{(s,t)}} \times w_{(s,t)} > 0$ $w_{s,t}$ is the weight of edge from node s to node t , $dist_w(u, v) = \sum_{(s,t) \in P} cost(s, t)$ The shortest path from node s to node t is the path whose $dist_w(u, v)$ is smallest. $C_{Bw}(u) = \sum_{s \neq u \neq t} \frac{\rho(s, u, t)}{\rho(s, t)} \rho(s, t)$ is the total number of shortest paths from node s to node t . $\rho(s, u, t)$ is the number of those paths that pass through u .
Closeness Centrality (with weight)	$cost(s, t) = \frac{1}{w_{(s,t)}} \times (w_{(s,t)} > 0) w_{s,t}$ is the weight of edge from node s to node t , $dist_w(u, v) = \sum_{(s,t) \in P} cost(s, t) dist_w(u, v)$ is the weighted distance of the shortest path from node u to node v . $C_{Cw}(u) = \frac{ N_u -1}{\sum_{v \in V} dist_w(u, v)}$ is the number of node's neighbors.
Degree Centrality (with weight)	$C_{D,w}(u) = \sum_{v \in N_u} w(u, v) N_u$ is the node set containing all the neighbors of node u . $w(u, v)$ is the weight of the edge connected node u and node v .
Eigenvector Centrality (with weight)	$C_E(u) = \alpha_{\max}(u) \alpha_{\max}$ is the eigenvector corresponding to the largest eigenvalue of the weighted adjacency matrix A_w .
Local average connectivity-based method (with weight)	$LAC_w(u) = \frac{\sum_{s \in N_u} \sum_{t \in N_s} \cap N_u w(s, t)}{ N_u } N_u$ is the node set containing all the neighbors of node u . $w(s, t)$ is the weight of the edge connected node s and node t .
Network Centrality (with weight)	$C_{Nw}(u) = \sum_{v \in N_u} \frac{\sqrt{\sum_{s \in N_u \cap N_v} w(u, s) \times \sum_{t \in N_u \cap N_v} w(v, t)}}{\min(\sum_{s \in N_u} w(u, s) - 1, \sum_{t \in N_v} w(v, t) - 1)} N_u$ is the node set containing all the neighbors of node u . $ N_u \cap N_v $ is the node set containing all the common neighbors of node u and node v . $w(u, s)$ is the weight of the edge connected node u and node s .
Subgraph Centrality (with weight)	$C_{Sw}(u) = \sum_{l=0}^{\infty} \frac{\mu_l(u)}{l!} = \sum_{v=1}^N (v_v^u)^2 e^{\lambda_v} \mu_l(u)$ is the u th diagonal entry of the l th power of the weighted adjacency matrix of network. $v_1, v_2, \dots, v_N$ is be an orthonormal basis of R^N composed by eigenvectors of A associated to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$. v_v^u is the u th component of v_v .
Information Centrality (with weight)	$C_{Iw}(u) = \left[\frac{1}{ V } \sum_{v \in V} \frac{1}{I_{u,v}} \right]^{-1} I_{u,v} = (c_{u,u} + c_{v,v} - 2 \times c_{u,v})^{-1} (c_{u,v}) = (D_w - A_w + J_w)^{-1} \frac{1}{I_{u,u}} = 0$ A is the weighted adjacency matrix of network, D is the diagonal matrix of all nodes' degree, J is a matrix where all elements are 1.

Ordering by weighted centrality: a try to make the RI strategy less local



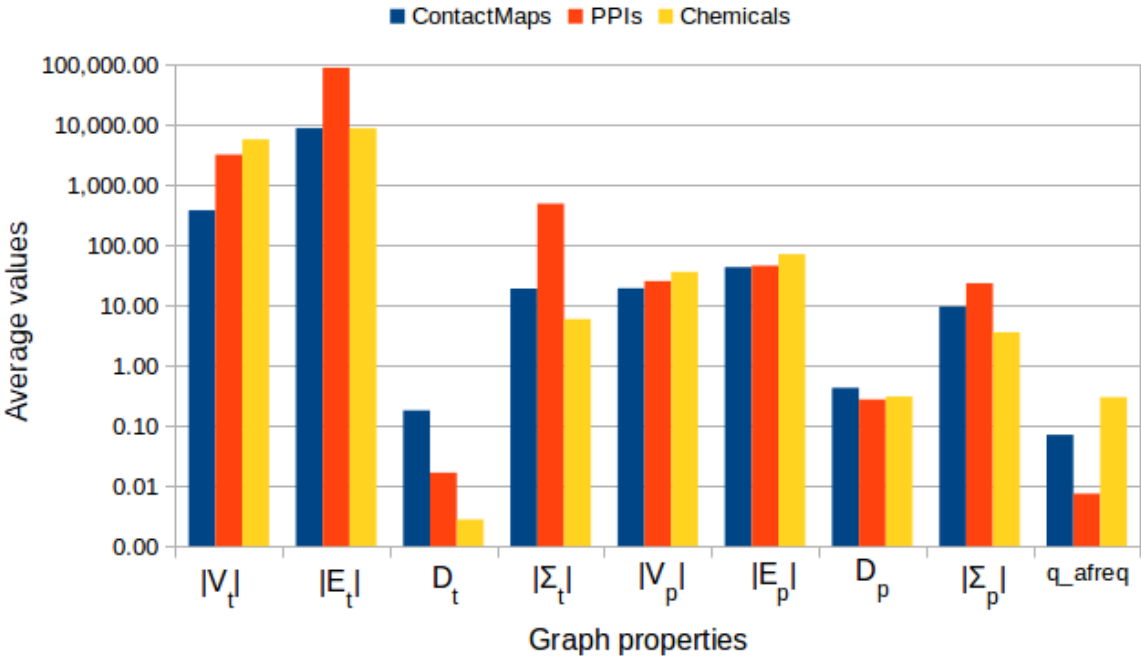
Ordering by weighted centrality: a try to make the RI strategy less local

We evaluated performances of the investigated strategies over a widely used biological graphs

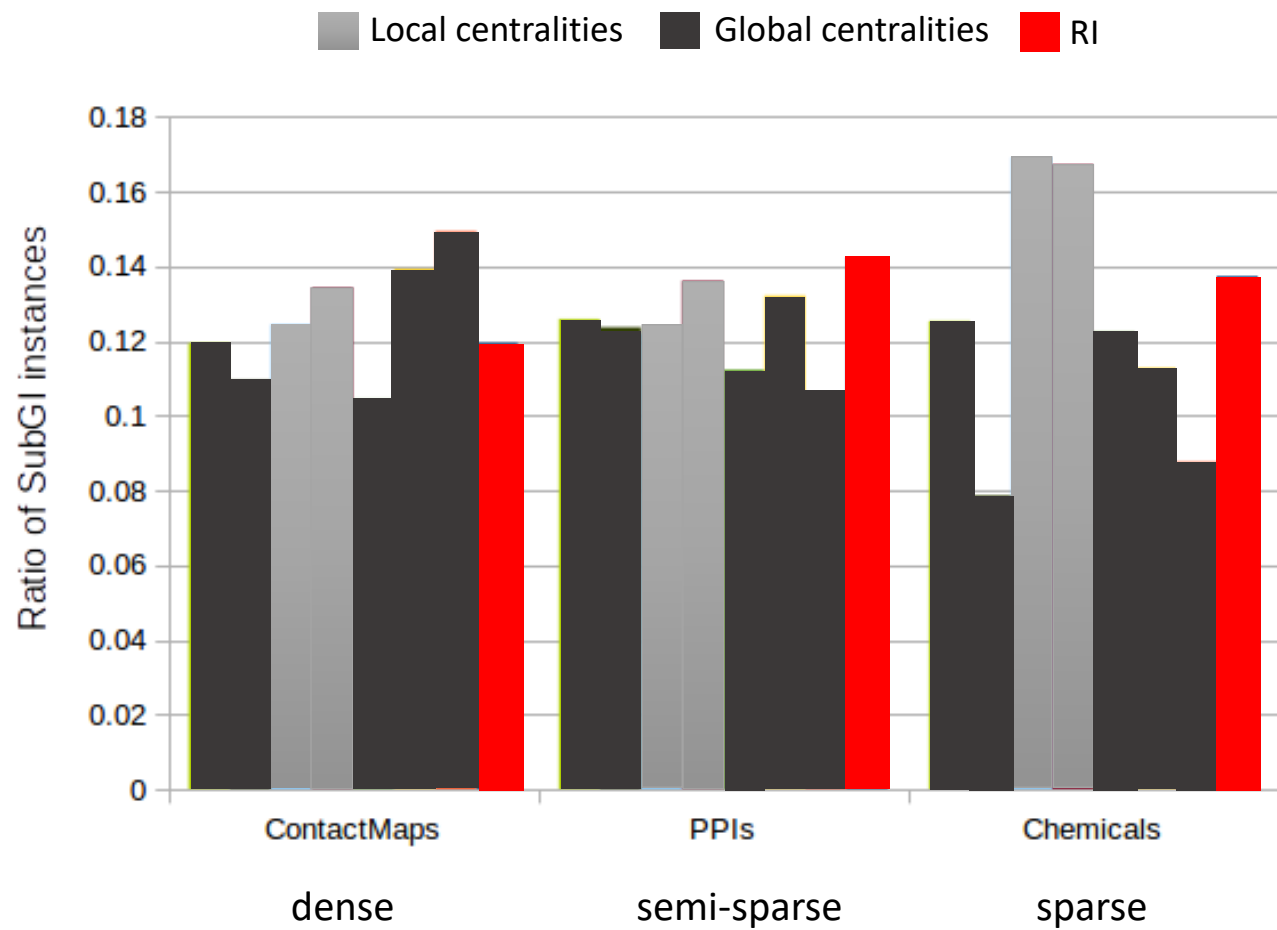
- Protein contact maps (**dense graphs**)
- Protein-protein networks (**semi-sparse graphs**)
- Chemical protein structures (**sparse graphs**)

Pattern graphs have been extracted by varying the number of edges

- From 8 to 256 edges
- Equal number of patterns for each edge size

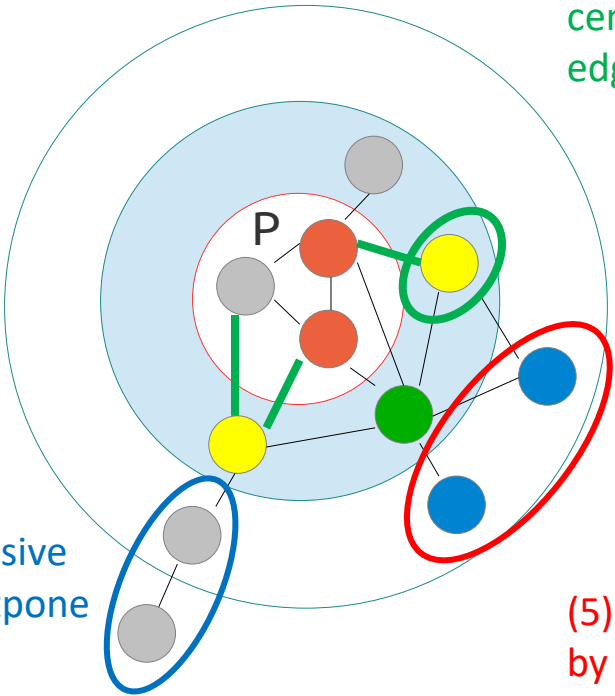


Ordering by weighted centrality: a try to make the RI strategy less local



Lesson learned and Observations

- (1) Target-aware strategies, by means of label co-frequency, give significant improvements.
- (2) Local strategies work better!!!



(4) This yellow vertex seems to be more central (connected), but its number of edges toward the core is lower.

(3) Tails generate the most expensive combinatorics. It is better to postpone the searching of them.

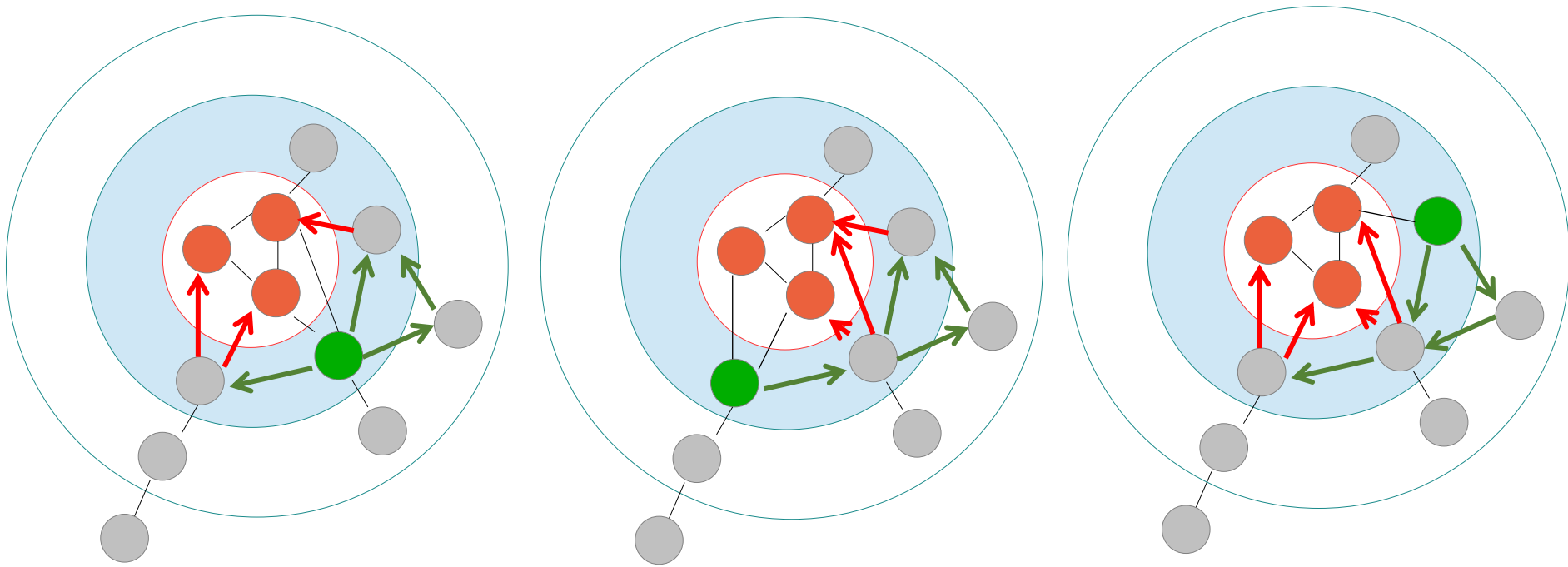
(5) This set could be further separated by back-connected vertices and only forwarding vertices.

The basis of a new ordering: centrality with respect to a core set

Centralities are statically defined and they do not take into account the influence of a “core” set of vertices.

In order to choose the vertex that is the more constrained w.r.t. the current core set (**orange vertices**)

- We image the core as a source of information centered to the vertex we want to choose (**green vertex**)
- We spread the information along the local topology (**green edges**)
- We measure the information that comes back to the core (**red edges**)



This is the one who maximizes the “**cycles**” to the core.

The basis of a new ordering: centrality with respect to a core set

- Let $f((u,v))$, for $(u,v) \in E$, to be the co-label weight $(1 - \text{frequency})$ of the labels of u and v .
 - If (u,v) not in E , then $f((u,v))=0$
- Let u_s the vertex to be chosen
- Let $P_s = (u_0, u_1, u_2, \dots, u_{s-1})$ to be a partial ordering (the current core set)
- Let $N(u_s) = \{v : (u_s, v) \in E\}$, namely the neighborhood of u_s

We perform a 2-levels BFS starting from u_s and excluding the vertices in P_s

- B_s^1 is the set of vertices at level 1 of the BFS, $B_s^1 = \{v : v \in N(u_s) - P_s - \{u_s\}\}$
- B_s^2 is the set of vertices at level 2 of the BFS, $B_s^2 = \{v : v \in N(B_s^1) - P_s - \{u_s\} - B_s^1\}$

The **forward centrality** scores (the information that flows forwarding from the core) is given by:

- (3) For $u \in B_s^1$, $c(u) = \sum_{v \in P_s \cup \{u_s\}} f((u,v)) + \sum_{v \in B_s^1} c(v) * f((u,v))$
- (5) For $u \in B_s^2$, $c(u) = \sum_{v \in B_s^1} c(v) * f((u,v)) + \sum_{v \in B_s^2} c(v) * f((u,v))$

The **core-backward centrality** (the information that comes back to the core) is given by:

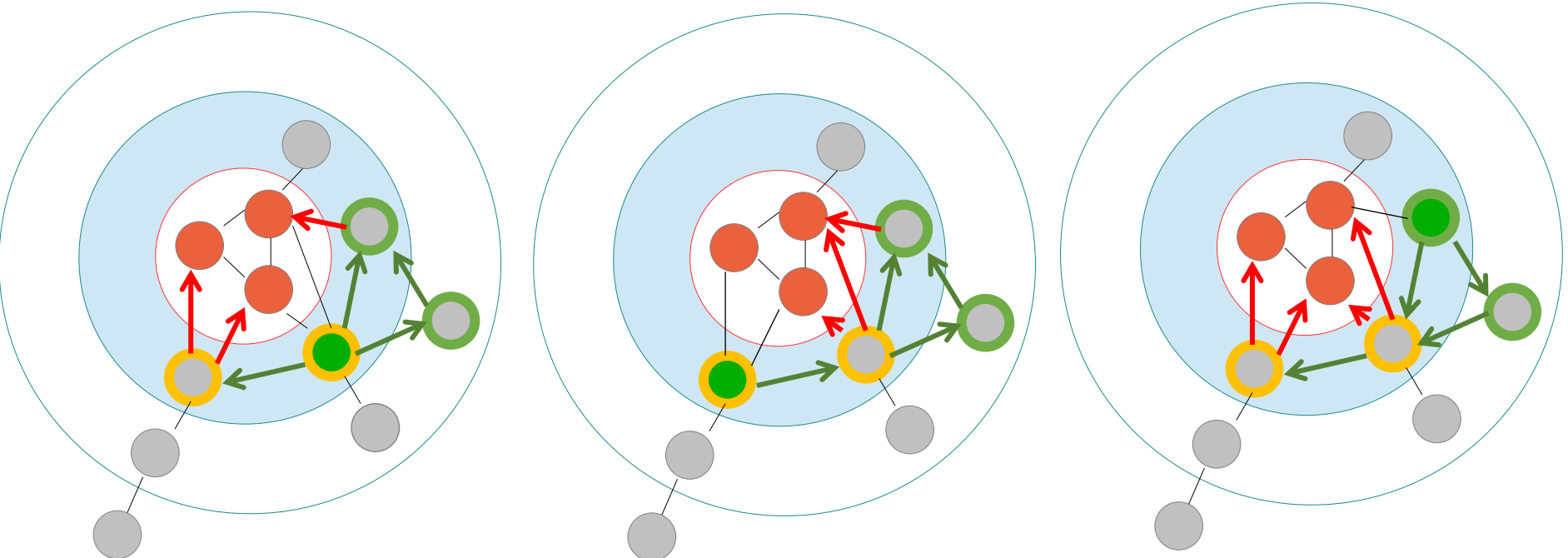
- (1) $C(u_s) = \sum_{v \in P_s} f((u_s,v))$
- (2) For $u \in B_s^1$, $C(u) = \sum_{v \in P_s \cup \{u_s\}} c(u) * f((u,v))$
- (4) For $u \in B_s^2$, $C(u) = \sum_{v \in P_s \cup \{u_s\}} c(u) * f((u,v))$

We assign ordering preferences by compare the measures (1),(2),(3),(4) and (5) in the given order.

Dispersion of the flooded information

- We want to have a measure of the dispersion regarding the information that does not returning back to the core set
- We count the vertices of the 2-levels BFS that are connected to paths not leading to the core set

 Non-dispersion vertices  Dispersion vertices

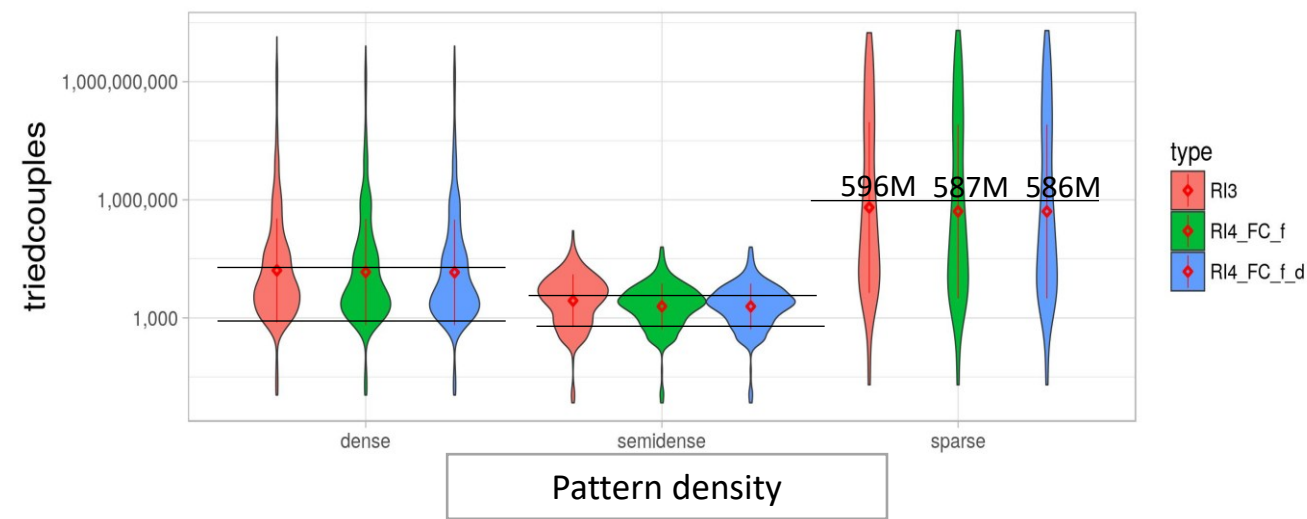


- We multiply each vertex score for the ration between the amount of the 2-levels BFS vertices and the dispersion vertices
- This is the one who maximizes the “cycles” to the core, and does not lead to periphery.

Differences with respect to the previous versions

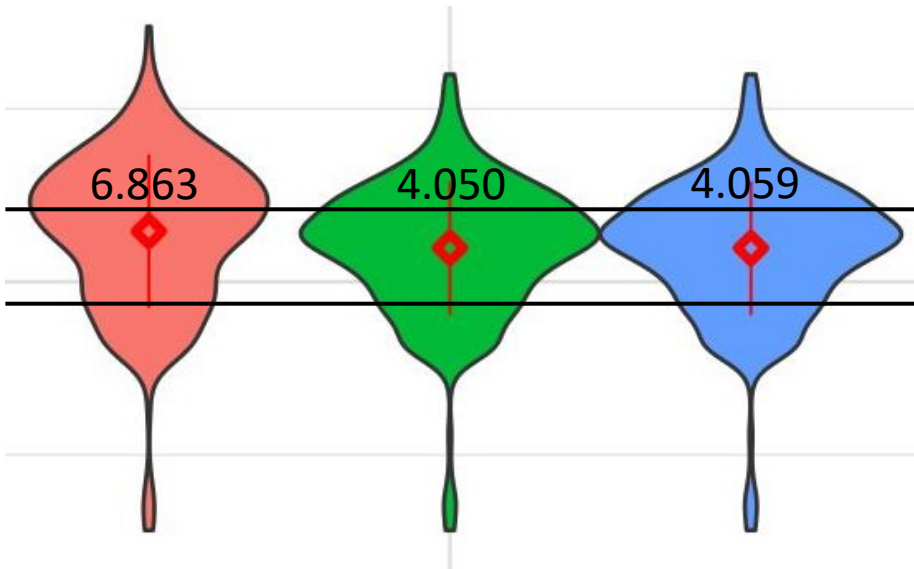
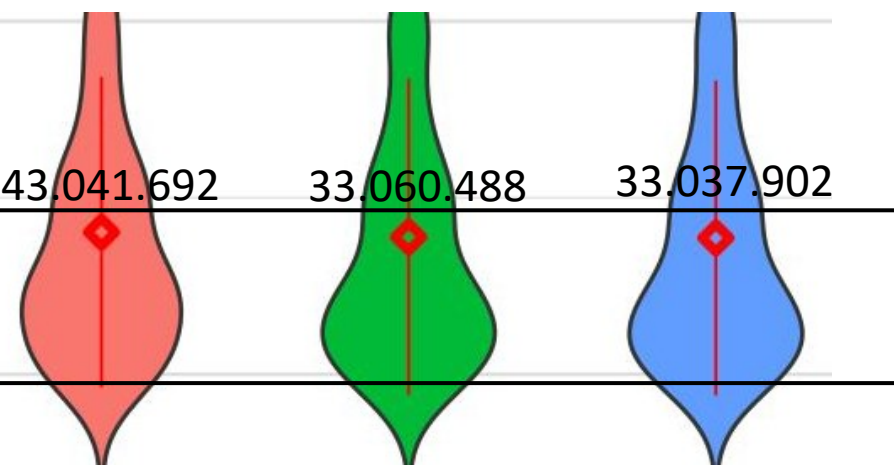
- weighted degree centrality for the first vertex:
The first vertex of the ordering is chosen by searching for the vertices linked to the edges with the minimum co-label frequency, and among them by choosing the vertex who minimizes the average co-label frequency of its edges
- The centrality-to-core measure is used for the general case of the vertex ordering
- The order in which edges are verified follows their co-label frequency. Edges with lower frequency are verified first

Results

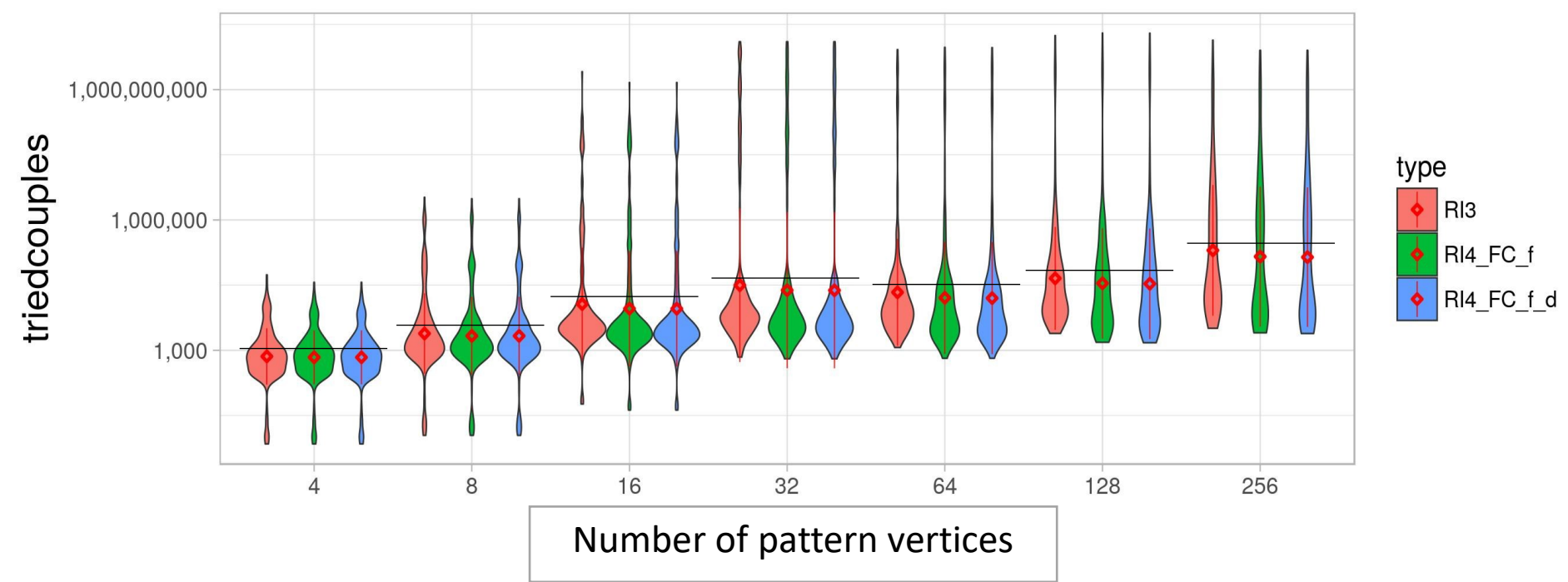


Timeouts RI3: 180
Timeouts RI4: 130

Victories RI3 = 350
Victories RI4 = 900
Draws = 310



Results



ArcMatch

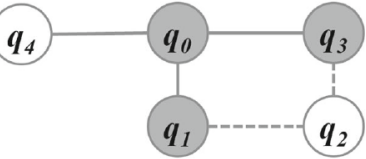
The concept of vertex domain naturally extends to edges.

Given two query vertices, q_i and q_j , their domains $D(q_i)$ and $D(q_j)$, and an edge $\{q_i; q_j\} \in E_q$ linking them, the domain of such an edge is given by

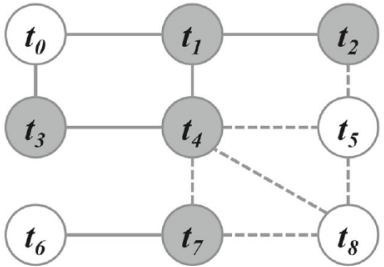
$$D(\{q_i; q_j\}) = \{ \{t_k; t_h\} \in E_t : t_k \in D(q_i) \wedge t_h \in D(q_j) \}$$

Vertex domains and **edge domains** can be combined into a single structure, here called a domain graph (DG), first introduced in Han et al. (2013) and then refined in Bi et al. (2016) and subsequently in Han et al. (2019).

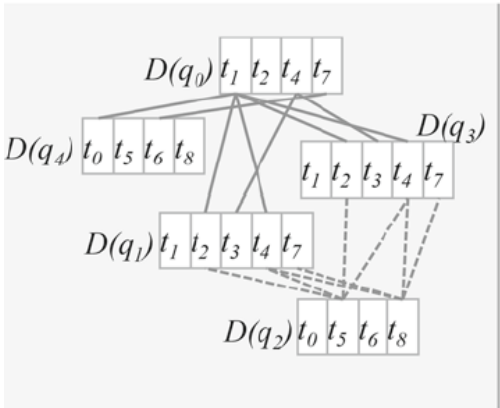
(a) Query graph



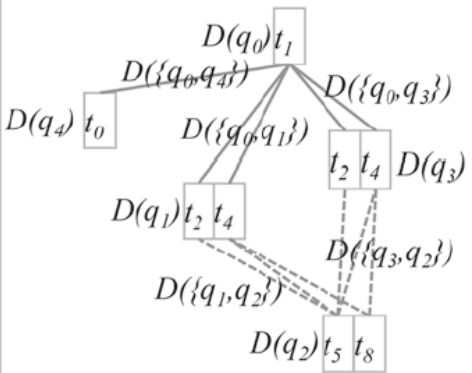
(b) Target graph



(b) Connections between vertices in domains

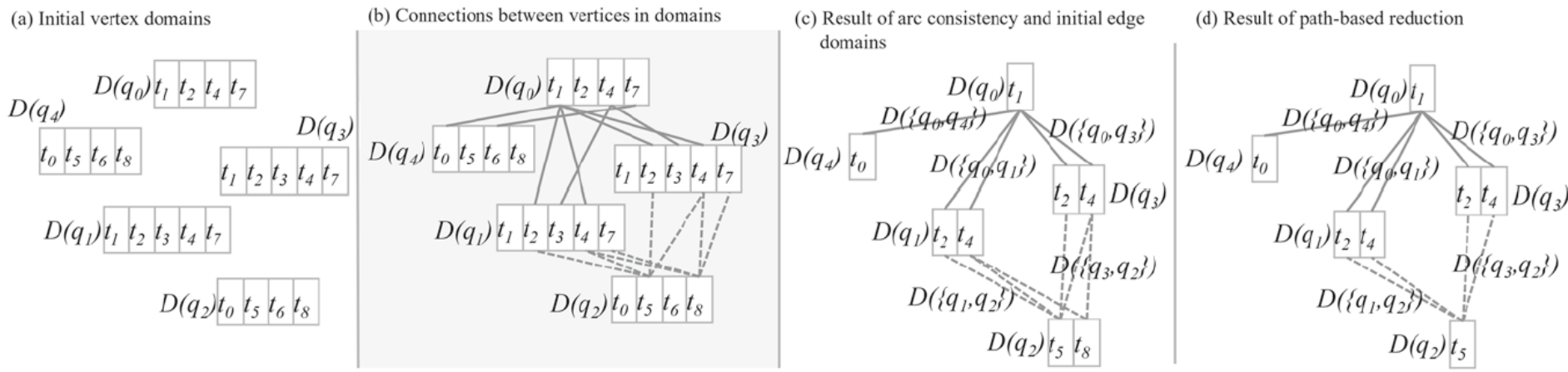


(c) Result of arc consistency and initial edge domains



ArcMatch

ArcMatch introduces a new domain reduction techniques based on edge-domains that generalizes the concept of arc-consistency to paths, called **path-based reduction**.



The **main steps** of the algorithm are:

1. compute initial vertex domains
2. run arc consistency over vertex domains
3. compute initial edge domains
4. run path-based reduction procedure
5. choose a static variable ordering
6. run the backtracking phase to locate matching occurrences.

Input:
a path ω ,
the path reduction parameter lp ,
and the vertex and edge domains D .

- Extract paths from the query graph.
- Verify paths of maximum length or rings.

Symbol	Definition	Description
G	$(V, E) : E \in \{\{u, v\} : u, v \in V\}$	A graph as a set of vertices V plus a set of edges E
α	$\alpha : V \mapsto A$	A function mapping vertices to labels
β	$\beta : E \mapsto B$	A function mapping edges to labels
G_q	(V_q, E_q)	A query graph
G_t	(V_t, E_t)	A target graph
$\deg(u)$	$ \{\{v_i, v_j\} : v_i = u \text{ or } v_j = u\} $	The degree of the vertex u
$N(u)$	$\{v : \{u, v\} \in E \text{ or } \{v, u\} \in E\}$	The neighborhood of the vertex u
$N(V')$	$(\cup_{u \in V'} N(u)) \setminus V' : V' \subseteq V$	The neighborhood of the set of vertices V'
ω	$(v_1, v_2, \dots, v_n) : v_i \in V, \{v_i, v_{i+1}\} \in E, \forall i : 1 \leq i \leq n-1$	A path of length $ \omega = n$
ω_i	$(v_1, v_2, \dots, v_i) : \omega = (v_1, v_2, \dots, v_n), 1 \leq i \leq n$	The first i vertices of ω
$\omega[i]$	$v_i : \omega = (v_1, v_2, \dots, v_n), 1 \leq i \leq n$	The i th vertex of the path ω
$\omega[i..j]$	$(v_i, v_{i+1}, \dots, v_{j-1}, v_j) : \omega = (v_1, v_2, \dots, v_n), 1 \leq i \leq j \leq n$	The subpath of ω from the i th to the j th vertex of it, both included
θ	$(v_1, v_2, \dots, v_{ P }) : v_i \in V$	An ordering of the vertices of V
θ_i	$(v_1, v_2, \dots, v_i) : \theta = (v_1, v_2, \dots, v_{ P }), 1 \leq i \leq n$	A partial ordering, namely the first i vertices of θ
$\theta[i]$	$v_i : \theta = (v_1, v_2, \dots, v_{ P }), 1 \leq i \leq n$	The i th vertex of θ
(q_i, t_i)	$q_i \in V_q, t_i \in V_t$	A matching pair
M	$M : V_q \mapsto V_t$	A SubGI mapping of G_q into G_t as an injective function
M	$((q_1, t_1), (q_2, t_2), \dots, (q_n, t_n))$	A SubGI mapping of G_q into G_t as an ordered set of matching pairs
M_i	$((q_1, t_1), (q_2, t_2), \dots, (q_i, t_i)) : 1 \leq i \leq n$	The first i th pairs of the mapping M
$\mathbb{M}$	$\{M^1, M^2, \dots, M^k\}, \mathbb{M} = k$	The entire set of matches between a query G_q and a target graph G_t
$D(q_i)$	$\{t_h \in V_t : t_h \simeq q_i\}$	The domain of the vertex q_i
$D(\{q_i, q_j\})$	$\{\{t_h, t_k\} \in E_t : t_h \in D(q_i), t_k \in D(q_j), \beta(\{t_h, t_k\}) = \beta(\{q_i, q_j\})\}$	The domain of the edge $\{q_i, q_j\}$

ArcMatch

Algorithm 3 Path reduction by verifying the existence of at least one path in the domain graph corresponding to the query path ω .

Input:
a path ω ,
a variable *isRing* to specify if ω is a ring or not,
and the vertex and edge domains D .

```
1: procedure VERIFYPATH( $\omega$ , isRing,  $D$ )
2:   reduced  $\leftarrow$  false
3:   for  $v \in D(\omega[1])$  do
4:      $\hat{\omega} \leftarrow (v)$ 
5:     if not VerifyPathDFS( $\omega, \hat{\omega}, isRing, D$ ) then
6:        $D(\omega[1]) \leftarrow D(\omega[1]) \setminus \{v\}$ 
7:       reduced  $\leftarrow$  true
8:     end if
9:     if reduced then
10:      RefineDomains( $v$ )
11:    end if
12:  end for
13: end procedure
```

- Verify a query path on the domain graph.
 - Traverse the domain graph according to the given query path via a DFS
- Discard target vertices of the starting domain is no supporting path is found

Algorithm 4 Path reduction to verify the existence of at least one path $\hat{\omega}$ in the domain graph corresponding to the query path ω .

Input:
a query path ω ,
a target path $\hat{\omega}$,
a variable *isRing* to specify if ω is a ring or not,
and the vertex and edge domains D .
Return:
true if $\hat{\omega}$ is a path in the target graph that corresponds to the query path ω according to the vertex and edge domains, *false* otherwise.

```
1: procedure VERIFYPATHDSF( $\omega, \hat{\omega}, isRing, D$ )
2:   if  $|\hat{\omega}| = |\omega|$  then return true
3:   else
4:      $i \leftarrow |\hat{\omega}|$ 
5:      $u_q \leftarrow \omega[i]$ 
6:      $v_q \leftarrow \omega[i + 1]$ 
7:     for  $(\{u_t, v_t\} \in D(\{u_q, v_q\}) : u_t = \hat{\omega}[i])$  do
8:       if isRing AND  $(|\hat{\omega}| = |\omega| - 1)$  then
9:         if  $v_t = \hat{\omega}[1]$  then return true
10:       end if
11:     else
12:       if  $v_t \notin \hat{\omega}$  then
13:          $\hat{\omega}[i + 1] \leftarrow v_t$ 
14:         if VerifyPathDFS( $\omega, \hat{\omega}, isRing, D$ ) then return true
15:       end if
16:        $\hat{\omega} \leftarrow \hat{\omega}[1 \dots i]$ 
17:     end if
18:   end if
19: end for
20: return false
21: end if
22: end procedure
```

ArcMatch

Algorithm 5 Refinement of domains after the alteration of the domain of the query vertex s .

```

Input:
a query vertex  $s$ ,
the query edge set  $E_q$ ,
and the vertex and edge domains  $D$ .

1: procedure REFINEDOMAINS( $s, E_q, D$ )
2:   for  $\{s, v_q\} \in E_q$  do
3:     for  $\{u_t, v_t\} \in D(\{s, v_q\})$  do
4:       if  $u_t \notin D(s)$  then
5:          $D(\{s, v_q\}) \leftarrow D(\{s, v_q\}) \setminus \{u_t, v_t\}$ 
6:       end if
7:     end for
8:   end for
9:    $reduced \leftarrow true$ 
10:  while  $reduced$  do
11:     $reduced \leftarrow false$ 
12:    for  $\{u_q, v_q\} \in E_q$  do
13:      for  $\{u_t, v_t\} \in D(\{u_q, v_q\})$  do
14:        if  $u_t \notin D(u_q)$  OR  $v_t \notin D(v_q)$  then
15:           $D(\{u_q, v_q\}) \leftarrow D(\{u_q, v_q\}) \setminus \{u_t, v_t\}$ 
16:           $reduced \leftarrow true$ 
17:        end if
18:      end for
19:    end for
20:    for  $\{u_q, v_q\} \in E_q$  do
21:      for  $u_t \in D(u_q)$  do
22:        if  $\nexists \{x, y\} \in D(\{u_q, v_q\}) : x = u_t$  then
23:           $D(u_q) \leftarrow D(u_q) \setminus u_t$ 
24:           $reduced \leftarrow true$ 
25:        end if
26:      end for
27:    end for
28:  end while
29: end procedure

```

- Discard target vertices of the starting domain is no supporting path is found
- For each discarded vertex, remove its edges from the edge domains
- Continue to **refine**

ArcMatch

Algorithm 6 Backtracking procedure for searching all the occurrences starting from a given target vertex $\hat{\theta}[1]$ by following the variable ordering θ .

Input:
 a query vertex ordering θ ,
 a list $\hat{\theta}$ of matching target vertices according to the query vertex ordering,
 the query edge set E_q ,
 and the vertex and edge domains D .

```

1: procedure SEARCHOCCURRENCES( $\theta, \hat{\theta}, E_q, D$ )
2:    $i \leftarrow |\hat{\theta}|$ 
3:    $s \leftarrow \underset{x}{\theta[\operatorname{argmin}\{\{u, v\} \in D(\{\theta[x], \theta[i]\}) : 1 \leq x < i\}]}$ 
4:   for  $\{u_t, v_t\} \in D(\{\theta[i], \theta[i]\}) : u_t = M(s) \text{ AND } v_t \notin \hat{\theta}$  do
5:      $feasible \leftarrow true$ 
6:     for  $\{\theta[i], \theta[j]\} \in E_q : j < i$  do
7:       if  $\{v_t, \hat{\theta}[j]\} \notin D(\{\theta[i], \theta[j]\})$  then
8:          $feasible \leftarrow false$ 
9:         break
10:      end if
11:    end for
12:    if  $feasible$  then
13:       $\hat{\theta}[i] \leftarrow v_t$ 
14:      if  $|\theta| = |\hat{\theta}|$  then
15:        report match  $M = ((\theta[i], \hat{\theta}[i]) : 1 \leq i \leq |\theta|)$ 
16:      else
17:         $SearchOccurrences(\theta, \hat{\theta}, E_q, D)$ 
18:      end if
19:       $\hat{\theta} \leftarrow \hat{\theta}[1 \dots i - 1]$ 
20:    end if
21:  end for
22: end procedure
    
```

In each recursive step, a **parent** query vertex s is chosen **dynamically**. Such a parent is the currently matched vertex which **minimizes the number of candidates** to the current state i (line 3). The minimization is obtained by counting the query edges that are in the domain $D(\{s, \theta[i]\})$

Algorithm 7 Matching procedure

Input:
 the query graph $G_q = (V_q, E_q)$,
 the target graph $G_t = (V_t, E_t)$,
 the path reduction parameter lp ,
 the vertex and edge domains D ,
 and the edge labelling function β .

```

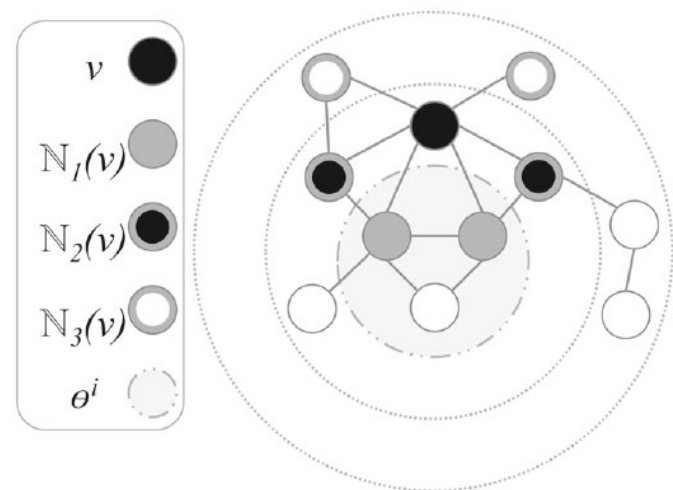
1: procedure MATCH( $G_q, G_t, lp, D, \beta$ )
2:   for  $q_i \in V_q$  do ▷ initial vertex domains
3:      $D(q_i) \leftarrow \{v \in V_t : \alpha(q_i) = \alpha(v) \text{ AND } deg(q_i) \leq deg(v)\}$ 
4:   end for
5:    $ArcConsistency(E_t, E_q, D, \beta)$  ▷ arc consistency over vertex domains
6:   for  $\{q_i, q_j\} \in E_q$  do ▷ initial edge domains
7:      $D(\{q_i, q_j\}) \leftarrow \{(x, y) \in E_t : x \in D(q_i) \text{ AND } y \in D(q_j) \text{ AND } \beta(\{x, y\}) = \beta(\{q_i, q_j\})\}$ 
8:   end for
9:   for  $q_i \in V_q$  do ▷ path-based reduction
10:      $\omega \leftarrow (q_i)$ 
11:      $PathReduction(\omega, lp, D)$ 
12:   end for
13:    $\theta \leftarrow variable\_ordering(G_q, D)$  ▷ variable ordering
14:   for  $u_t \in D(\theta[1])$  do ▷ backtracking
15:      $\hat{\theta} \leftarrow (u_t)$ 
16:      $SearchOccurrences(\theta, \hat{\theta}, E_q, D)$ 
17:   end for
18: end procedure
    
```

ArcMatch

- $N_1(v) = |\mathbb{N}_1(v)|$
- $N_2(v) = |\mathbb{N}_2(v)|$
- $N_3(v) = |\mathbb{N}_3(v)|$
- $N_4(v) = deg(v)$
- $N_5(v) = |D(v)|$

Define **search strategy** according ot these sets, wth two exception:

- Query vertices with **singleton** are put before
- **Pheriferal** query vertices are put after



$$\begin{aligned}\mathbb{N}_1(v) &= \{u \in \theta_i : \{u, v\} \in E\} \\ \mathbb{N}_2(v) &= \{u \in N(\theta_i) : \{u, v\} \in E\} \\ \mathbb{N}_3(v) &= N(v) \setminus \{\mathbb{N}_1(v) \cup \mathbb{N}_2(v)\}\end{aligned}$$

ArcMatch -Results

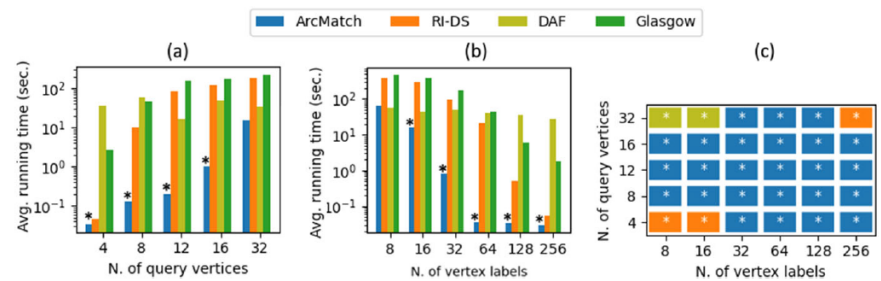


Fig. 7 Comparison of it with other systems that allow an arbitrarily large number of returned matches on the Protein-Protein Interaction (PPI) graphs with vertex labels but ignoring edge labels. Subfigures **a** and **b** depict the average running times, grouping queries by the number of query vertices along with varying the numbers of vertex labels, and by the number of vertex labels along with varying numbers of query vertices, respectively. Subfigure **c** shows the algorithm with the lowest average running time, grouping queries by the number of query vertices and the number of vertex labels. Ends to dominate with 8 or more query vertices, as long as there are a sufficient number of vertex labels

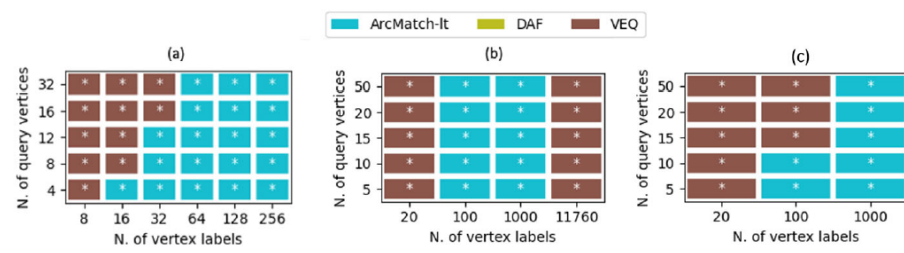


Fig. 10 The figure shows the algorithm with the lowest average running time, grouping queries by the number of query vertices and the number of vertex labels for the PPI benchmark (subfigure **a**), for the co-authorship benchmark (subfigure **b**), and for the email benchmark (subfigure **c**). It outperforms the other approaches for a sufficiently high number of vertex labels when they are uniformly distributed (thus excluding the 11760-labels labeling of the co-authorship network) over the target graph vertices

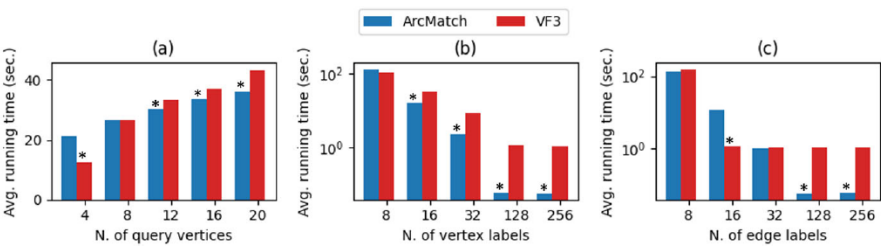


Fig. 14 Comparison of it with VF3 when searching for all the induced subgraph isomorphism occurrences of queries over the Protein-Protein Interaction benchmark. Average running times are grouped by: **a** the number of query vertices along with varying numbers of vertex and edge labels; **b** the number of vertex labels along with varying numbers of query vertices and edge labels. **c** the number of edge labels along with varying numbers of query vertices and vertex labels. When the number of vertex labels is greater than 8 or the number of query vertices is greater than 8, it is faster, thanks to the effectiveness of its reduction techniques

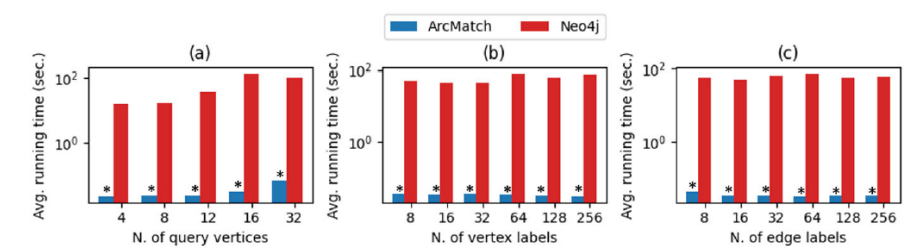


Fig. 15 Comparison of it with Neo4j when searching all the induced subgraph isomorphism occurrences of queries over the PPI (Protein-Protein Interaction) benchmark. Average running times are grouped by: **a** the number of query vertices along with varying numbers of vertex and edge labels; **b** the number of vertex labels along with varying numbers of query vertices and edge labels. **c** the number of edge labels along with varying numbers of query vertices and vertex labels

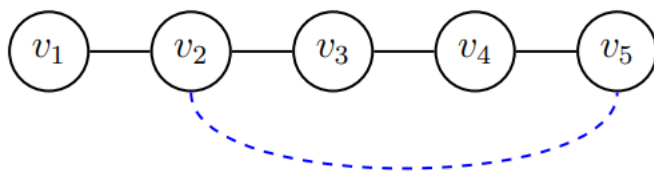
ArcMatch – Extension to ties

Thanks to Maria Stefania Scaletta

Algoritmo 8 Algoritmo per la ricerca delle cravatte

```
Input:
  un percorso  $\omega$ ,
  il parametro di riduzione del percorso  $l_p$ ,
  e i domini dei vertici e degli archi  $D$ .

1: procedure PATHREDUCTION( $\omega, l_p, D$ )
2:    $i \leftarrow |\omega|$ 
3:    $u \leftarrow \omega[i]$ 
4:   extended  $\leftarrow$  false
5:   for  $v \in N(u)$  do
6:     if  $v \notin \omega$  then                                     ▷ Try to extend it
7:       extended  $\leftarrow$  true
8:        $\omega[i+1] \leftarrow v$ 
9:       if  $i+1 = l_p$  then
10:        VerifyPath( $\omega, \text{false}, \text{True}, \dots$ )
11:      end if
12:      PathReduction( $\omega, l_p, D$ )
13:       $\omega \leftarrow \omega[1 \dots i]$ 
14:   else
15:     if  $v = \omega[1]$  then                                     ▷ It's a ring
16:       VerifyPath( $\omega, \text{true}, \text{false}, 1, D$ )
17:     else
18:       if  $i \geq 4$  then                                       ▷ It's a tie
19:          $v \leftarrow \omega[i]$     ▷ Assumendo che  $v$  sia il nodo finale della
cravatta, ma la logica qui è un po' ambigua rispetto all'immagine
20:         for  $k$  from 2 to  $i-2$  do
21:           if  $v = \omega[k]$  then
22:             VerifyPath( $\omega, \text{false}, \text{true}, k, D$ )
23:             break
24:           end if
25:         end for
26:       end if
27:     end if
28:   end if
29: end for
30: if not extended then
31:   VerifyPath( $\omega, \text{false}, \text{false}, 0, D$ )
32: end if
33: end procedure
```

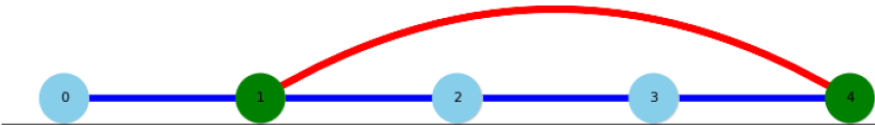
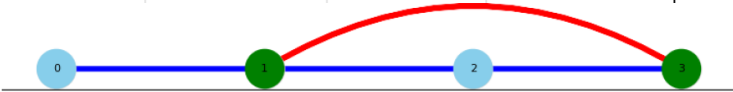


```
1: procedure VERIFYPATHDFS( $\omega, \omega', isRing, isTie, k, D$ )
2:   if  $|\omega'| = |\omega|$  then return true
3:   else
4:      $i \leftarrow |\omega'|$ 
5:      $u_q \leftarrow \omega[i]$ 
6:      $v_q \leftarrow \omega[i+1]$ 
7:     for  $\{(u_t, v_t) \in D(\{u_q, v_q\}) : u_t = \omega'[i]\}$  do
8:       if  $isRing$  AND  $(|\omega'| = |\omega| - 1)$  then
9:         if  $v_t = \omega'[1]$  then return true
10:       end if
11:     end if
12:     if  $isTie$  AND  $(|\omega'| = |\omega| - 1)$  then
13:       if  $v_t = \omega'[k]$  then return true
14:     end if
15:   end if
16:   if  $v_t \notin \omega'$  then                                     ▷ Estensione sul cammino
17:      $\omega'[i+1] \leftarrow v_t$ 
18:     if VERIFYPATHDFS( $\omega, \omega', isRing, isTie, D$ ) then return
true
19:   end if
20:    $\omega' \leftarrow \omega'[1 \dots i]$ 
21: end if
22: end for
23: return false
24: end if
25: end procedure
```

ArcMatch – Extension to ties

Thanks to Maria Stefania Scalella

Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time (MT)	Vecchio MT	Vecchio MT / MT
10	8	4	4	0,00142	0,00020	0,138972897
20	19	4	4	0,00228	0,00059	0,257620017
50	73	4	4	0,01449	0,00376	0,259475365
100	262	4	4	0,24343	0,12750	0,523772748
200	985	4	4	5,46614	2,98142	0,545434255
500	6.302	4	4	135,04100	96,88320	0,717435446
1.000	25.270	4	4	1136,91000	1335,36500	1,1746
2.000	99.910	4	4	6890,16705	8804,25057	1,2778
5.000	625.996	4	4	126037,82440	169056,73661	1,3413
10.000	2.498.820	4	4	1135983,85457	1603975,78185	1,4120

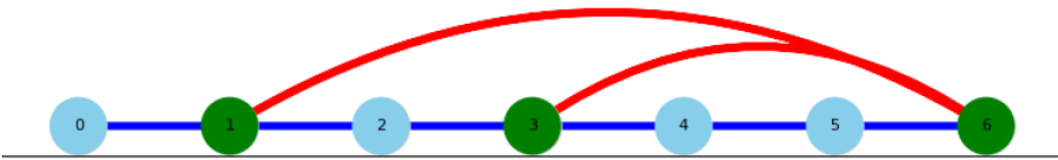
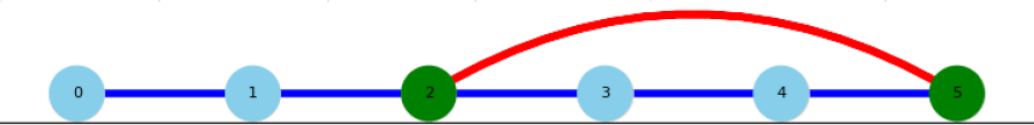


Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time (MT)	Vecchio MT	Vecchio MT / MT
10	8	5	5	0,00131	0,00020	0,155645544
20	19	5	5	0,00335	0,00077	0,230653875
50	73	5	5	0,02643	0,00903	0,341846913
100	262	5	5	1,15532	0,81230	0,703094381
200	985	5	5	9,92387	10,15930	1,023723608
500	6.302	5	5	105,75000	139,69900	1,321030733
1.000	25.270	5	5	1057,50000	1700,50000	1,608037825
2.000	99.910	5	5	10575,00000	11500,75000	1,087541371
5.000	625.996	5	5	105750,00000	215750,00000	2,040189125
10.000	2.498.820	5	5	1500000,00000	1999977,99000	1,33331866

ArcMatch – Extension to ties

Thanks to Maria Stefania Scalella

Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time	Vecc MT	vecchio/nuovo
10	8	6	6	0,00087	0,00069	0,790348752
20	19	6	6	0,00181	0,00141	0,776723743
50	73	6	6	0,02720	0,02178	0,800697115
100	262	6	6	1,03086	1,17973	1,144413402
200	985	6	6	8,97253	12,57560	1,401566782
500	6.302	6	6	284,24800	362,65200	1,275829557
1.000	25.270	6	6	1800,50000	2100,75000	1,166759234
2.000	99.910	6	6	15111,76890	18501,22000	1,224292148
5.000	625.996	6	6	100691,42000	125000,11000	1,24141769
10.000	2.498.820	6	6	850010,77000	1000457,49000	1,176993899

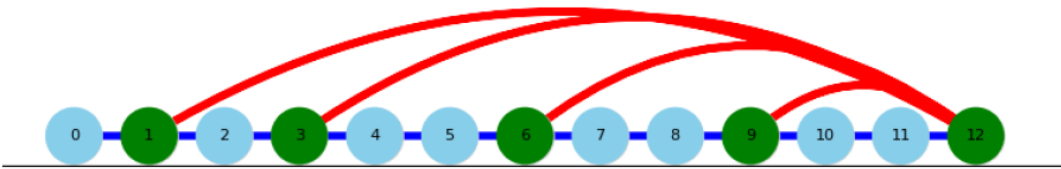


Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time	Vecc MT	vecchio/nuovo
10	8	7	8	0,00211	0,00036	0,168889785
20	19	7	8	0,00207	0,00033	0,158699005
50	73	7	8	0,04044	0,02207	0,545741863
100	262	7	8	2,54563	3,91719	1,538790005
200	985	7	8	18,52520	34,10230	1,840860018
500	6.302	7	8	450,00000	723,21400	1,607142222
1.000	25.270	7	8	1498,98000	2000,77000	1,3347543
2.000	99.910	7	8	10000,55500	12476,84000	1,247614757
5.000	625.996	7	8	150500,68400	180001,68400	1,196019043
10.000	2.498.820	7	8	1501090,27900	2000000,16280	1,332365009

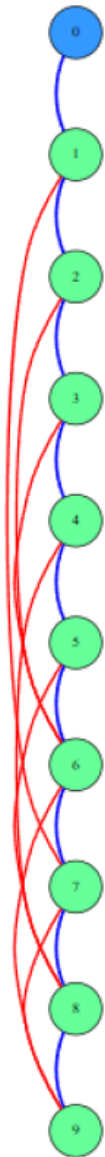
ArcMatch – Extension to ties

Thanks to Maria Stefania Scaletta

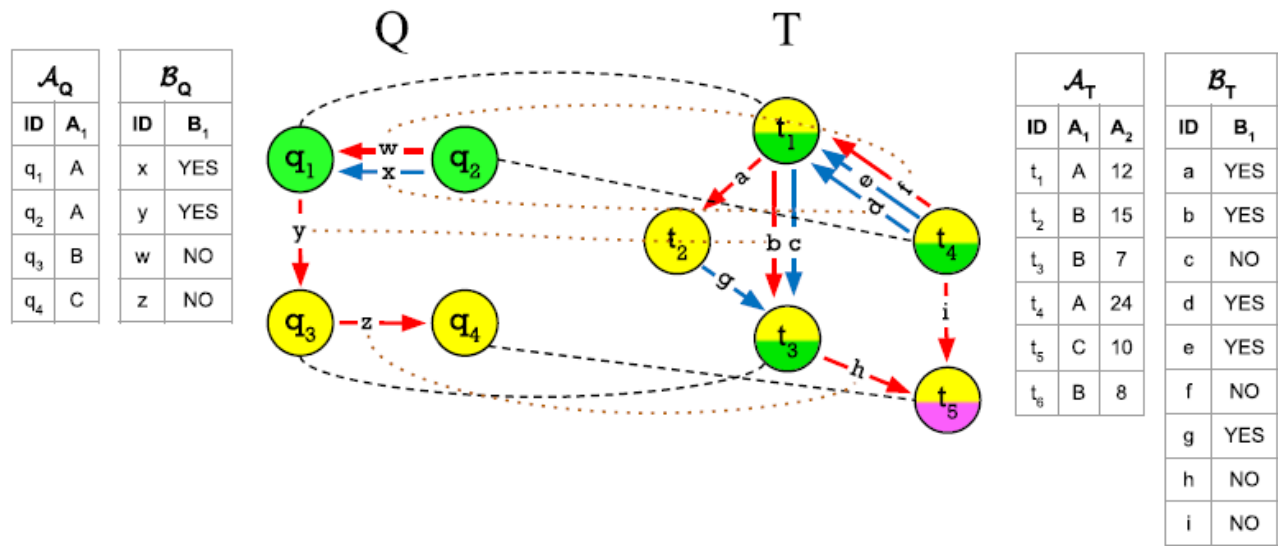
Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time	Vecc MT	vecchio/nuovo
10	8	13	16	0,0000	0,0000	1,274650425
20	19	13	16	0,0000	0,0000	1,339999497
50	73	13	16	0,0623	0,0585	0,939340342
100	262	13	16	8,5000	14,8796	1,750541176
200	985	13	16	60,9998	100,6676	1,650293935
500	6.302	13	16	1800,3432	2500,8732	1,389109143
1.000	25.270	13	16	4500,9631	6000,2367	1,333100620
2.000	99.910	13	16	35000,9862	45000,9876	1,285706275
5.000	625.996	13	16	400000,0019	500000,4341	1,250001079
10.000	2.498.820	13	16	4000000,6533	5000000,2486	1,249999858



Nodi Target	Archi Target	Nodi Query	Archi Query	Matching Time	Vecc MT	vecchio/nuovo
10	8	10	16	0,0001	0,0000	0,193548886
20	19	10	16	0,0000	0,0000	2,826073874
50	73	10	16	0,0784	0,0528	0,673201065
100	262	10	16	8,7596	12,0976	1,381067629
200	985	10	16	50,3657	75,0000	1,489108659
500	6.302	10	16	255,4859	350,0000	1,369938615
1.000	25.270	10	16	1200,3758	1500,0000	1,249608664
2.000	99.910	10	16	8000,4748	10100,5885	1,262498633
5.000	625.996	10	16	77000,3748	101005,0001	1,311746863
10.000	2.498.820	10	16	803050,5960	1004506,7840	1,250863631



MultiGraphMatch



MultiGraphMatch

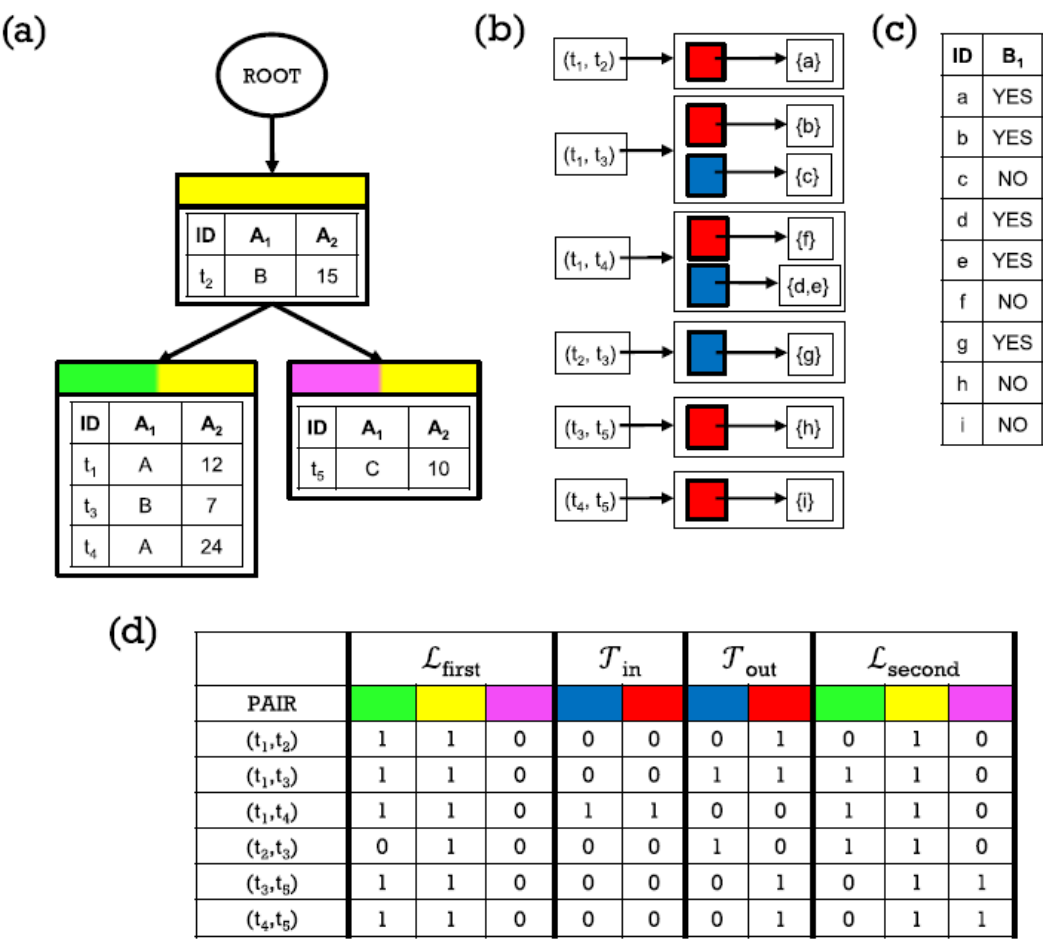


Fig. 4. Indexing data structures associated to the target T of Figure 3: (a) Node label graph, (b) Edge types map, (c) Edge properties table, and (d) Bit signature matrix.

MultiGraphMatch

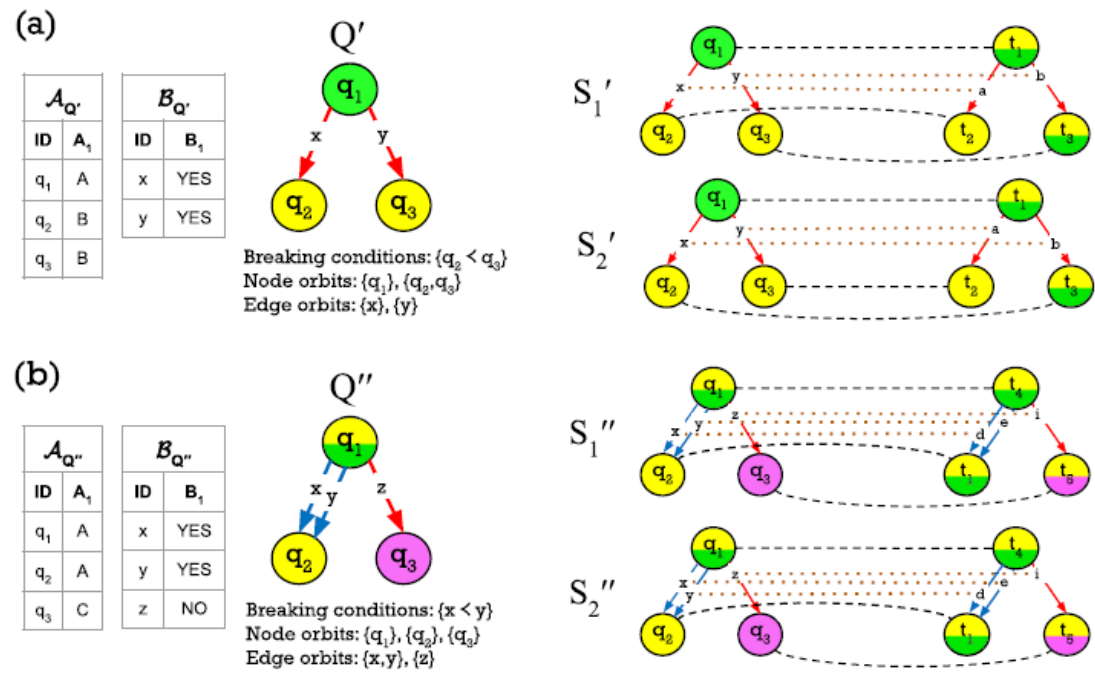
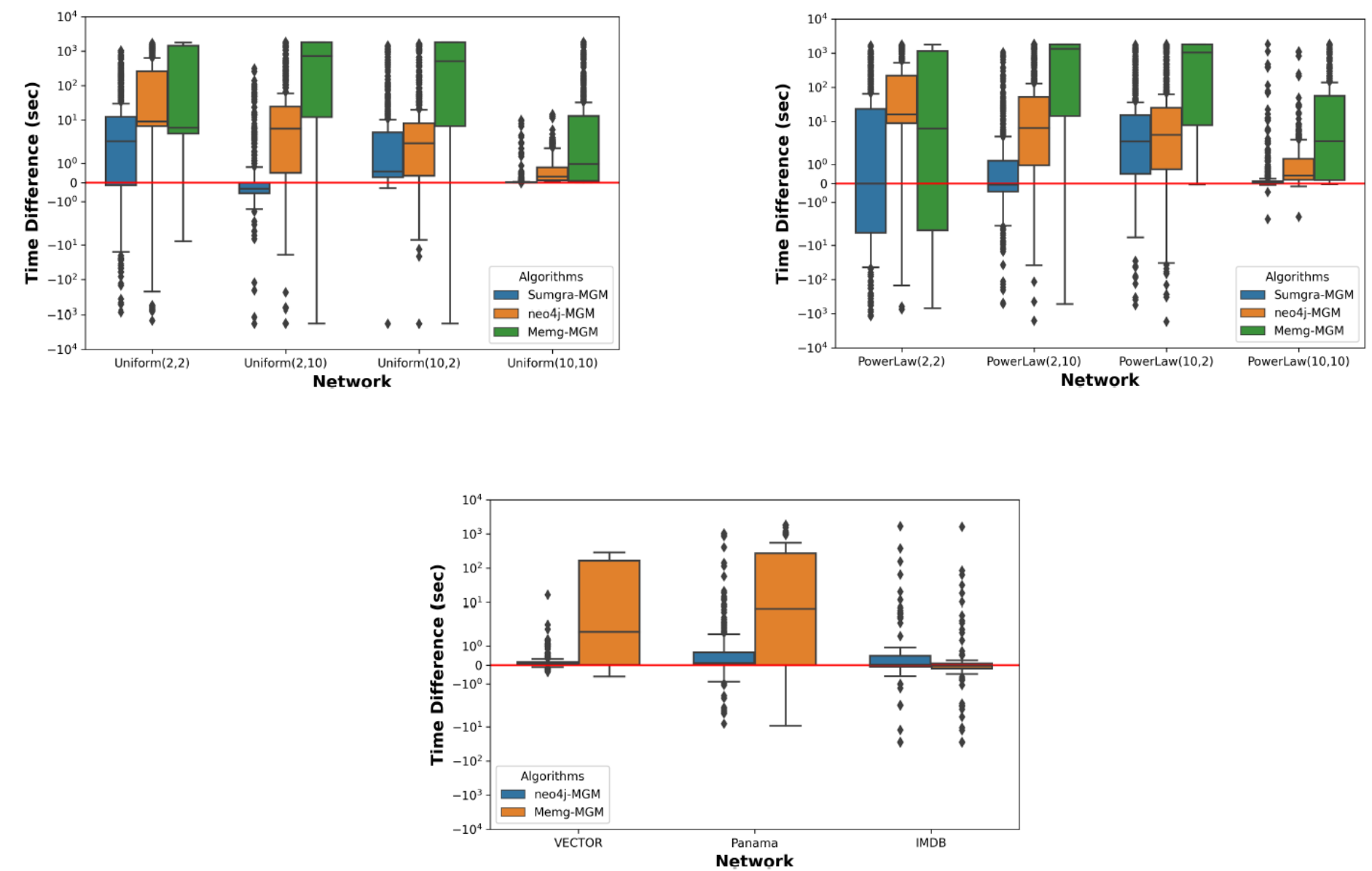


Fig. 5. Example of symmetry breaking condition on nodes and edges and application of conditions on matching queries Q' and Q'' with the target multigraph T of Figure 3. (a) In query Q' nodes q_2 and q_3 are in the same orbit, thus yielding the breaking condition $q_2 < q_3$. Assuming that $t_2 < t_3$ in T , after the application of such condition on T , solution S'_2 is discarded. (b) In query Q'' edges x and y are in the same orbit, thus yielding the breaking condition $x < y$. Assuming that $d < e$ in T , after the application of such condition on T , solution S''_2 is discarded.

MultiGraphMatch



The timeline and references

Giovanni Micale, Antonio Di Maria, Roberto Grasso, Vincenzo Bonnici, Alfredo Ferro, Dennis Shasha, Rosalba Giugno, and Alfredo Pulvirenti.

MultiGraphMatch: a subgraph matching algorithm for multigraphs.
ACM Trans. Knowl. Discov. Data (2025). <https://doi.org/10.1145/3728361>

Bonnici V., Grasso R., Micale G., di Maria A., Shasha D., Pulvirenti A., Giugno R.
ArcMatch: high-performance subgraph matching for labeled graphs by exploiting edge domains.
Data mining and knowledge discovery (2024). DOI: 1007/s10618-024-01061-8.

Boldini, G., Diana, A., Arceri, V., Bonnici, V., Bagnara, R. (2024). A Machine Learning Approach for Source Code Similarity via Graph-Focused Features. In: Nicosia, G., Ojha, V., La Malfa, E., La Malfa, G., Pardalos, P.M., Umeton, R. (eds) Machine Learning, Optimization, and Data Science. LOD 2023. Lecture Notes in Computer Science, vol 14505. Springer, Cham. https://doi.org/10.1007/978-3-031-53969-5_5

Licheri, N., Bonnici, V., Beccuti, M. and Giugno, R.
GRAPES-DD: exploiting decision diagrams for index-driven search in biological graph databases.
BMC bioinformatics, 22(1), pp.1-24, 2021. Doi: 10.1186/s12859-021-04129-0

Licheri, N., Amparore, E., Bonnici, V., Giugno, R., & Beccuti, M.
An entropy heuristic to optimize decision diagrams for index-driven search in biological graph databases.
Proceedings of the CIKM 2021 Workshops co-located with 30th ACM International Conference on Information and Knowledge Management (CIKM 2021). Gold Coast, Queensland, Australia, November 1-5, 2021

Aparo A., Bonnici V., Micale G., Ferro A., Shasha D., Pulvirenti A., Giugno R.
Fast Subgraph Matching Strategies based on Pattern-only Heuristics.
Interdisciplinary Sciences: Computational Life Sciences, 11(1), 21-32, 2019. doi: 10.1007/s12539-019-00323-0.

Bombieri N., Bonnici V., Giugno R.
Parallel Searching on Biological Networks.
Euromicro International Conference on Parallel, Distributed, and Network-Based Processing PDP2019. Pavia February 13-15, 2019, pp. 1-12

Bonnici V., Giugno R., Bombieri N.
An Efficient Implementation of a Subgraph Isomorphism Algorithm for GPUs.
2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Madrid, Spain, December 3-6, 2018, pp. 2674-2681. doi:10.1109/BIBM.2018.8621444

Bonnici, V., Busato, F., Aldegheri, S., Akhmedov, M., Cascione, L., Carmena, A.A., Bertoni, F., Bombieri, N., Kwee, I., Giugno, R.
cuRnet: an R package for graph traversing on GPU
BMC bioinformatics 19.10 (2018): 221. doi: 10.1186/s12859-018-2310-3

A. Aparo, V. Bonnici, G. Micale, A. Ferro, D. Shasha, A. Pulvirenti, R. Giugno.
Simple Pattern-only Heuristics Lead To Fast Subgraph Matching Strategies on Very Large Networks.
PACBB'18: 12th International Conference on Practical Applications of Computational Biology & Bioinformatics.
Toledo (Spain). 20-22 Giugno, 2018. doi: 10.1007/978-3-319-98702-6_16

Bonnici V., Caligola S., Aparo A., Giugno R.
Centrality Speeds the Subgraph Isomorphism Search Up in Target Aware Contexts. In Computational Intelligence Methods for Bioinformatics and Biostatistics. CIBB 2018. 5-8 September 2018, Caparica, Portugal. Lecture Notes in Computer Science, vol 11925. Springer, Cham. Dpi: https://doi.org/10.1007/978-3-030-34585-3_3.

V. Bonnici, R. Giugno.
On the variable ordering in subgraph isomorphism algorithms.I
EEE/ACM Transaction on Computational Biology and Bioinformatics. 01/2017.
doi:10.1109/TCBB.2016.2515595.

V. Bonnici, F. Busato, G. Micale, N. Bombieri, A. Pulvirenti, R. Giugno.
APPAGATO: an Approximate Parallel and stochastic GrAph querying Tool for biological networks.
Bioinformatics, 04/2016. doi:10.1093/bioinformatics/btw223

F. Rinnone, G. Micale, V. Bonnici, G.D. Bader, D. Shasha, A. Ferro, A. Pulvirenti, R. Giugno.
NetMatchStar: an enhanced Cytoscape network querying app.
F1000Research (4), 11/2015. doi:10.12688/f1000research.6656.2

V. Bonnici, R. Giugno, A. Pulvirenti, D. Shasha, A. Ferro.
A subgraph isomorphism algorithm and its application to biochemical data.
BMC Bioinformatics (14). 04/2013. doi:10.1186/1471-2105-14-S7-S13

R. Giugno, V. Bonnici, N. Bombieri, A. Pulvirenti, A. Ferro, D. Shasha.
GRAPES: A Software for Parallel Searching on Biological Graphs Targeting Multi-Core Architectures.
PLoS ONE (8), 10/2013. doi:10.1371/journal.pone.0076911

Bonnici V, Giugno R, Pulvirenti A, Shasha D, Ferro A.
Core algorithms to search in biological structured data .
EMBNET NEWS, vol. 18, ISSN: 2226-6089. 2012

V. Bonnici, A. Ferro, R. Giugno, A. Pulvirenti, D. Shasha.
Enhancing Graph Database Indexing By Suffix Tree Structure.
Lecture Notes in Computer Science. Volume 6282 LNBI, 2010, Pages 195-203. doi.org/10.1007/978-3-642-16001-1_17

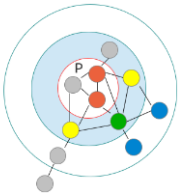
V. Bonnici, A. Ferro, R. Giugno, A. Pulvirenti, D. Shasha.
RelaxGrep: approximate graph searching by query relaxation.
Proceeding on 5th IAPR International Conference on Pattern Recognition in Bioinformatic. Lecture Notes in Bioinformatics, Springer. 22-24 September 2010, Nijmegen, The Netherlands. 2010.

V. Bonnici, A. Ferro, R. Giugno, A. Pulvirenti, D. Shasha.
Enhancing Graph Database Indexing By Suffix Tree Structure.
Proceeding on 5th IAPR International Conference on Pattern Recognition in Bioinformatic Lecture Notes in Bioinformatics, Springer. 22-24 September 2010, Nijmegen, The Netherlands. 2010.

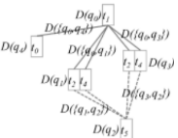
The timeline

One-to-One

(2013-2019) **RI and RI-DS**
A search strategy for solving exact subgraph isomorphism that is based on maximizing pruning power and reduce computational costs



(2024) **ArcMatch**
High-performance subgraph matching for labeled graphs by exploiting edge domains.



(2025) **MultiGraphMatch**
:A subgraph matching algorithm for multigraphs



(2019) **GRASS**
A parallel implementation of RI. A first approach for subgraph isomorphism on GPGPU.

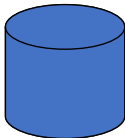


(2015) **APPAGATO**
Approximate subgraph isomorphism on GPGPU.

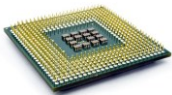
(to appear) **RIBS**
Approximate subgraph isomorphism via non-redundant path enumeration

One-to-Many

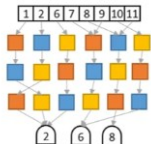
(2010) **GraphGrepSX**
Enhancing graph database indexing.



(2013) **GRAPES**
Parallel searching in biological graphs targeting multi-core architectures.



(2021) **GRAPES-DD**
Exploiting decision diagrams for index-driven search in biological graphs

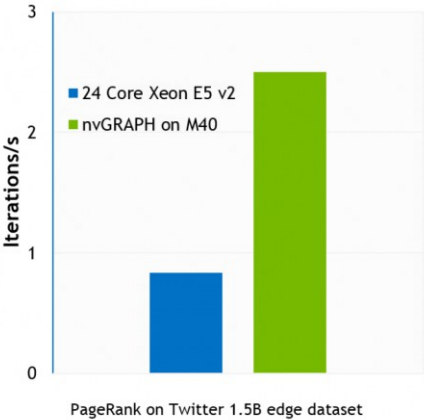


GP-GPU architecture and memory (un)coalescing



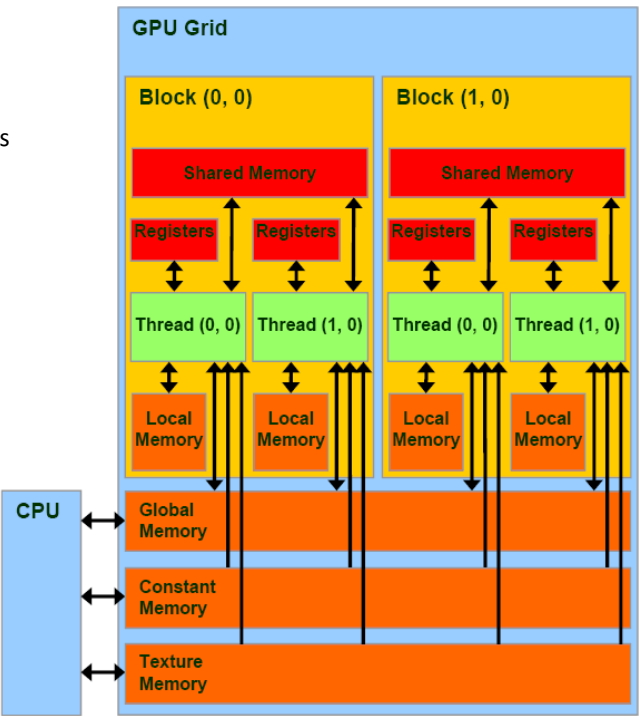
NVIDIA Tesla M40
12GB GDDR5 7 Tflops
€ 5.920

nvGRAPH: 3x Speedup

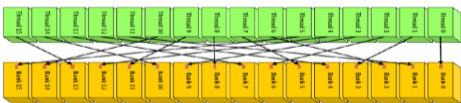
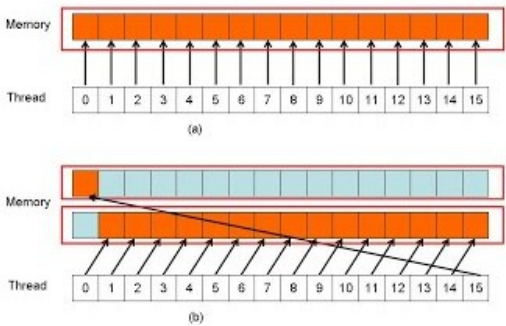
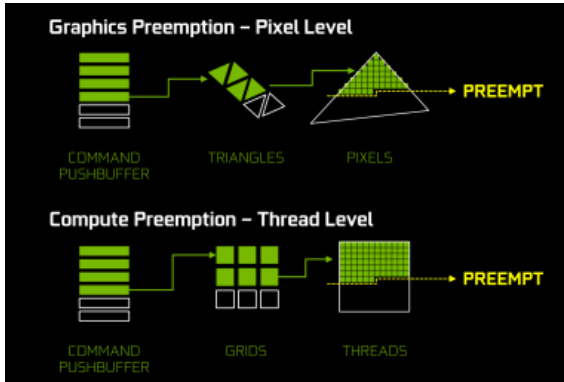


CPU System:
server w/ 2x12-core Xeon E5-2697 CPU, 30M
Cache, 2.70 GHz, 512 GB RAM

<https://developer.nvidia.com/nvgraph>



Memory uncoalescing
-> thread imbalance
-> bad preemption



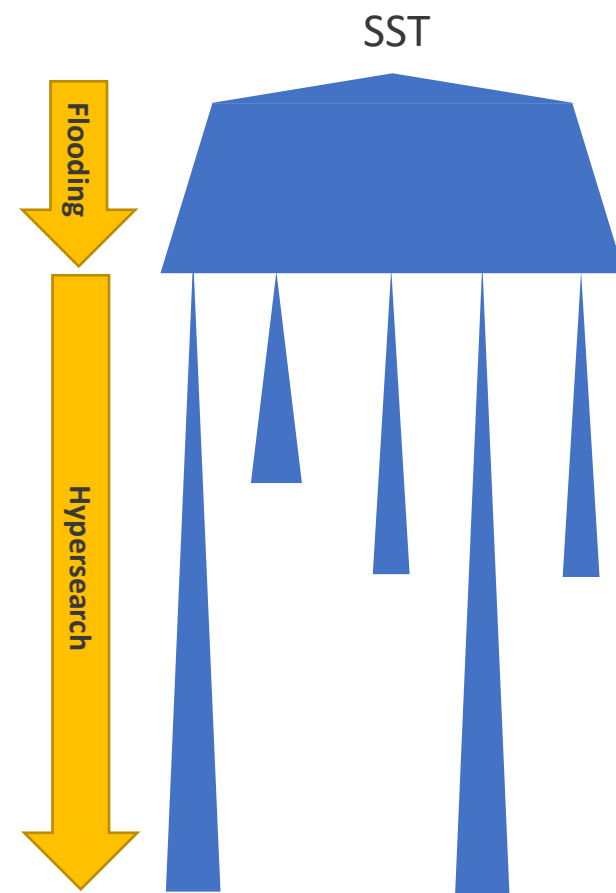
GRASS – (not only) RI on GPU

GRASS uses the static **RI** ordering to define the **Search-Space Tree** (SST)

Furthermore:

GRASS splits the visit of the SST into two distinct **phases**

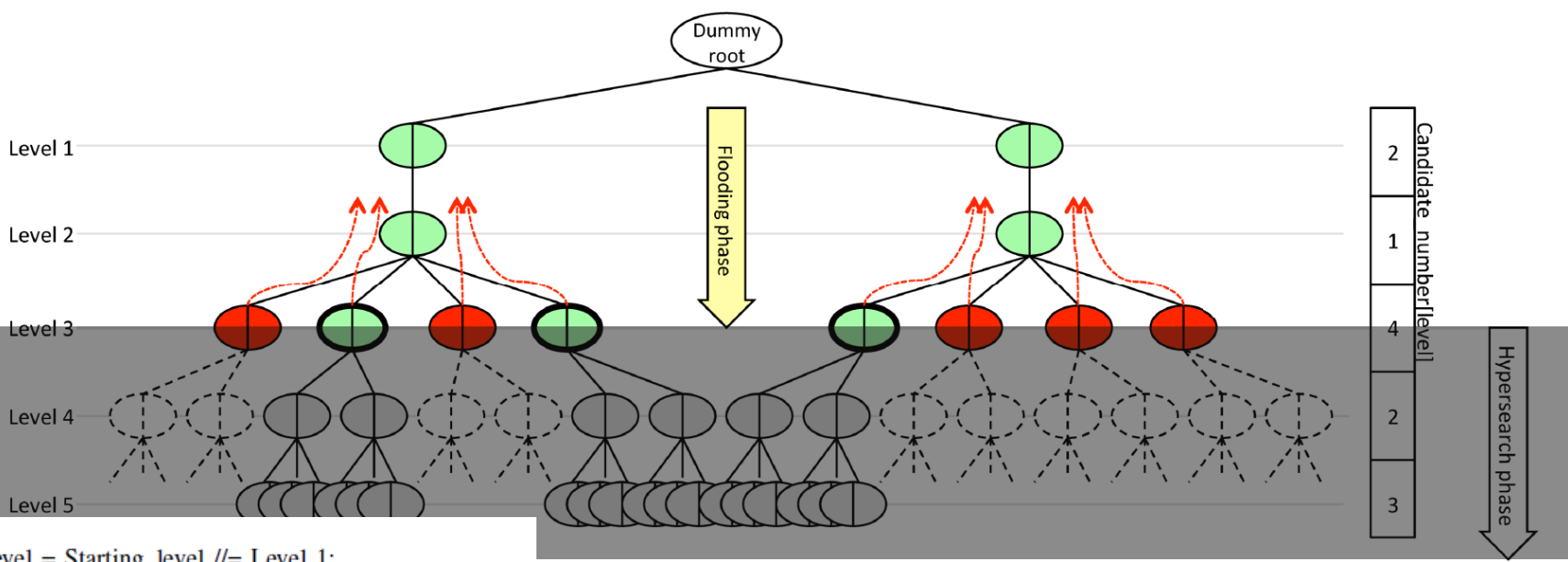
- The **initial dense levels** of the SST are visited by the **flooding phase**
 - A massive scanning/verification of the space is performed by all the GPU blocks.
 - High redundancy is admitted in order to avoid memory uncoalescing
- The **deep SST branches** are visited by the **hypersearch phase**
 - A combined DFS/BFS visit of the branches produced by the flooding phase is performed by assigning each branch to a specific thread block.
 - Each block works independently.



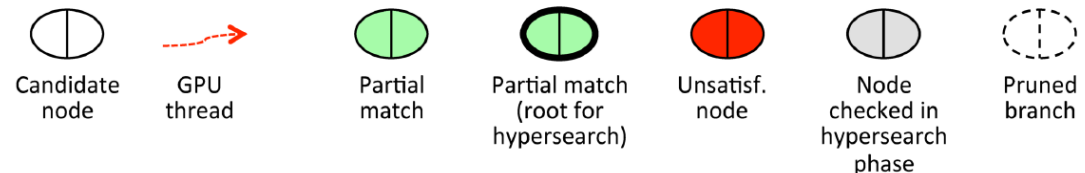
GRASS – Flooding phase

The initial dense levels of the SST are visited by the flooding phase

- A massive scanning/verification of the space is performed by all the GPU blocks
- High redundancy is admitted in order to avoid memory uncoalescing

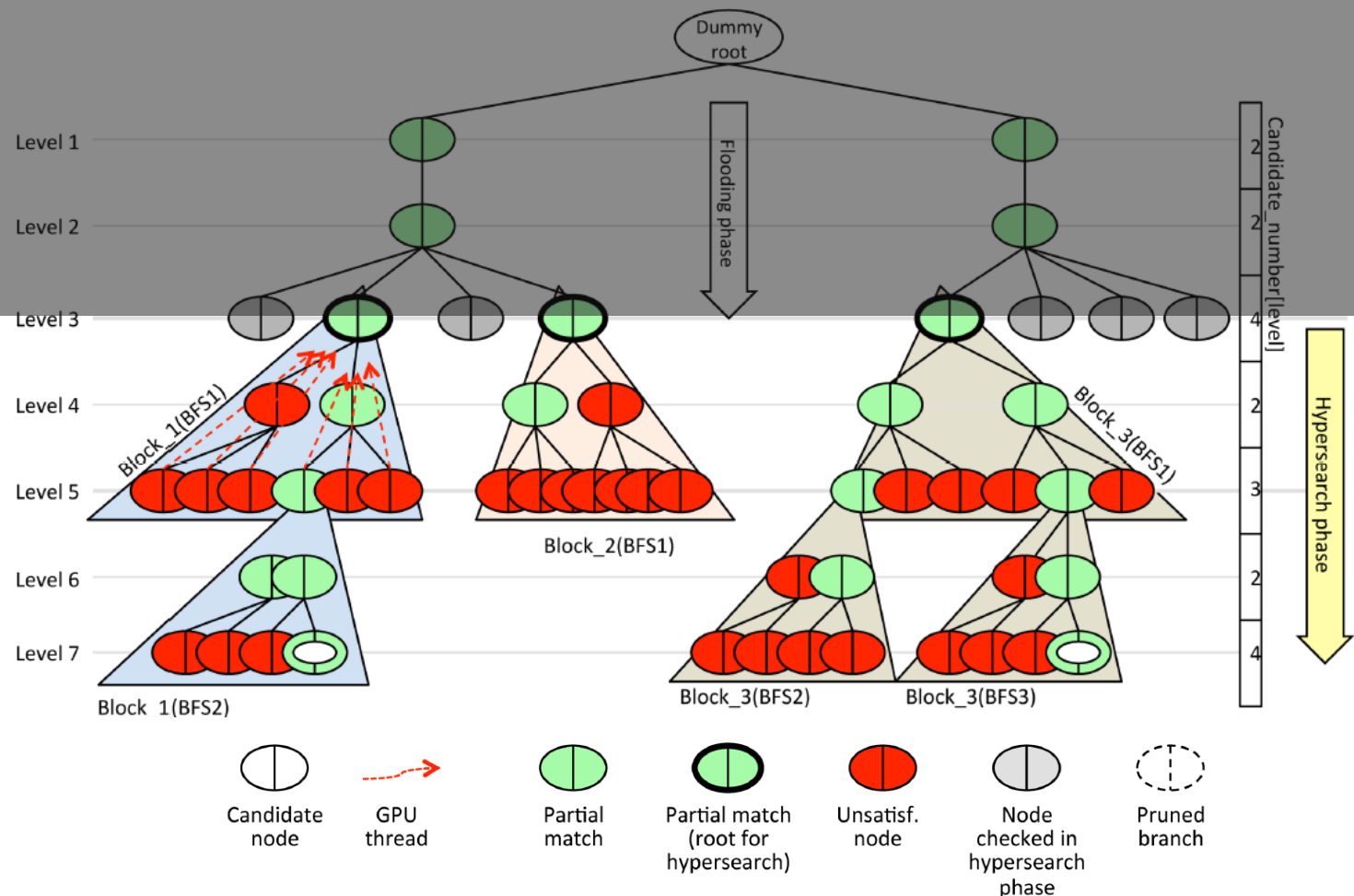


```
1: Flooding_level = Starting_level // Level 1;  
2: Candidate_counter = 1;  
3: while Candidate_counter ≤ Max_threads do  
4:   Candidate_counter *= Candidate_number[Flooding_level];  
5:   Flooding_level++;  
6: end while
```

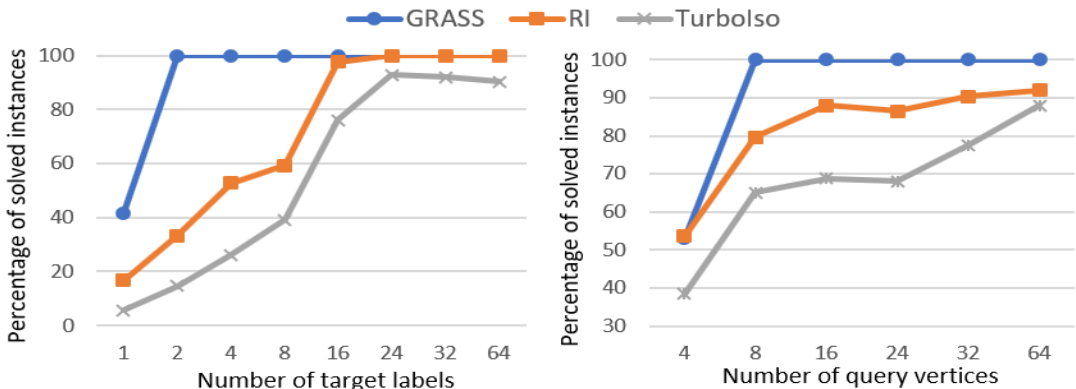


GRASS – Hypersearch

- The deep SST branches are visited by the hypersearch phase
- A combined DFS/BFS visit of the branches produced by the flooding phase is performed by assigning each branch to a specific thread block. Each block works independently.

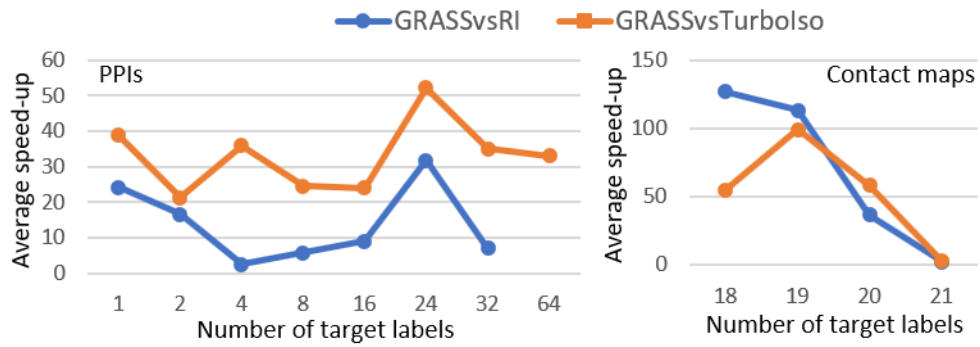
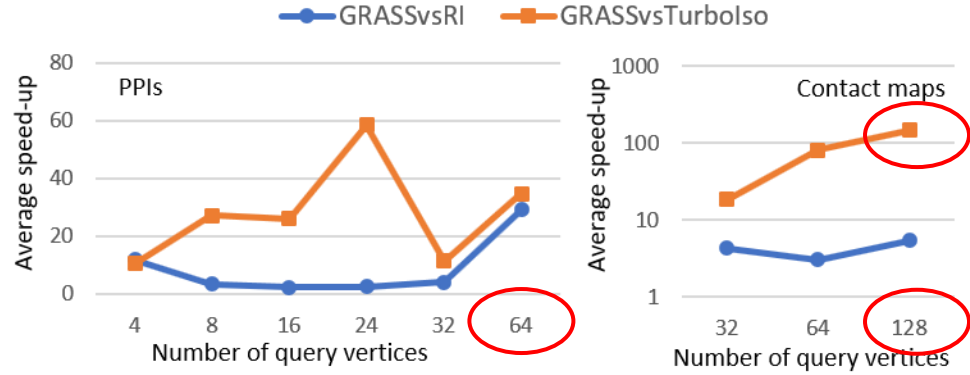


GRASS – Results on biological graphs



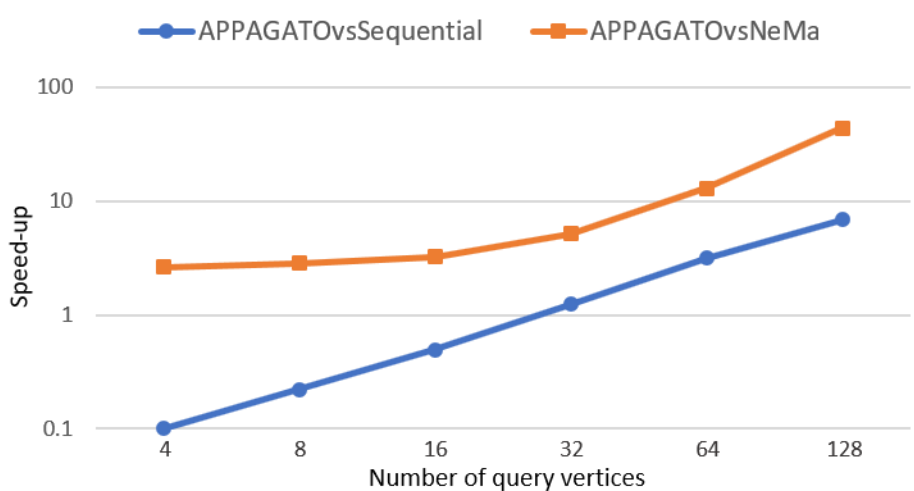
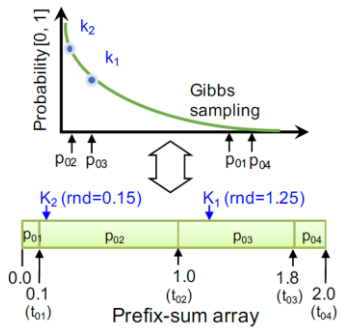
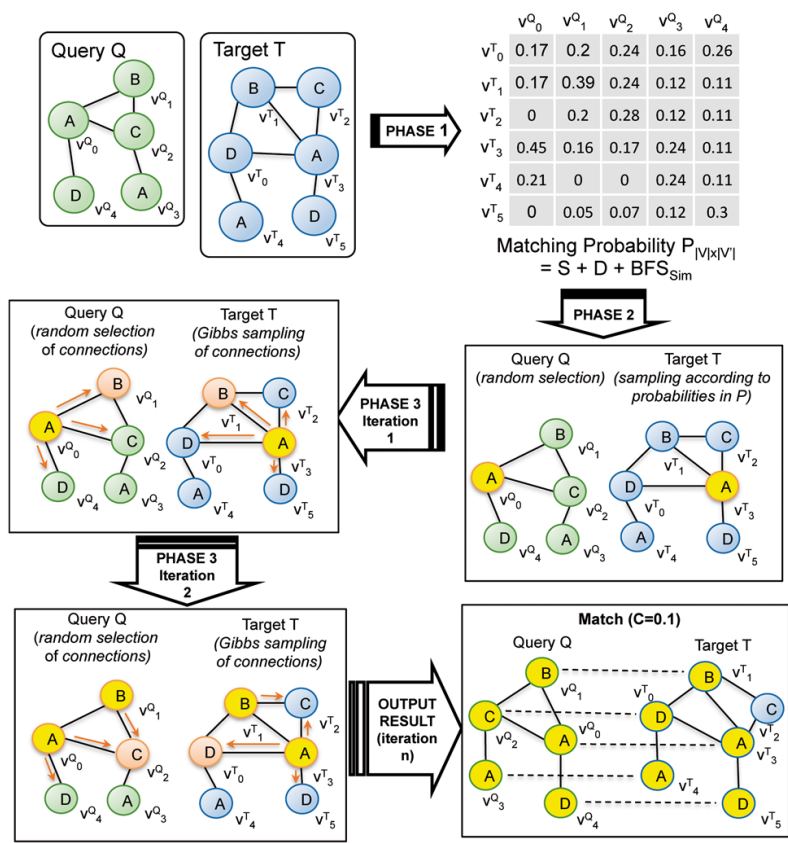
Percentage of **solved SubGI instances**, within a 10 minutes timeout, by the three compared tools over the **PPI dataset**. Results are grouped **by number of target labels** and **by number of query vertices**, respectively on the left and right charts.

Average **speed-up** of GRASS w.r.t. RI and Turbolso over the two **real datasets** (PPIs on the left side and contact maps on the right side). Averages are computed by grouping total running times of SubIG instances **by number of query vertices**.



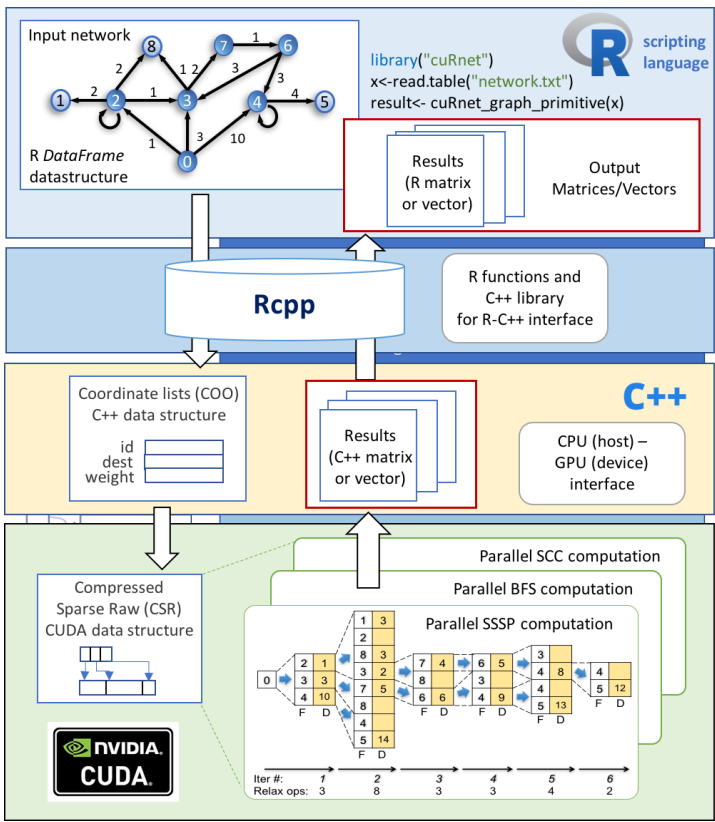
Average **speed-up** of GRASS w.r.t. RI and Turbolso over the two **real datasets** (PPIs on the left side and contact maps on the right side). Averages are computed by grouping total running times of SubIG instances **by number of target labels**.

APPAGATO – Approximate subgraph isomorphism on GP-GPU



cuRnet

cuRnet is a package that provides a wrap of parallel graph algorithms developed in **CUDA** for the R environment. It makes available GPU solutions to R end-users in a **transparent** way, by including basic data structures for representing graphs, and automatic front-end and back-end data conversion.



R scripts
C++ source code
CUDA kernels

```
library("cuRnet")
x <- read.table("network.txt", header=TRUE)
...
distances <- cuRnet_sssp(x)
```

```
cuRnet_sssp <- function(dataFrame, from) {
  .Call('cuRnet_sssp', PACKAGE = 'cuRnet', dataFrame, from)
}
```

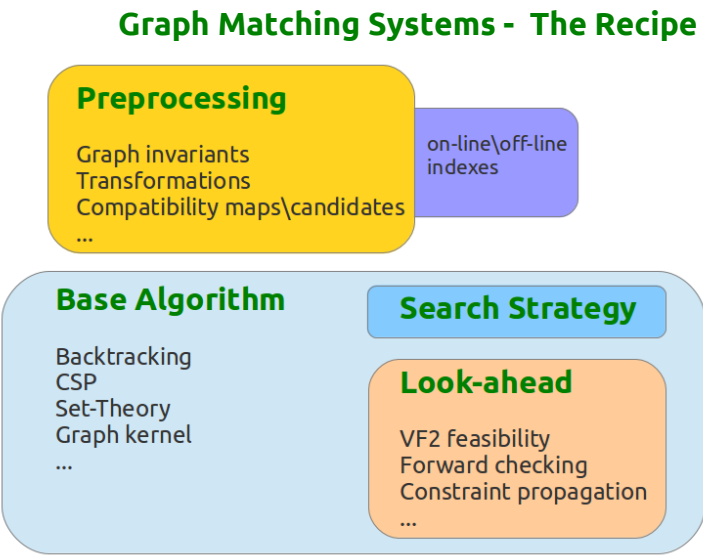
```
//[[Rcpp::export]]
Rcpp::NumericMatrix
cuRnet_sssp(Rcpp::DataFrame dataFrame, Rcpp::CharacterVector from){
  ...
  HostGraph graph(dataFrame, EdgeType::DIRECTED); //building AdjList in C++
  ...
  distances = cu_sssp(graph, from); //C++ function invoking CUDA
```

```
// in cu_sssp():
DeviceGraph devGraph(graph);
devGraph.copyToDevice();
...
kernels::BF_Kernel1<(a), false>
  <<< _Div(graph.V, (BLOCKDIM / (a)) * ITEM_PER_WARP),
    BLOCKDIM, SMem_Per_Block<char, BLOCKDIM>::value >>>
```

SubGI on large graphs and collections of graphs

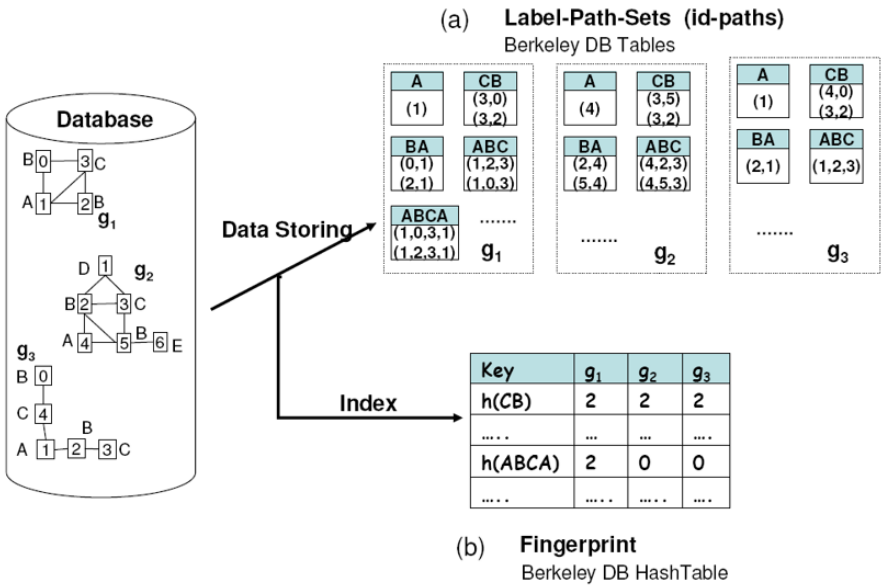
Approaches for subGI search are mainly divided by two aspects:

- Depending on the number of pattern and target graphs that are taken into account
 - One-to-one methodologies solve the problem of searching one single query into one single target graph
 - **One-to-many**: one pattern graph searched within a collection of many target graphs
 - Many-to-one: many patterns searched at once within a single target graph
 - Many-to-many: many pattern graphs searched at once in a collection of target graphs
- If they use or not indexing methodologies:
 - A non-indexed approach only rely on the performance of the searching process
 - **Index-based** approaches exploit graph-features for
 - **Filter graphs** within the collection of targets
 - such that the core subGI algorithm is run only on a selection of target graphs
 - **Filter vertices**, within a single target graph, that are unfeasible to match with the query vertices
 - Such that single vertices or regions (group of vertices) of the target graphs are excluded from the searching process
 - Several **types of features** (paths, rings, tress, subgraphs) can be extracted and stored in a wide variety of **data structures** determining the performance of a given approach

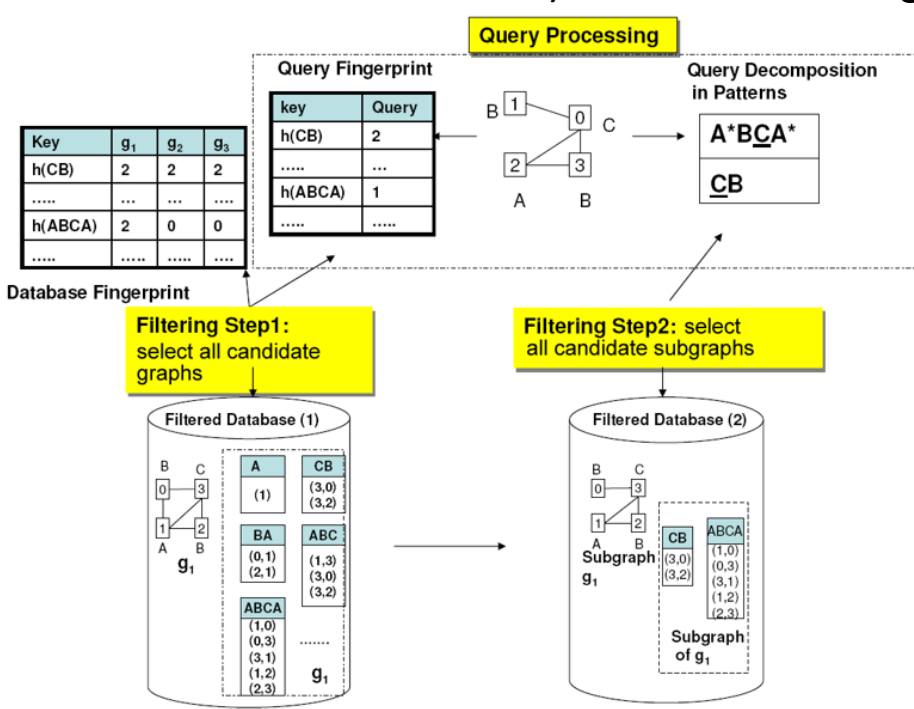


GraphGrep

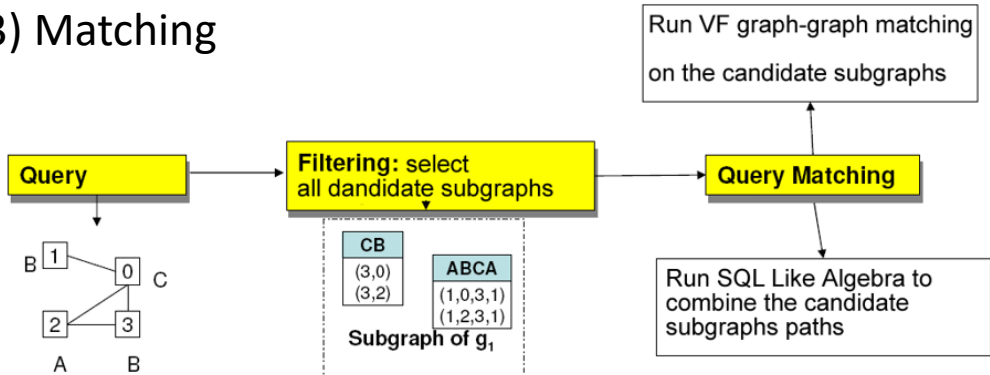
1) Index building



2) Candidate filtering

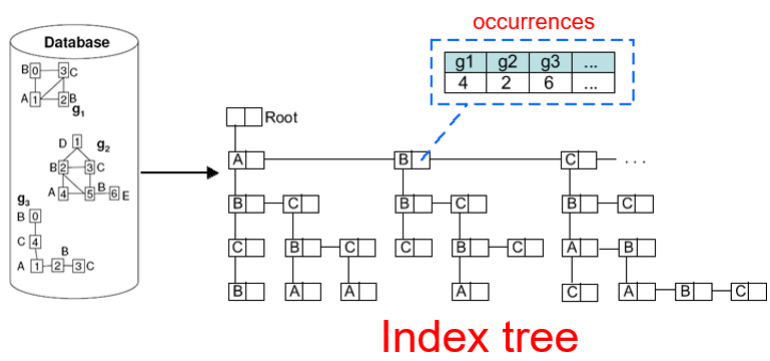


3) Matching

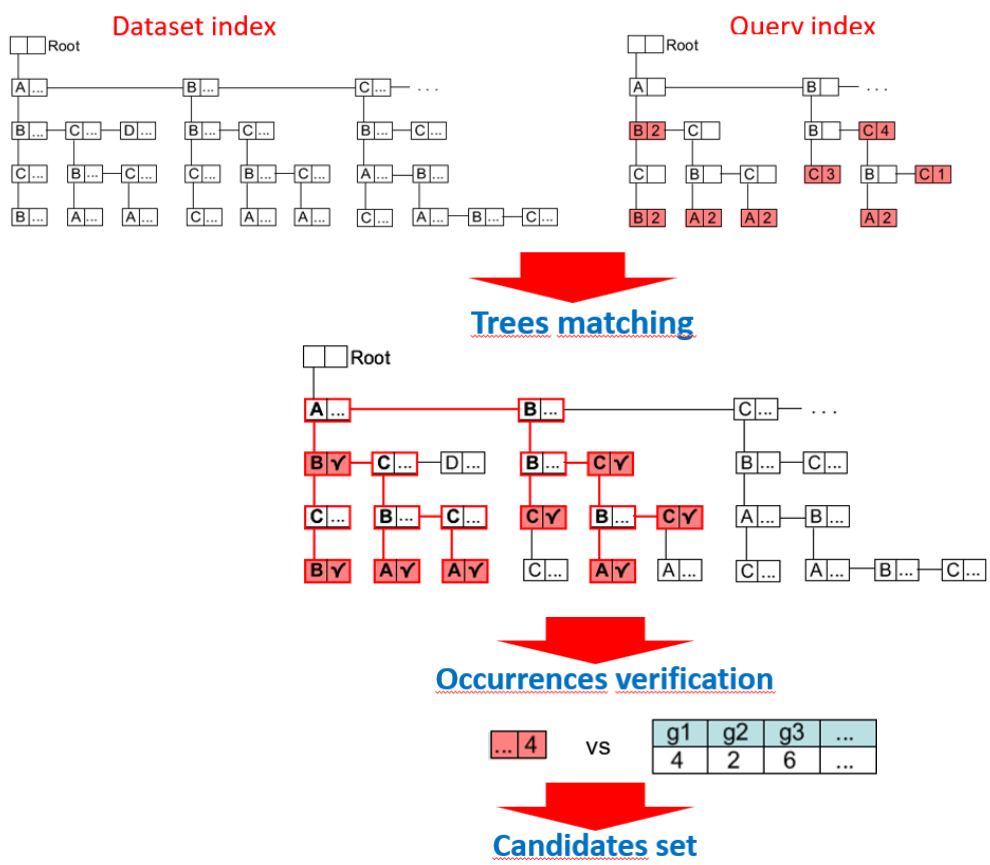


GraphGrep-SX

1) Index building



2)Candidate filtering



3) Matching

GraphGrep-SX

Table 2. Indexes size (Kb)

DB dim.	GraphGrepSX	GraphGrep	CTree	GCoding	SING
8000	3684	293992	13884	6687	8445
24000	11020	928912	41372	20088	22279
42000	18668	1577012	70208	30651	42830

Table 3. Filtering time (sec)

Query dim.	GraphGrepSX	GraphGrep	CTree	GCoding	SING
4	0.05	0.012	1.34	0.0042	0.51
8	0.05	0.006	1.57	0.01	0.76
16	0.041	0.005	1.51	0.026	0.17
32	0.04	0.014	1.01	0.059	0.071

Table 4. Query time (sec)

Query dim.	GraphGrepSX	GraphGrep	CTree	GCoding	SING
4	14.9	15.41	13.27	23.61	12.4
8	17.5	7.1	44.24	15.79	15.24
16	2.08	12.78	51.07	5.39	0.798
32	1.07	3.56	50.91	1.25	0.136

Table 5. Number of generated candidates

Query dim.	GraphGrepSX	GraphGrep	CTree	GCoding	SING
4	26865	29196	16704	16188	23170
8	21337	13920	5840	8567	14012
16	1629	7053	289	1648	214
32	142	3193	3	142	4

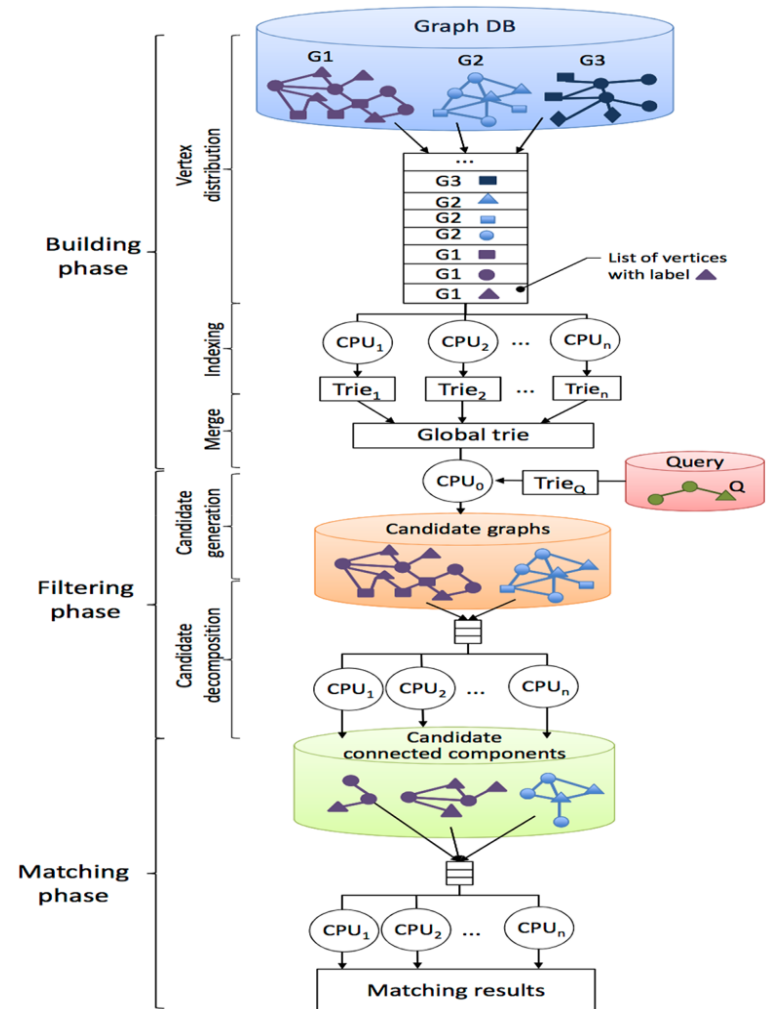
GRAPES – a (paths) index-based approach for one-to-one and one-to-many subGI

GRAPES is a system for searching one pattern graph within a collections of target graphs, as well as within A single target graph.

As its predecessors, GraphGrepSX and SING, it uses **labelled paths, up to a given length, and their multiplicities as graph features** for:

- discriminating **target graphs** which do not contains pattern paths
- discriminating **target vertices** that are not source of labelled paths present in the pattern

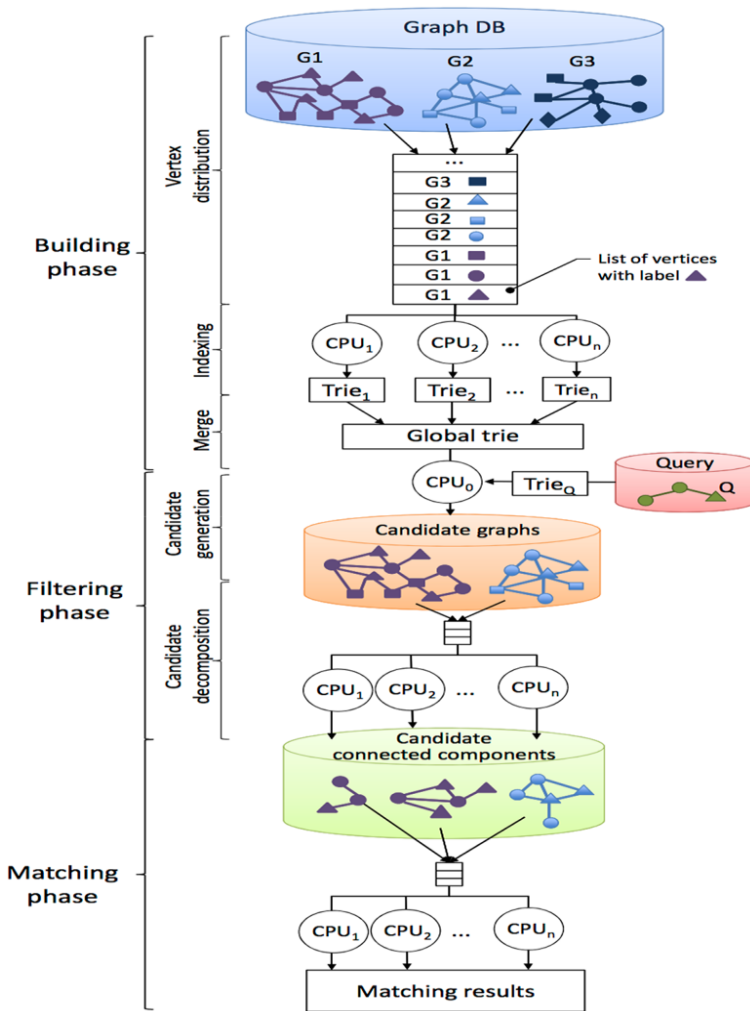
A **path** of **length** n is defined as a tuple of n vertices $(v_1, v_2, v_3, \dots, v_n)$ such that $(v_i, v_{i+1}) \in E$, and $v_i \neq v_j$ for $i, j \in [1, n]$, for $i \neq j$. v_1 is called the **starting node** of the path.



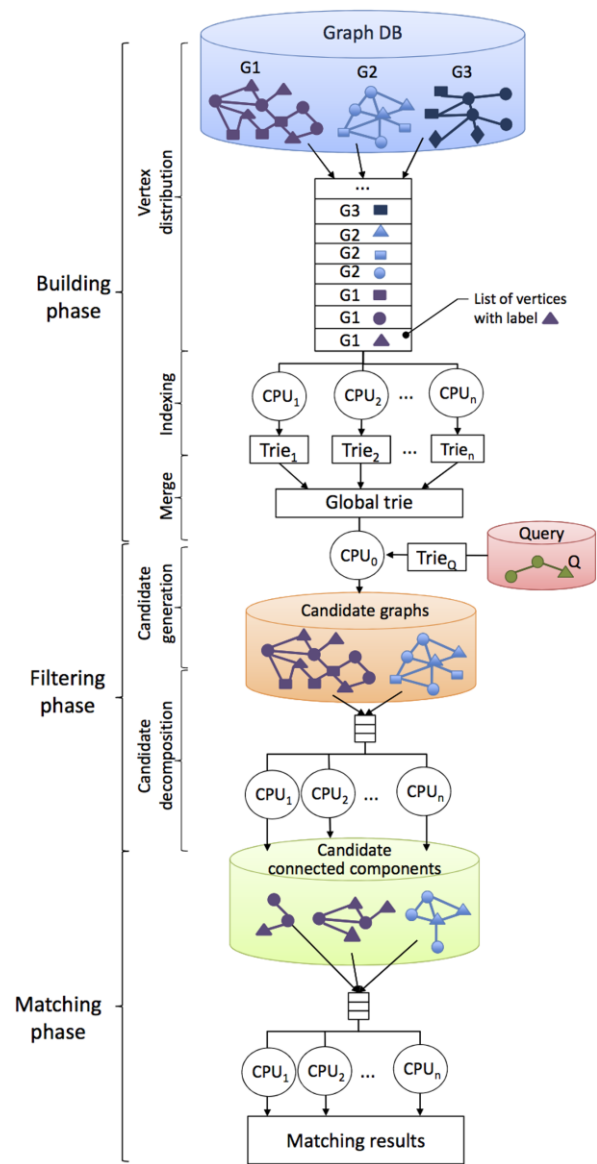
GRAPES – a (paths) index-based approach for one-to-one and one-to-many subGI

SubGI searches via **GRAPES** are performed in three phases:

- **Building phase:** labelled paths from target graphs are extracted and stored in a trie data structure
 - The index can be reused for multiple queries
- **Filtering phase:** given a pattern graph, labelled paths are extracted from the pattern graph and searched in the trie DS for filtering target graphs and target vertices
- **Matching phase:** a subGI core algorithm (VF2) is run only for the target graphs which passed the filtering phase by removing from them unfeasible vertices

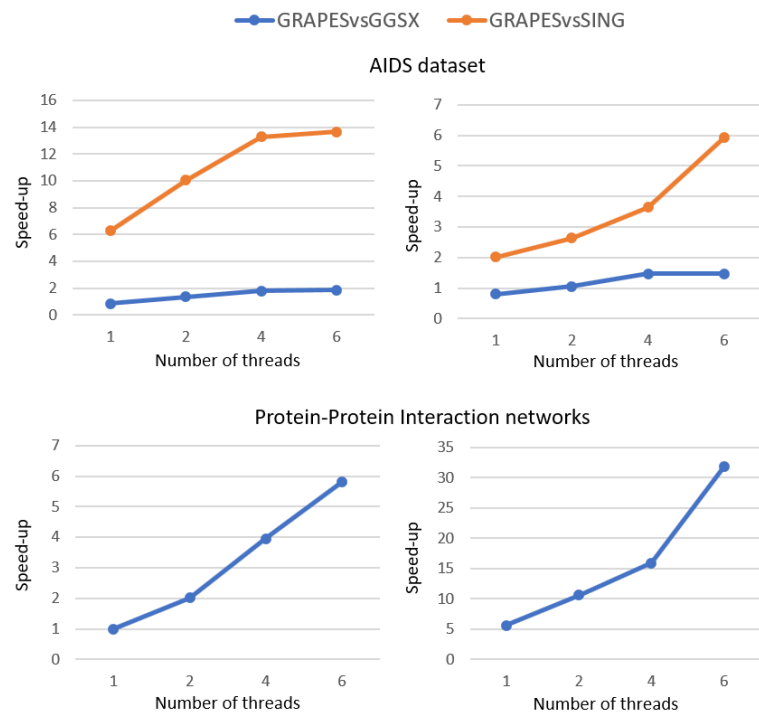


GRAPES – Parallel searching on biological graphs targeting multi-core architectures



	GRAPES	GraphGrepSX	SING	Description
TRIE indexing	x	x		Compact indexing. Efficiency on indexing and filtering
Path indexing	x	x	x	Lighter than subgraphs or trees
Store path node	x		x	More filtering power
Parallel Indexing and Matching	x			Necessary for biological data size and density
Connected components	x			Load balancing onto CPU cores

List of the new GRAPES components together with the components of GRAPES taken from GraphGrepSX [16] and SING [15].



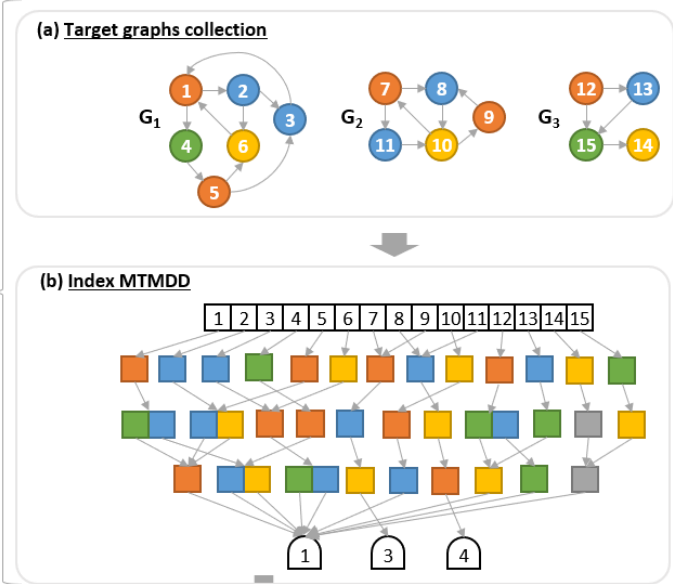
GRAPES-DD - Exploiting decision diagrams for index-based subGI

GRAPES-DD is based on GRAPES and it exploits **Multi-Terminal Multi-way Decision Diagram** as data structure for managing and storing feature paths.

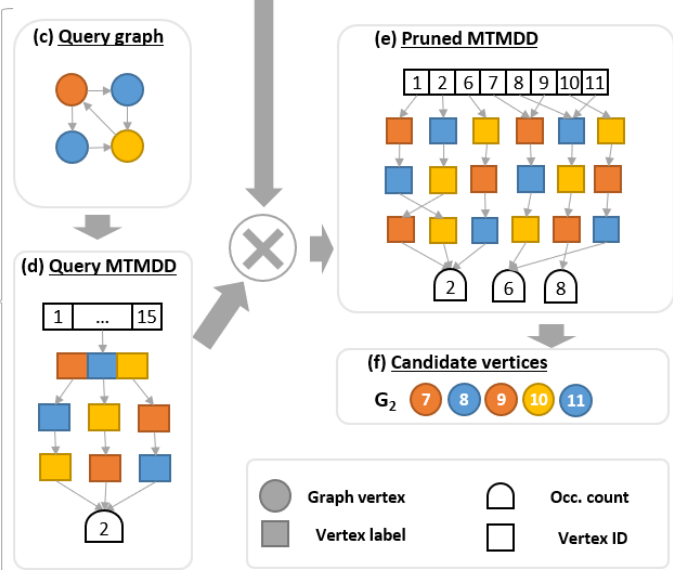
Given one or many target graphs, a MTMDD is built in order to store in a succinct ways all the labelled paths contained in the target graphs and their starting nodes.

Given a query, a MTMDD of the query is built and it is multiplied to the targets MTMDD in order to obtain the list of target vertices that are the starting point of labelled paths present in the query graph.

Indexing building phase



Filtering phase



Miner, A., & Parker, D. (2004). Symbolic representations and analysis of large probabilistic systems. In *Validation of Stochastic Systems* (pp. 296-338). Springer, Berlin, Heidelberg.

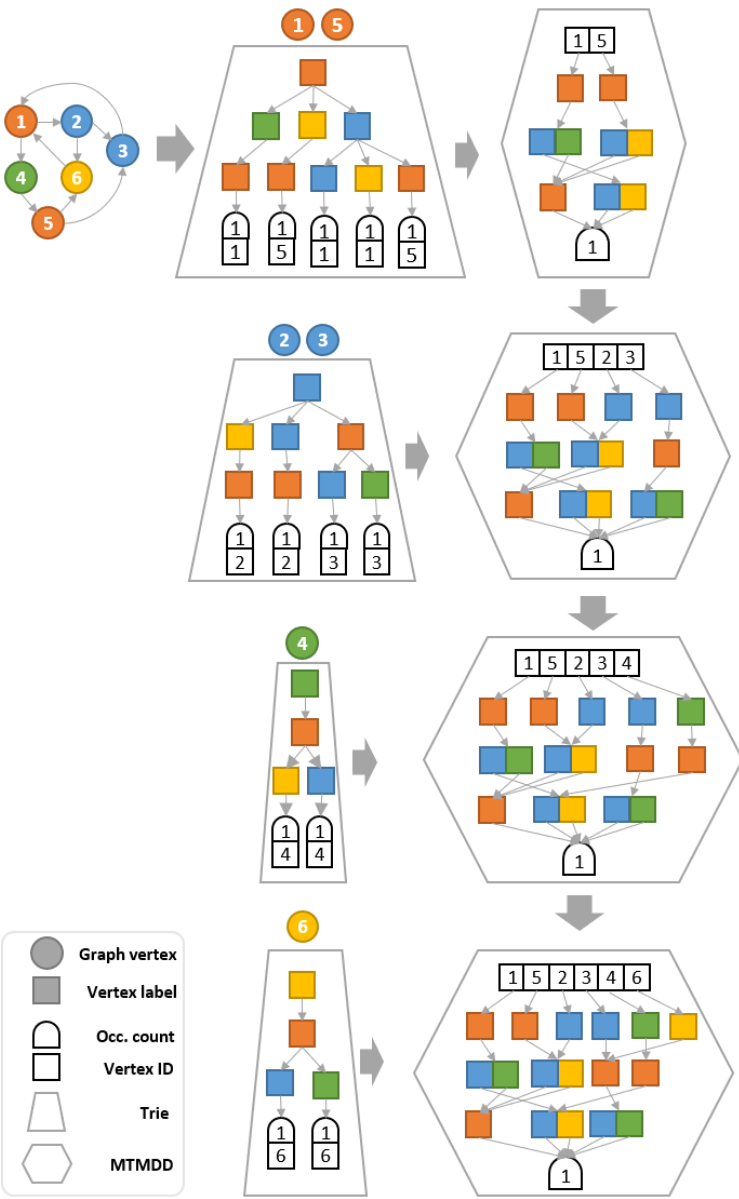
GRAPES-DD - Exploiting decision diagrams for index-based subGI

Given a graph, the **construction** of the MTMDD is performed in **successive steps**.
In each step, a **trie** containing all the labelled paths which starting node is labelled with a specific label.

Paths are extracted by running a Depth-First Visit form each vertices which label is the current scanned one.

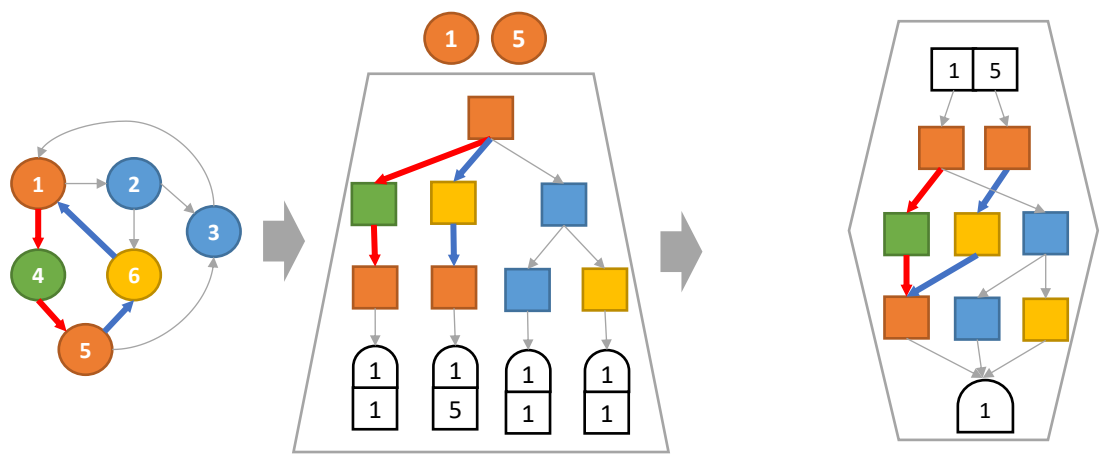
Then the trie is added up to the current **partial MTMDD**.

The **final MTMDD** is built when all the labels of the graphs have been scan for extracting the paths.
Target vertices within a collection of graphs are re-named such that their newly assigned **identifier** is unique within the collection.



GRAPES-DD - Exploiting decision diagrams for index-based subGI

- An MTMDD has **three types** of nodes:
- first-level nodes which represents IDs of **starting vertices**
 - intermediate-levels nodes which represent **labels** of the vertices composing the paths
 - leaf nodes which represents **multiplicities**



 Graph vertex

 Vertex label

 Occ. count

 Vertex ID

 Trie

 MTMDD

GRAPES-DD – Experimental setup

Synthetic graphs were generated by using

two different random models, by varying the models parameters

- **Barabasi-Albert** which are shown to capture some essential properties of biological graphs, such as the power-law degree distribution
- **Forest-Fire** model which emulate the diffusion of fire in a forest (it is not linked to biochemical/biological graphs)

Real **biochemical/biological graphs**

- **AIDS**: collection of 40k small molecules tested for the AIDS treatment
- **PDBS**: 600 (medium/sparse) topological structure of proteins
- **PCM**: 200 (small-medium/dense) graphs representing spatial proximity of protein amino acids
- **PPI**: 20 protein-protein interaction networks of 5 different species at different density

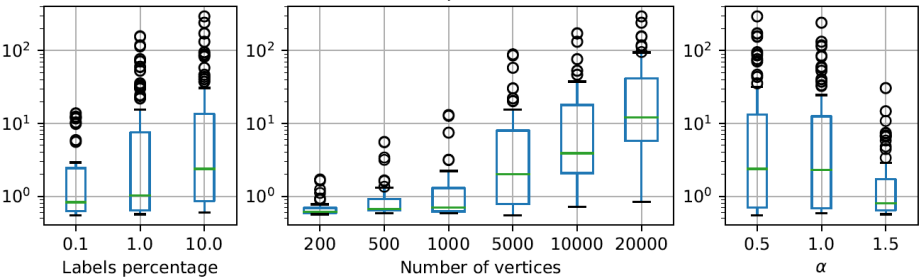
Pattern graphs, of different sizes, were extracted from target graphs and searched with the origin target graph or within the corresponding collection of target graphs.

GRAPES-DD – Results for synthetic graphs

			RAM req. (MB)			Storage req. (MB)			Build time (s)	
			GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES
Barabasi-A.	<i>l</i>	0.1%	3,649	7,935	2.2	305	3,493	11.5	470	109
		1%	8,229	66,838	8.1	1,543	28,772	18.6	646	214
		10%	7,876	81,552	10.4	10,071	34,368	3.4	668	265
	α	0.5	2,103	94,654	45.0	24,915	40,281	1.6	516	330
		1	8,351	58,519	7.0	16,602	25,145	1.5	766	213
		1.5	1,447	3,068	2.1	1,246	1,219	1.0	144	26
Forest-Fire	<i>l</i>	0.1%	834	3,929	4.7	121	1,689	13.9	63	29
		1%	1,308	21,255	16.2	738	9,229	12.5	77	66
		10%	1,167	24,936	21.4	5,351	10,650	2.0	62	70
	<i>p</i>	0.1	147	1,426	9.7	2,882	585	0.2	10	7
		0.3	188	2,451	13.1	3,358	1,024	0.3	14	9
		0.5	281	4,966	17.7	3,922	2,109	0.5	26	16
		0.7	487	11,694	24.0	4,535	5,020	1.1	57	37
		0.9	988	29,565	29.9	5,386	12,840	2.4	139	88

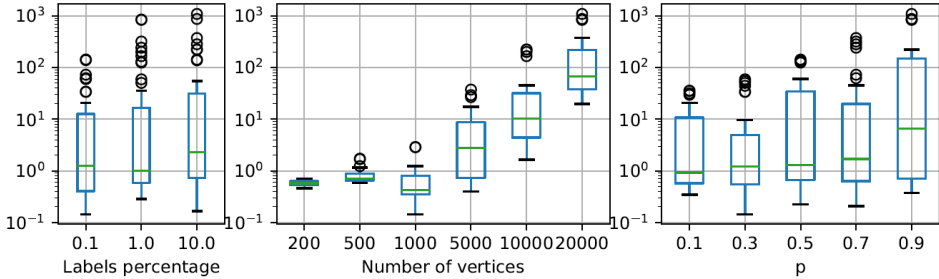
Barabasi-Albert graphs

RAM requirement ratio

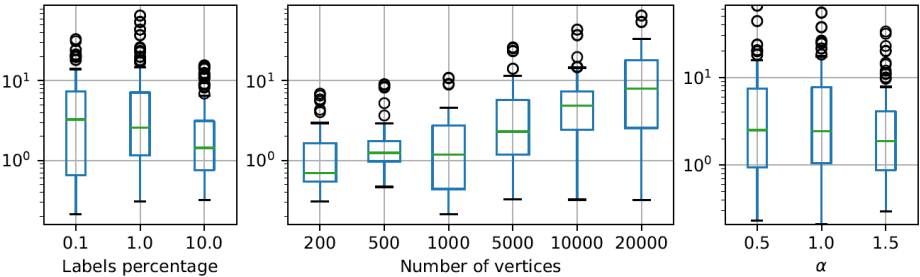


Forest-Fire graphs

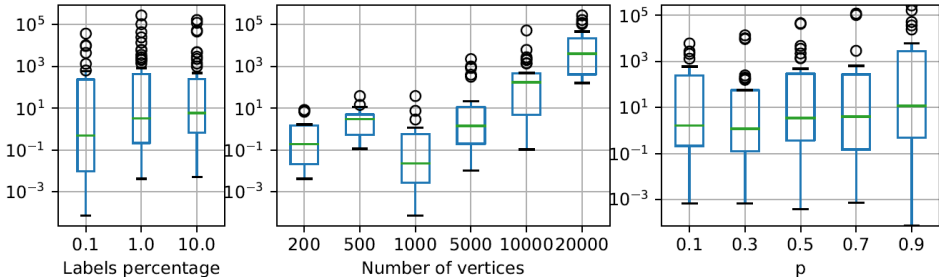
RAM requirement ratio



Storage requirement ratio



Storage requirement ratio



GRAPES-DD – Results on biological graphs

Collections of biochemical/biological graphs

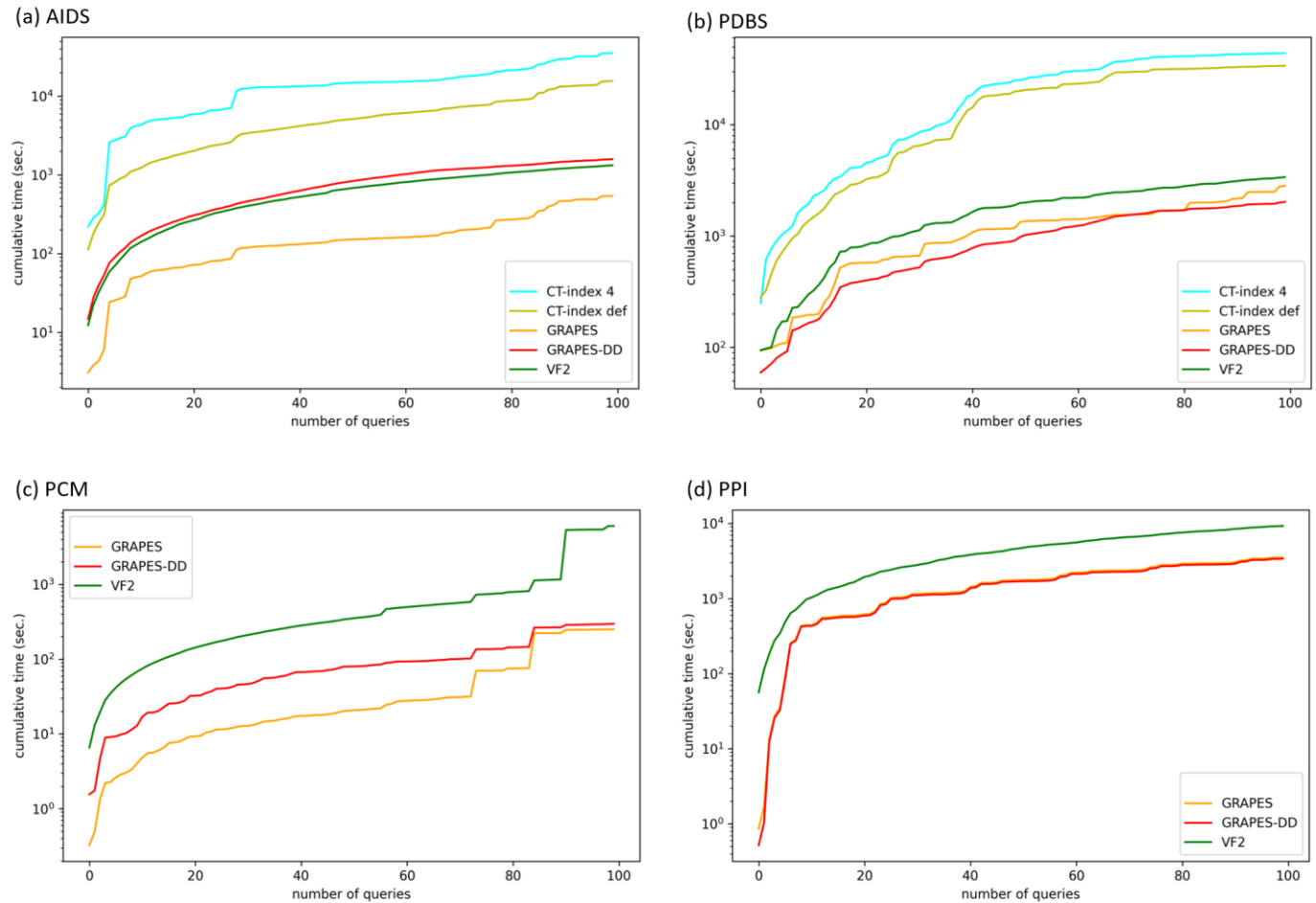
	RAM req. (MB)			Storage req. (MB)			Build time (s)	
	GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES
<i>AIDS</i>	5,304	1,064	0.20	164	39	0.24	170.12	16
<i>PDBS</i>	532	556	1.04	22	17	0.78	176.00	10.07
<i>PCM</i>	512	7,057	13.77	253	1,392	5.51	617.24	754.56
<i>PPI</i>	629	1,698	2.70	166	665	4.00	2,514.18	2,906.65

Single PPI networks

Species	V	E	RAM req. (MB)			Storage req. (MB)			Build time (s)	
			GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES	ratio	GRAPES-DD	GRAPES
S. cerevisiae	4,709	40,284	38.1	91.1	2.39	10.5	26.2	2.49	60.93	63.49
	5,230	53,699	56.3	136.3	2.42	11.7	42.4	3.63	604.47	630.92
	5,762	76,482	61.1	150.9	2.47	12.7	47.5	3.73	846.24	879.90
	5,936	89,674	46.5	121.2	2.61	12.9	36.8	2.86	128.52	135.01
C. elegans	1,557	2,472	7.1	5.7	0.80	0.2	0.3	1.45	0.24	0.20
	2,421	3,981	7.8	7.9	1.02	0.5	1.0	2.06	0.52	0.44
	3,664	7,005	10.4	14.6	1.40	1.0	2.8	2.65	1.33	1.13
	6,173	26,184	25.1	58.5	2.33	4.3	16.0	3.75	34.16	34.19
D. melanogaster	1,185	2,008	8.0	12.1	1.51	0.6	1.5	2.30	0.31	0.26
	2,488	6,151	12.1	32.1	2.65	1.9	6.6	3.47	3.28	3.21
	2,729	7,235	13.3	36.2	2.73	2.3	7.9	3.37	4.30	4.22
	7,928	37,542	52.3	198.7	3.80	13.1	64.3	4.90	144.05	156.07
M. musculus	1,810	2,413	8.0	13.1	1.64	0.7	2.2	2.94	0.42	0.36
	3,255	5,424	11.0	31.0	2.81	1.9	7.1	3.68	2.52	2.50
	3,758	6,853	13.1	43.6	3.33	2.6	11.2	4.30	4.47	4.61
	6,875	23,779	41.1	193.6	4.71	12.4	62.1	5.02	76.64	81.56
H. sapiens	4,638	10,665	17.3	55.0	3.18	3.6	14.6	4.07	5.60	5.36
	8,728	31,164	53.5	215.4	4.02	13.1	70.8	5.42	65.14	68.26
	9,826	48,835	87.5	351.4	4.02	21.5	120.1	5.59	213.95	230.16
	10,186	51,484	89.2	391.8	4.39	22.0	134.1	6.10	191.63	209.63

GRAPES-DD – Results on biological graphs

Querying 100 patterns on the collections of biochemical/biological graphs



GRAPES-DD – Open problems

Open problems

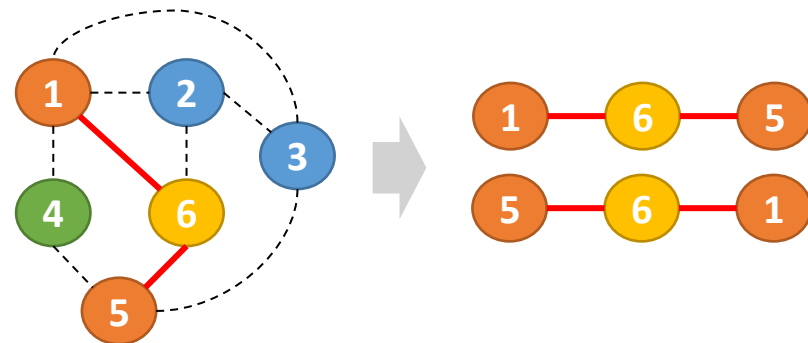
Paths of limited length are used, however longer paths will be more effective.

The limitation is mainly due to an enumerative problem regarding path extraction:

we extract each undirected path twice

Question:

Is it possible to enumerate all labelled paths (up to a given length) of a graph without such a redundancy?



3.3 Approximate subgraph isomorphism

Given a graph $G = (V, E)$ and a vertex labeling function $\alpha : V \mapsto A = a_1, a_2, \dots, a_m$, we define the label-counting vector of G as a function $L_G : A \mapsto \mathbb{N}$ such that $L(a_i \in A) = |\{v \in V : \alpha(v) = a_i\}|$. Given two label-counting vectors L^1 and L^2 , the distance between the two vectors is defined as $\delta(L^1, L^2) = \sum_{a_i \in A} L^1(a_i) - L^2(a_i)$.

Given a query graph G_q and a target graph G_t , and let $G'_t = (V'_t, E'_t)$ to be a subgraph of G_t , such that $|V'_t| = |V_q|$ and $E'_t = \{(u, v) \in E_t : u, v \in V'_t\}$, mapped to vertices of G_q via a mapping function $\widetilde{M}$. The label cost of $\widetilde{M}$ is defined as $\delta(L_{G_q}, L_{G'_t})$. The edge cost of $\widetilde{M}$ is defined as $\lambda(G_q, G'_t, \widetilde{M}) = |\{(u, v) \in E_t : (\widetilde{M}(u), \widetilde{M}(v)) \notin E'_t\}|$.

2

RIBS introduces a procedure for enumerating all the non-redundant paths having certain lables.

Such paths are in according with the labels of the query graph and represent the backbones of the approximate matches.

Definition 3.1 (*Labelling-preserving SubGI (LP-SubGI) problem*).
 Given the query graph undirected and node labeled $G_q = (V_q, E_q, L_q)$, with $V_q = \{q_1, q_2, \dots, q_n\}$ and $E_q = \{(q_1, q_2), (q_2, q_3), \dots, (q_{n-1}, q_n)\}$ and the label set L_q . LP-SubGI is looking for set of induced relaxed SubGI subgraphs Q_r , where each solution $G_t^c = (V_t^c, E_t^c) \in Q_r$ is a subgraph of $G_t = (V_t, E_t, L_t)$ meet the following criteria:

- **Node Mapping with Label Preservation(M)** : For each $t_i \in V_t^c \exists q_i \in V_q$ such that $t_i = \mathcal{M}(q_i)$ and $\alpha(t_i) = \alpha(q_i)$.
- **Label Set count:** $\delta(L^{G_q}, G_t^c) = 0$, the number of the labels in the solution subgraph is the same as the query graph.
- **Injectivity of Mapping:** If two nodes $q_i \neq q_j$ differ then also the mapping should differ

$$t_i = \mathcal{M}(q_i) \neq t_j = \mathcal{M}(q_j)$$

- **Relaxed mapping edge preserving:** this means that the number of arcs in the solution subgraph G_t^c may differ from the number of the query arcs by at most a factor γ :

$$-\gamma \leq |E_t^c| - |E_q| \leq \gamma$$

- *il sottografo trovato è una component connessa, ma non necessariamente fortemente connessa*

Fifteen years of research on the Subgraph Isomorphism Problem

Thanks

Prof. Rosalba Giugno	Dr. Federico Busato
Prof. Alfredo Pulvirenti	Dr. Giovanni Micale
Prof. Alfredo Ferro	Dr. Antonio Di Maria
Prof. Dennis Shasha	Dr. Roberto Grasso
Prof. Nicola Bombieri	Dr. Nicola Licheri
Prof. Marco Beccuti	Dr. Stefano Aldegheri
Dr. Donatella Granata	Dr. Rosario Di Stefano
	Dr. Simone Caligola
	Dr. Antonino Aparo
	Maria Stefania Scaella
	Andrea Agnetti